What does waiting cost?

The pick6 draft engine: survival, urgency, and one honest backtest

Written up for the repository `github.com/trisslazaj/pick6`

July 2026

Abstract

A fantasy football snake draft asks one question over and over: the player in front of you is good, but so is the next pick you will get — what does it cost you to wait? This paper is the complete write-up of *pick6*, a terminal draft assistant that answers that question numerically and shows its working. The engine models each player's availability as a logistic survival curve through the market's average draft position, conditions it on the fact that he is demonstrably still on the board right now, and corrects the resulting probabilities with a single scalar so that the number of players the model expects to disappear equals the number of picks that will actually happen. From those probabilities it computes the expected value of the best player who will still be there when your turn comes, and calls the difference *urgency*.

Every piece of that pipeline is derived from a definition rather than asserted, and every piece is graded. We backtest against three real completed 12-team half-PPR drafts using era-correct average draft position, producing 15,923 labelled predictions over 168 vantages on the 2024 fold and roughly 52,000 on each 2025 fold. On the 2024 fold — the only one with a per-player market spread, and so the only one that can grade the whole model — the shipped model scores a Brier of 0.0670 and a log-loss of 0.2250 against 0.1022 / 0.3580 for a constant predictor and 0.1111 / 0.5159 for the unconditional logistic. Decomposing the Brier score shows where the gain came from: the reliability term falls from 0.0231 to 0.0051 while resolution barely moves, which is to say the work bought *calibration*, not *discrimination*.

Three ideas were built, measured, and did not survive, and they get a section of their own, because the reasons they failed are more interesting than the reasons the survivors worked. One shipped feature — the room-warped price of §10 — has had its supporting evidence retracted in full: it did not replicate on the second and third folds, and the fold it was chosen on turned out to be priced off a curve built from drafts that happened a year later. It still ships, now on a structural argument rather than a score, and §16 says so. The thinness of this evidence is stated plainly, repeatedly, and in the tool's own output.

Contents

1 Introduction: the decision problem

1.1 The situation

You are sitting at pick 27 of a twelve-team snake draft. Your next pick after this one is pick 46. Eighteen players will come off the board in between — picks 28 through 45, made by nine other managers picking twice each — and you have about ninety seconds.

In front of you are perhaps a hundred and fifty players with something to recommend them. The best receiver available is a little better than the best running back available. But there are sixty more receivers behind him and only a handful of running backs, and three of the last six picks were running backs, and the tier of backs you have been eyeing has two players left in it.

The question is not "who is the best player available". Your eyes can answer that. The question is:

> *Which of these players will still be here in eighteen picks, and what is the best one I could plausibly be choosing from then?*

Because if the answer is "the receiver will still be here and the running back will not", you take the running back, even though the receiver grades out higher. The cost of waiting is not the same for the two positions, and that asymmetry — not raw player quality — is the thing a draft is actually won and lost on.

1.2 What the engine computes, in one paragraph

Everything in pick6 is an answer to the question above. For each available player it estimates $\tilde{p}_j$, the probability he is still on the board at your next pick. For each position it computes $\mathbb{E}$[best available at my next pick] from those probabilities. It subtracts that from the value of the best player at the position *now*, multiplies by how badly your roster needs that position, and calls the result *urgency*. The board sorts by urgency. The top group is the pick. Everything else in the tool — the tier-cliff alarms, the run banner, the two-pick plan, the value-over-replacement tie-break — is either an input to that number or a different presentation of it.

1.3 What this document is for

The engine was specified before it was built, and the specification asserted rather more than it derived. This document is the derivation. It has three jobs:

1. **Derive, don't assert.** Where a formula in the code drops out of a definition, the two lines that get you there are printed here. Where it does not — where the project made a judgement call, picked a constant by feel, or accepted a simplification — that is said out loud in the same voice.
2. **Introduce every symbol before using it.** A logistic distribution, an order statistic, a hazard rate, empirical Bayes, a proper scoring rule: each of these is defined at the point where the engine first needs it, in terms of the football, not in terms of the next abstraction up.
3. **Map the maths to the code.** Section 18 is a table from every numbered equation in this paper to the Go function that implements it. If you only read one page, read that one with the repository open beside it.

The reader we have in mind is a strong programmer and a serious fantasy player who likes mathematics but has not formally studied probability theory. Comfort with algebra and with reading code is assumed. Measure theory is not; neither is any prior exposure to survival analysis, Bayesian statistics, or forecast verification.

1.4 The one caveat that governs everything

The headline result — that blending this specific league's own draft history into the price curve improves calibration — rests on **one draft**. There is exactly one season for which we can obtain both a completed draft from this league and the average draft position that the market was showing at the time. Twelve seats scored over the same 180 picks multiplies the *count* of predictions by twelve; it does not multiply the *evidence*, because all twelve seats are scoring the same real picks. Section 16 says this at length. The tool says it too, on every run.

1.5 Data provenance

Nothing in pick6 is a projection of our own. Every number describing a player is imported:

- **Average draft position, and its spread.** Fantasy Football Calculator's free API, format `half-ppr`, twelve teams. Each row carries `adp`, `stdev`, `times_drafted`, `high`, `low` and `bye`. The 2026 snapshot used throughout is 203 players over 1,187 drafts (window 2026-07-24 – 2026-07-29); the 2024 snapshot used for the backtest is 178 players over 906 drafts (window 2024-08-31 – 2024-09-01). Attribution is requested by the source and given here and in the tool.
- **Player values.** FantasyCalc's `redraftValue`, plus its cross-source identifier crosswalk. Covers QB/RB/WR/TE only.
- **Player identity, injury status, and league history.** Sleeper's unauthenticated API: the full player dump, and this user's own completed drafts.
- **Tiers.** A hand-transcribed rankings CSV where one covers a player; derived from value gaps otherwise.

FFC's API is free for personal and commercial use and asks only for attribution, which is given here, in the tool's footer, and in the repository README. The project takes no credit for any player valuation in it.

Transformations of imported values are fair game. Inventing a player value is not, and the one place where that temptation is strongest — kickers and defenses, which no source values at all — is handled in §11.3 by anchoring onto the imported curve rather than by making a number up.

2 Notation and the draft

2.1 Players and values

Let $\mathcal{P}$ be the set of players on the board. Each $j \in \mathcal{P}$ carries:

$v_j \in \mathbb{R}_{\geq 0}$	*value*: the single number everything compares. Imported.
$a_j \in \mathbb{R}_{>0}$	*average draft position* (ADP): the market's average price, in picks.
$\varsigma_j > 0$	the observed standard deviation of his draft position.
$\sigma_j > 0$	the *scale* of his survival curve, derived from ς_j in §3.3.
$\mathsf{pos}(j)$	his position: one of QB, RB, WR, TE, K, DEF.
$t_j \in \mathbb{Z}_{\geq 0}$	his *tier* within his position; 0 means "untiered".

Values are only ever compared by *difference*, so their absolute scale is irrelevant — but it must be consistent. On the shipped 2026 board the imported value curve runs from 10,457 at the top to single digits at the bottom, and it is convex: value falls fast at the top of the board and slowly at the bottom, which is what makes a gap between two adjacent players mean something. A linear rank would make every gap identical and destroy the entire urgency signal.

2.2 The snake

A draft has T teams (*seats*, or *slots*) and R rounds, so RT overall picks numbered $1, \dots, RT$. Everything is 1-indexed, including the Go code, which carries one -1 at each array access rather than an off-by-one in the arithmetic.

Definition 2.1 (Round and index)**.** For overall pick $p \in \{1, \dots, RT\}$,

$$r(p) = \left\lfloor \frac{p-1}{T} \right\rfloor + 1 \qquad \text{the round containing } p, \tag{1}$$

$$i(p) = \big((p-1) \bmod T\big) + 1 \qquad \text{the position of } p \text{ within its round.} \tag{2}$$

The draft order is a permutation $\omega : \{1, \dots, T\} \to \{1, \dots, T\}$, read from Sleeper's `draft_order` field; $\omega(i)$ is the seat that picks i-th in an odd round. In a *snake* draft the order reverses every other round:

Definition 2.2 (Seat map)**.**

$$\mathrm{slot}(p) = \begin{cases} \omega\big(i(p)\big) & r(p) \text{ odd}, \\ \omega\big(T + 1 - i(p)\big) & r(p) \text{ even}. \end{cases} \tag{3}$$

Two facts are worth proving, not because they are hard but because getting either one wrong corrupts every probability downstream and does so silently. A board that mis-attributes picks still draws.

Proposition 2.3 (Every pick belongs to exactly one seat)**.** *The map $p \mapsto (r(p), i(p))$ is a bijection from $\{1, \dots, RT\}$ onto $\{1, \dots, R\} \times \{1, \dots, T\}$, and for each fixed round the seat map $i \mapsto$ slot of (3) is a bijection onto $\{1, \dots, T\}$. Consequently $\mathrm{slot}(\cdot)$ assigns exactly one seat to every pick.*

Proof. Write $p-1 = qT + m$ with $0 \le m < T$; the division algorithm says this decomposition exists and is unique, and $r(p) = q+1$, $i(p) = m+1$. Ranging p over $1, \dots, RT$ ranges q over $0, \dots, R-1$ and m over $0, \dots, T-1$ exactly once each, which is the first claim.

For the second: in an odd round the seat map is ω itself, a permutation. In an even round it is $\omega \circ \rho$ where $\rho(i) = T + 1 - i$. The map ρ is an involution on $\{1, \dots, T\}$ (it is its own inverse, since $T + 1 - (T + 1 - i) = i$), hence a bijection, and a composition of bijections is a bijection. □

Corollary 2.4 (One pick per seat per round)**.** *In each round, each of the T seats picks exactly once.*

Proof. Immediate from the second half of Proposition 2.3: within a fixed round, the T values of i map onto the T seats bijectively. □

2.3 My pick sequence

Let $s \in \{1, \dots, T\}$ be *my seat position* — the index at which my seat appears in ω, so $\omega(s) = \text{mySlot}$.

Proposition 2.5 (My pick in round r)**.**

$$\text{myPick}(r) = \begin{cases} (r-1)T + s & r \text{ odd}, \\ (r-1)T + (T - s + 1) & r \text{ even}. \end{cases} \tag{4}$$

Proof. In an odd round I pick when $\omega(i) = \omega(s)$, i.e. when $i = s$; substituting into $p = (r-1)T + i$ gives the first line. In an even round I pick when $\omega(T + 1 - i) = \omega(s)$, i.e. $T + 1 - i = s$, i.e. $i = T - s + 1$, which gives the second. □

Definition 2.6 (The horizon)**.** Let p_{now} be the current overall pick — the next one to be made. Then

$$q_1 = \min\{\, \text{myPick}(r) : \text{myPick}(r) \ge p_{\text{now}} \,\} \qquad \text{my next pick,} \tag{5}$$

$$q_2 = \min\{\, \text{myPick}(r) : \text{myPick}(r) > q_1 \,\} \qquad \text{the one after that,} \tag{6}$$

$$N = q_1 - p_{\text{now}} \qquad \text{picks until mine.} \tag{7}$$

$q_2 = 0$ by convention when the draft ends first. The *intervening set* is $\{p_{\text{now}}, \dots, q_1 - 1\}$, the N picks that can remove a player before I act.

If it is already my turn then $q_1 = p_{\text{now}}$, $N = 0$, and the intervening set is empty. This is not a degenerate case to be special-cased — it is the correct answer, and the whole engine collapses gracefully onto it: nobody can be taken in zero picks, so every survival probability is exactly 1, so urgency is exactly 0 everywhere, and the board falls back to a value ordering. That is what you want on the frame where you are actually choosing.

2.4 The turn

A small consequence worth deriving, because it is the geometry that makes drafting from the edge of the snake feel different from drafting from the middle.

Corollary 2.7 (The gap at the turn)**.** *For odd r,* $\text{myPick}(r) + \text{myPick}(r+1) = 2rT + 1$*, so the gap between my two picks around the turn is*

$$\text{myPick}(r+1) - \text{myPick}(r) = 2(T - s) + 1,$$

while the gap from an even round into the next odd round is $2s - 1$.

Proof. For r odd, $\text{myPick}(r) = (r-1)T + s$ and $\text{myPick}(r+1) = rT + (T - s + 1)$ by (4). Adding gives $(2r-1)T + T + 1 = 2rT + 1$; subtracting gives $rT + T - s + 1 - (r-1)T - s = 2T - 2s + 1$. The even-to-odd case is the same computation with the two lines of (4) swapped: $rT + s$ minus $(r-1)T + T - s + 1$ is $2s - 1$. □

Concretely, at $T = 12$ from seat $s = 3$: the round-1 pick is 3, the round-2 pick is 22 and the round-3 pick is 27, so the gaps are 19 and then 5. The two gaps sum to $2T = 24$ every time.

Do not confuse a gap with N. The gap is $\text{myPick}(r+1) - \text{myPick}(r)$ and counts my own pick at one end; N of (7) counts only the picks that can still remove somebody. Standing immediately after picking at 3 we have $p_{\text{now}} = 4$ and $q_1 = 22$, so $N = 18$ (picks 4 through 21); after picking at 22, $N = 4$. Gaps 19 and 5; horizons 18 and 4. `PicksUntilMine` prints the second pair, and the exactly-N tilt of §4 is solved against it.

Seat 3 lives with long waits followed by back-to-back picks; seat 6 waits about twelve picks every time. The engine does not need to know this — it falls out of (4) — but it explains why the two-pick lookahead of §8 matters far more at the edges of the snake than in the middle.

2.5 A note on letters

Two dozen quantities and twenty-six letters; a few get reused, so here is the whole list of collisions up front rather than as a surprise later.

- p is a *pick number* in §2–§3 and a *probability* from §4 onward (p_j, $\tilde{p}_j$, $\hat{p}$). Pick numbers always come with a subscript that is a word (p_{now}) or no subscript at all; probabilities always carry a player index or a hat.
- R is the number of *rounds*, always. My remaining picks are L (§6) and the rounds still to play are R_{rem}.
- s is my *seat*, always. The logistic's scale parameter is σ (not s, as most references write it), dedicated lineup slots are ℓ_P, and the scoring rules of §13 are $\mathcal{S}$.
- Lower-case n is always a *count of something*: n_j drafts behind a player's spread, n_0 a pseudo-count, n_k drafts reaching depth k, n_b predictions in a reliability bin, bare n the total prediction count in §13. Capital N is picks-until-mine and nothing else.
- D_j is the random pick at which player j comes off the board; $\text{dem}(P)$, spelled out, is a position's draft demand.

2.6 State

The engine is a set of pure functions over one struct. It holds the players, a taken set, every seat's roster so far, my seat, T, R, p_{now}, ω, and the league's starting-lineup shape. It recomputes everything from scratch on every pick event. This is a few thousand floating-point operations and a handful of bisections; there is no caching anywhere, deliberately, because a cache is a place for two frames of the board to disagree.

3 The survival model

3.1 What we are trying to estimate

Fix a player j and a pick number p. Define the event

$$A_j(p) = \{\text{player } j \text{ is still undrafted immediately before pick } p\}.$$

We want $\Pr[A_j(q_1) \mid A_j(p_{\text{now}})]$: given that he is sitting there right now, what is the chance he is still sitting there when my turn comes?

Everything in this section is about turning two numbers the market gives us — an average price a_j and a spread ς_j — into that conditional probability, without pretending to know more than those two numbers support.

3.2 The price curve: why a logistic

Definition 3.1 (Logistic distribution)**.** A random variable X has a *logistic distribution* with location μ and scale $\sigma > 0$ when its cumulative distribution function is

$$F(x) = \Pr[X \le x] = \frac{1}{1 + e^{-(x-\mu)/\sigma}}. \tag{8}$$

It is a symmetric, bell-shaped distribution — it looks like a normal distribution to the eye — with heavier tails and, crucially, a CDF in closed form.

Model the pick at which player j comes off the board as a logistic random variable D_j with location a_j (his ADP — the market's average price is the centre of the distribution) and scale σ_j. Then his *survival function*, the probability he is still available at pick p, is

$$S_j(p) \;=\; \Pr[D_j \ge p] \;=\; 1 - F(p) \;=\; \frac{1}{1 + e^{(p-a_j)/\sigma_j}}. \tag{9}$$

Read it at the table. At $p = a_j$ the exponent is zero and $S_j = 1/2$: a player whose ADP is exactly your next pick is a coin flip, which is what an average price *means*. Well before his ADP the exponent is very negative and $S_j \approx 1$. Well after, $S_j \approx 0$.

Three reasons for the logistic over the obvious alternative, a normal (Gaussian):

1. **Closed form.** The normal CDF has no elementary expression; you need `erf` and a numerical approximation. (9) is one `exp` and one division, and — see §3.6 — its logarithm is a function we can evaluate stably at any magnitude. The engine calls this a few thousand times per frame and needs it to be boring.
2. **Slightly fatter tails.** Draft position is not Gaussian: a player occasionally goes twenty picks early because one manager loves him. Excess kurtosis 1.2 against the normal's 0 is not a heavy-tailed model by any serious standard, but it is the right direction, and it is free.
3. **The scale parameter has a clean physical reading.** This is the real reason, and §3.5 makes it precise: deep in the tail, σ_j becomes a *per-pick hazard* — the chance that each passing pick is the one that takes him. A parameter that means something at the table is worth more than a marginally better distributional fit.

We are not claiming draft position is really logistic. We are claiming that a two-parameter symmetric curve through the market's mean and spread is the most a mean and a spread can honestly support, and that of the available two-parameter curves this is the most convenient.

3.3 From the market's spread to the curve's scale

The source gives us ς_j, the *standard deviation* of the pick at which j was taken across its sample of real drafts. The curve wants σ_j, the scale parameter of (9). These are not the same number, and the conversion is exact.

Proposition 3.2 (Variance of a logistic)**.** *If X is logistic with scale σ, then* $\mathrm{Var}(X) = \sigma^2\pi^2/3$*; hence* $\mathrm{sd}(X) = \sigma\pi/\sqrt{3}$ *and*

$$\sigma \;=\; \mathrm{sd}(X)\cdot\frac{\sqrt{3}}{\pi} \;=\; \frac{\mathrm{sd}(X)}{1.8138\ldots}. \tag{10}$$

Proof. Scale first: if X is logistic with location μ and scale σ then $(X-\mu)/\sigma$ is standard logistic ($\mu=0$, $\sigma=1$), directly from (8). Variance scales by σ^2 and is unaffected by location, so it suffices to show the standard logistic has variance $\pi^2/3$.

The standard logistic's moment generating function is a classical result,

$$M(t) = \mathbb{E}\big[e^{tX}\big] = \Gamma(1-t)\,\Gamma(1+t) = \frac{\pi t}{\sin \pi t}, \qquad |t|<1, \tag{11}$$

the second equality being Euler's reflection formula. Take logarithms to get the cumulant generating function $K(t)=\log M(t)$, whose Taylor coefficients are the cumulants; in particular the variance is $2\times$ the coefficient of t^2, since $K(t) = \mathbb{E}[X]\,t + \tfrac{1}{2}\,\mathrm{Var}(X)\,t^2 + O(t^3)$.

Now do the algebra. Using $\sin u = u\,\big(1-u^2/6+O(u^4)\big)$ with $u=\pi t$,

$$K(t) = \log(\pi t) - \log\sin(\pi t) = \log(\pi t) - \log\Big(\pi t\big(1-\tfrac{(\pi t)^2}{6}+O(t^4)\big)\Big) = -\log\Big(1-\tfrac{(\pi t)^2}{6}+O(t^4)\Big).$$

Since $\log(1-x) = -x + O(x^2)$,

$$K(t) = \frac{\pi^2 t^2}{6} + O(t^4) = \frac{1}{2}\left(\frac{\pi^2}{3}\right)t^2 + O(t^4),$$

so $\mathrm{Var}(X) = \pi^2/3$, and $\mathbb{E}[X]=0$ falls out of the missing t^1 term (as it must, by symmetry). □

So the engine sets

$$\sigma_j \;=\; \mathrm{clamp}\Big(\frac{\varsigma_j}{1.8138},\ \sigma_{\min},\ \sigma_{\max}\Big), \qquad \sigma_{\min}=0.5,\ \sigma_{\max}=25, \tag{12}$$

falling back to $\sigma_{\mathrm{def}}=6.0$ when the source reports no spread at all.

Why a per-player scale, and not a constant

The original specification used a single hardcoded $\sigma = 6.0$ for everybody. That number is not wrong — it is well calibrated for the median player. Measured across the 2026 FFC half-PPR pool (the 203-player snapshot of §1.5; every 2026 figure in this section is that pool, and not the PPR or standard feeds, whose spreads run wider), ς runs from 0.4 to 39.2 with a median of 10.3, and $10.3/1.8138 = 5.68$. So 6.0 is within about six percent of the median player's own scale. But the median is not where a draft assistant earns anything:

- The tightest player on the board is Puka Nacua at $\varsigma = 0.4$, giving $\sigma = 0.22$ — clamped up to $\sigma_{\min} = 0.5$. Even after the clamp he is **twelve times** more predictable than $\sigma = 6$ assumes. He is simply gone. Stop hoping.
- The widest is Evan McPherson, a round-14 kicker, at $\varsigma = 39.2$, giving $\sigma = 21.6$. He is **three and a half times** less predictable than $\sigma = 6$ assumes. He might last three more rounds. Stop panicking.

Note what the second bullet does *not* say: $\sigma_{\max} = 25$ would need $\varsigma \geq 45.3$, and nobody on this board is near that. So of the two clamps in (12), the ceiling binds on *zero* of the 203 players and the floor on three. The 2024 era board of §13.3 reads the same 3 low, 0 high, which `calibrate -tune` prints on every run.

Both tails are exactly where the tool is supposed to be useful, so we use the real number where we have it. The constant survives only as the fallback for user-supplied CSVs and any source without a spread column.

3.4 Conditioning on the present

Here is the single most important correction in the whole model, and it is one line of elementary probability.

Proposition 3.3 (Conditional survival)**.** *For $q \geq p$,*

$$\Pr\left[A_j(q) \mid A_j(p)\right] \;=\; \frac{S_j(q)}{S_j(p)}. \tag{13}$$

Proof. Availability is monotone: once a player is drafted he stays drafted, so for $q \geq p$ the event $A_j(q)$ *implies* $A_j(p)$, i.e. $A_j(q) \subseteq A_j(p)$. Therefore $A_j(q) \cap A_j(p) = A_j(q)$, and the definition of conditional probability gives

$$\Pr[A_j(q) \mid A_j(p)] = \frac{\Pr[A_j(q) \cap A_j(p)]}{\Pr[A_j(p)]} = \frac{\Pr[A_j(q)]}{\Pr[A_j(p)]} = \frac{S_j(q)}{S_j(p)}. \qquad \square$$

That is the whole derivation — it is the definition of conditional probability applied to a nested pair of events — but its effect on the board is enormous.

Example 3.4 (The faller)**.** A player with $a_j = 18$ and $\sigma_j = 3$ is still on the board at pick 21, and my next pick is 22: one pick to go.

The *unconditional* curve says $S_j(22) = 1/\big(1 + e^{(22-18)/3}\big) = 1/(1 + 3.794) = 0.209$. The board would show 21% and imply he is almost certainly gone — across a single pick.

The *conditional* ratio divides by what we already know: $S_j(21) = 1/(1 + e^1) = 0.269$, so

$$\Pr[A_j(22) \mid A_j(21)] = \frac{0.2086}{0.2689} = 0.776.$$

Seventy-eight percent, which is right: the worst that can happen in one pick is that one team takes him.

The unconditional curve is not answering our question at all. It answers "how likely was this player to last to pick 22, asked before the draft started", which is a question about the past. We already know how the past went: he is sitting there. The ratio throws away exactly the information we have already observed and keeps exactly the uncertainty that remains.

Conditioning barely moves anyone whose ADP is still ahead of the current pick, because then $S_j(p_{\text{now}}) \approx 1$ and dividing by it does nothing. It only repairs the players the market has already walked past — and those are precisely the players a draft assistant exists to point at.

Section 14 scores this directly: the unconditional logistic ("baseline: unconditional") is the *worst* model in the whole comparison, at Brier 0.1111 against 0.1022 for a constant predictor. It is worse than not modelling at all.

3.5 The deep tail is a constant per-pick hazard

Proposition 3.5 (Tail behaviour)**.** *Fix $q > p$. As $p - a_j \to \infty$ (the player has fallen far past his price),*

$$\frac{S_j(q)}{S_j(p)} \;\longrightarrow\; \exp\!\left(-\frac{q-p}{\sigma_j}\right). \tag{14}$$

Proof. Write $x = (p - a_j)/\sigma_j$ and $y = (q - a_j)/\sigma_j$, so $y - x = (q - p)/\sigma_j > 0$. Then

$$\frac{S_j(q)}{S_j(p)} = \frac{1 + e^x}{1 + e^y} = e^{x-y} \cdot \frac{e^{-x} + 1}{e^{-y} + 1}.$$

As $x \to \infty$ (and hence $y \to \infty$), both e^{-x} and e^{-y} vanish and the second factor tends to 1, leaving $e^{x-y} = e^{-(q-p)/\sigma_j}$. □

Definition 3.6 (Hazard rate)**.** The *hazard* of an event over a small window is the probability it happens in that window given that it has not happened yet. Here the window is one pick.

Equation (14) says that in the tail, surviving k picks costs a factor e^{-1/σ_j} each time: the picks are independent trials with a constant per-pick survival probability. The per-pick *hazard* — the chance that this pick is the one that takes him — is therefore

$$h_j \;=\; 1 - e^{-1/\sigma_j}. \tag{15}$$

At the table: a player with $\sigma_j = 6$ has $h_j = 15.4\%$, so roughly one pick in six-and-a-half takes him. A player with $\sigma_j = 0.5$ has $h_j = 86.5\%$; he is not surviving the next pick, never mind the next nineteen. That is what the scale parameter *is*, once the market has passed a player: how many picks he can absorb before somebody bites.

3.6 The numerics, and a genuinely instructive floating-point bug

Equation (13) is a ratio of two numbers that both go to zero fast. Getting it right in floating point is not automatic, and getting it wrong produces a specific, seductive failure that this project shipped and then measured.

Definition 3.7 (Softplus)**.** $\operatorname{softplus}(x) = \log(1 + e^x)$. It is smooth, increasing, ≈ 0 for $x \ll 0$, and $\approx x$ for $x \gg 0$.

Proposition 3.8 (Log-space form)**.** *With $\alpha = (q - a_j)/\sigma_j$ and $\beta = (p - a_j)/\sigma_j$,*

$$\log S_j(p) = -\operatorname{softplus}(\beta), \qquad \frac{S_j(q)}{S_j(p)} = \exp\big(\operatorname{softplus}(\beta) - \operatorname{softplus}(\alpha)\big). \tag{16}$$

Proof. $S_j(p) = 1/(1 + e^{\beta})$ by (9), so $\log S_j(p) = -\log(1 + e^{\beta})$, which is the first claim. The ratio is the exponential of the difference of the logs. □

The implementation evaluates softplus as `math.Log1p(math.Exp(x))` below a clamp of 30 and as x above it. That substitution is safe: at $x = 30$, $\log(1 + e^{30}) - 30 = e^{-30} \approx 9.4 \times 10^{-14}$, a relative error under 10^{-14}.

The wrong way, and why it is so tempting

The obvious defensive move is to clamp *each exponent* at ± 30 before exponentiating, compute $S_j(q)$ and $S_j(p)$ separately, and divide. This overflows nothing and looks careful. It is catastrophically wrong, and the direction of the error is the worst possible one.

Take a genuinely tight player who has fallen: $a_j = 10$, $\sigma_j = 0.5$, and a five-pick horizon. By (14) the true conditional survival is $e^{-5/0.5} = e^{-10} = 4.54 \times 10^{-5}$, and by Proposition 3.5 it stays there no matter how much further he falls — the hazard is memoryless in the tail. Now watch what per-S clamping does as he falls (Table 1).

p_{now}	β	α	log-space (16)	clamp-each-S
20	20	30	4.54×10^{-5}	4.54×10^{-5}
22	24	34	4.54×10^{-5}	2.48×10^{-3}
25	30	40	4.54×10^{-5}	**1.000**
30	40	50	4.54×10^{-5}	**1.000**

Table 1: The clamping bug, with $a_j = 10$, $\sigma_j = 0.5$, horizon 5 picks. The truth is constant. Clamping each survival separately saturates both exponents at the same clamped value, so their ratio collapses to 1: the model reports a player who has fallen twenty picks past his price as *certain* to survive.

The mechanism is simple once you see it. Clamping does not distort $S_j(q)$ and $S_j(p)$ by similar amounts — it distorts them to the *same value*, because both exponents get pinned to the clamp. The ratio of two identical numbers is 1. And the failure is monotone in exactly the wrong direction: **the further past his ADP a player has fallen, the safer the broken model says he is.** A tool whose entire purpose is to flag fallers would have been reporting 100% survival for precisely the players it was built to reason about.

The log-space form has no such failure mode because the difference $\text{softplus}(\beta) - \text{softplus}(\alpha)$ degrades gracefully: past the clamp both softpluses are their arguments, so the difference is $\beta - \alpha = -(q-p)/\sigma_j$ exactly — which by Proposition 3.5 is precisely the right answer. The approximation is not merely safe, it is the correct asymptotic.

The lesson generalises: *when you need a ratio of tiny numbers, never compute the numbers.* Compute the difference of their logarithms. A clamp applied before a subtraction destroys the very quantity you are subtracting for.

Two more guards

- **Missing ADP.** A live draft feed registers handcuffs and rookies no ADP source ranked. Those players get $a_j = 999$ (`UndraftedADP`) — off the drafted radar, survives everything. Zero would be catastrophic: it would put them at the very top of the board's price curve.
- **Horizons in the past.** On a finished draft, q_1 falls back to the final pick, which is behind p_{now}. Unguarded, $\alpha < \beta$ and the ratio exceeds 1; a replay frame prints "105%". The engine clamps $q \leftarrow \max(q, p_{\text{now}})$, which restores the correct answer: no picks intervene, so everyone survives with probability 1.

3.7 Falling players are flagged, not model-fixed

There is a real temptation, on seeing a player twenty picks past his price, to adjust his ADP. The engine does not, and the reason is a discipline worth stating: our ADP is a measurement, and a measurement that disagrees with reality is information, not an error to be smoothed away.

Instead, the engine marks him. Player j is *falling* when

$$\frac{p_{\text{now}} - a_j}{\sigma_j} \;\geq\; \texttt{FallerSigmas} = 1.0, \tag{17}$$

i.e. when the draft has run a full one of *his own* standard scale units past his price. Measuring in his own σ is the point: eight picks past ADP is seismic for a first-rounder with $\sigma = 1$ and unremarkable for a round-13 flier with $\sigma = 20$.

The UI renders a falling player's numbers amber. His survival stays honest — by Proposition 3.3 it is already correct, and by Proposition 3.5 it is already generous — and his urgency needs no special case. The flag is the entire feature.

3.8 Sigma shrinkage: empirical Bayes on the spread

The problem, with a real example

The New York Jets defense, on the shipped 2026 board: ADP 174.8, spread $\varsigma = 1.8$, `times_drafted` $= 5$. Equation (12) turns that into $\sigma = 1.8/1.8138 = 0.99$, and (15) turns *that* into a per-pick hazard of 63%. The model is asserting that it knows, to within about one pick, when a round-15 defense will be drafted.

It knows no such thing. It has five observations. Five drafts that happened to agree is what five drafts usually look like when the thing being measured is a round-15 defense nobody thinks about — and the same board has players with two hundred observations behind their spread. The two numbers should not be trusted equally, and (12) trusts them identically.

Across the 2026 pool, `times_drafted` runs 5/49/215 (minimum / median / maximum) and 53 of 203 players are supported by fewer than 25 drafts. Be careful with the headline number here: FFC's snapshot covers 1,187 drafts, but that is the size of the *window*, not the support behind any one player's spread — a player only appears in the drafts he was actually taken in, and the deepest names were taken in five of them. The spread that matters is 5 against 215, a factor of 43, and that is the spread the pseudo-count n_0 has to be read against.

Shrinkage, and why it applies to the variance

Definition 3.9 (Empirical Bayes)**.** An estimate is *shrunk* when it is pulled toward a central value; *empirical Bayes* means that central value (the *prior*) is estimated from the same data pool rather than supplied from outside. The classic result — Stein's paradox, made practical by Efron and Morris — is that shrinking many noisy estimates toward a common centre beats using each one raw, even though each raw estimate is individually unbiased.

The football reading: *a spread computed from five drafts is mostly noise; one from two hundred is signal. Blend each player's measured spread with the typical spread for players priced where he is, weighted by how much data he actually has.*

The blend is on **variance**, not on standard deviation. Two reasons, one statistical and one arithmetic.

Statistical. Suppose ς_j^2 is a sample variance from n_j observations, and put a conjugate prior on the underlying variance — a scaled inverse-χ^2 with n_0 degrees of freedom and scale τ^2. The posterior for the variance is again scaled inverse-χ^2, with degrees of freedom $n_j + n_0$ and scale

$$\hat{\varsigma}_j^2 \;=\; \frac{n_j\,\varsigma_j^2 + n_0\,\tau^2}{n_j + n_0}. \tag{18}$$

This is exactly a sample-size-weighted average of the observation and the prior, and it is what conjugacy buys you: the arithmetic of "how much should I believe my own measurement" comes out as a weighted mean with the weights being counts. (Being scrupulous: writing $\nu = n_j + n_0$ for the posterior degrees of freedom, the posterior *mean* of the variance carries an extra factor $\nu/(\nu-2)$ over the posterior *scale*, and we use the scale. At $n_0 = 25$ every player on the board has $\nu \geq 30$, so that factor runs from 1.07 at the thinnest-sampled player down to 1.03 at the median, and n_0 is a tuned pseudo-count rather than a derived one, so the distinction is well inside the noise we are already living with. It is stated here so nobody mistakes (18) for a theorem it is not.)

Arithmetic. Variance is the quantity that adds. Standard deviations do not combine linearly, and averaging square roots is not the square root of the average — by Jensen's inequality, since $\sqrt{\cdot}$ is concave, averaging the roots always lands *below* the root of the average. Concretely for the Jets defense with $\tau = 18.84$ (see below) and $n_0 = 25$:

$$\text{variance blend: } \sqrt{\tfrac{5(1.8)^2+25(18.84)^2}{30}} = 17.21, \qquad \text{stdev blend: } \tfrac{5(1.8)+25(18.84)}{30} = 16.00.$$

The variance blend is the more conservative of the two, which for a spread estimate — where under-stating spread means over-stating your own certainty — is the direction you want to err.

The prior is a line, not a constant

What is "the typical spread for a player priced here"? Not the pool median: spread grows with depth, measurably. Fitting ordinary least squares of ς against a over each pool gives

$$\text{2024 pool (178 players):} \quad \tau(a) = 0.76 + 0.0876\,a, \quad \text{median 6.9,} \tag{19}$$

$$\text{2026 pool (203 players):} \quad \tau(a) = 2.19 + 0.0953\,a, \quad \text{median 10.3.} \tag{20}$$

On the 2026 line, the prior expects 3.1 picks of disagreement about the ADP-10 player and 18.8 about the ADP-175 player. Shrinking a thin sample toward a single pooled median instead would tell a round-15 flier that the market agrees about him as tightly as it does about a round-5 back — which is the same mistake as the raw estimate, merely in the other direction. Where the fitted line would predict a non-positive spread (an extrapolation past the ends of the pool), the pool median stands in.

The prior must be fitted *per pool*. Fitting on 2026 and scoring 2024 would grade the shrink against a market it never saw — the same era mismatch that makes the two 2025 drafts unusable in §13.3.

What it does to the board

σ across the 2026 board	min	p25	median	p75	max
raw, from (12)	0.50	3.61	5.68	8.71	21.61
shrunk, $n_0 = 25$	0.59	4.17	6.48	8.94	14.08

Table 2: Sigma quantiles before and after shrinkage, 203 players (quantiles by linear interpolation). Players pinned at the $\sigma_{\min} = 0.5$ floor: 3 before, 0 after. The shrink pulls in both tails, but it is not symmetric — the top end moves much more, because extreme spreads live on thinly-sampled deep players.

The Jets defense goes from $\sigma = 0.99$ to $\sigma = 9.49$, i.e. from a 63% per-pick hazard to 10%. That is the correction the whole subsection exists for. Note that the shrink does not *always* widen: a well-sampled player with an unusually large spread gets pulled in, which is why the maximum drops from 21.61 to 14.08. The shrink is not a safety margin, it is a weighted average, and it moves whichever way the evidence is thin.

§14 grades it. On the raw untilted model, shrinkage moves Brier from 0.0868 to 0.0856 and log-loss from 0.3007 to 0.2895. Once the tilt of §4 is applied on top — which is what ships, so it is the comparison that counts — it moves Brier from 0.0677 to 0.0670 and log-loss from 0.2288 to 0.2250. Small, consistent, and in the right direction on every horizon segment. It ships not because it moved the headline but because it is right, it is cheap, and the Jets-defense failure it fixes is a real one that a 63% per-pick hazard would have printed on screen.

4 The exactly-N tilt

4.1 The problem: a budget the model does not respect

Write $p_j = \Pr[A_j(q_1) \mid A_j(p_{\text{now}})]$ for the conditional survival of each available player from (13). Each player is either removed before my turn or he is not, so under the model the number of removals is $X = \sum_j \mathbf{1}[j \text{ is taken}]$, a sum of indicators with $\Pr[\text{taken}] = 1 - p_j$. By linearity of expectation — which needs no independence assumption at all —

$$\mathbb{E}[X] \;=\; \sum_{j \text{ available}} (1 - p_j). \tag{21}$$

But we *know* X. Exactly $N = q_1 - p_{\text{now}}$ picks happen before my turn, and each one removes exactly one player. X is not a random variable at all; it is a constant we can read off the clock. And the model does not respect it.

How badly? Measured two independent ways:

- **On the live 2026 board.** Driving the scripted mock draft to completion from all twelve seats gives 1,969 board frames on which somebody is waiting. On 1,945 of them (21) exceeds N, and averaged over frames it expects 11.0 removals against a mean window of 7.9 picks. The over-prediction is not an occasional artifact of one board state; it is what the model does almost everywhere.
- **On the 2024 backtest** (15,923 predictions, 168 vantages, see §13.3). Averaged over vantages, the target was 12.0 picks, the model expected 16.8 removals, and the board actually lost 11.0 ranked players. Mean predicted survival was 0.8231 against an observed rate of 0.8844.

And critically, the error is **one-directional**. Table 3 is the reliability table before any correction: it groups predictions into ten bins by what the model said, and shows what actually happened in each bin.

bin	mean predicted	observed	n
0.0–0.1	0.0274	0.3413	1383
0.1–0.2	0.1481	0.6432	426
0.2–0.3	0.2495	0.6520	319
0.3–0.4	0.3475	0.7021	292
0.4–0.5	0.4491	0.7190	306
0.5–0.6	0.5496	0.7419	279
0.6–0.7	0.6504	0.7710	310
0.7–0.8	0.7527	0.7995	414
0.8–0.9	0.8544	0.8678	643
0.9–1.0	0.9931	0.9842	11551
overall	0.8231	0.8844	15923

Table 3: The original miscalibration. *Every* bin below 0.9 under-predicts survival, several of them by more than a factor of two. This is not noise — noise would scatter — it is a systematic bias, and it is the best single illustration of what a reliability table is for.

Read the top row at the table: of the 1,383 times the engine said "essentially gone, 2.7%", the player was actually still sitting there 34% of the time. That is not a model being unlucky. That is a model that is wrong in a fixable way.

4.2 Why it happens

The removals are not independent, and they are not free. Whoever is sitting between me and my next turn, they make exactly N picks between them, and each pick removes exactly one player. The model treats each player's fate as a separate coin flip against the market's price curve, and those coin flips collectively imagine a draft in which sixteen or seventeen players vanish in twelve picks. They cannot. The picks are a *budget*, and the independence assumption spends more than the budget has.

There is a second, smaller leak worth naming since it appears in the numbers above: the board only holds the players ADP ranked, and a real draft spends some of its picks on unranked names. That is why the target (12.0) and what the board really lost (11.0) differ. We do not correct for this — targeting N rather than 11.0 is a deliberate choice not to leak the labels into the model — and the residual over-correction it implies is small and in the safe direction. The live engine has the same leak, so correcting it here would grade a model the tool does not run.

4.3 The fix: one scalar on the cumulative hazard

Definition 4.1 (Cumulative hazard)**.** For a survival probability $p_j \in (0, 1]$, the *cumulative hazard* over the window is $H_j = -\log p_j \geq 0$. It is the total accumulated risk: $p_j = e^{-H_j}$, and by (14), in the tail $H_j = (q_1 - p_{\text{now}})/\sigma_j$, i.e. picks-in-the-window divided by his per-pick scale.

Now the correction. Multiply every player's cumulative hazard by one common factor c:

$$H_j \longmapsto c\, H_j \qquad \Longleftrightarrow \qquad p_j \longmapsto e^{-cH_j} = p_j^c.$$

Raising every survival probability to a common power *is* scaling every cumulative hazard by a common factor. This is the statistical model known as *proportional hazards*: it says the room drafts uniformly hungrier ($c > 1$) or uniformly slower ($c < 1$) than the market curve assumes, without changing anybody's relative position. One knob, and it is exactly the knob the situation calls for.

Choose c to make the budget balance:

Definition 4.2 (The tilt)**.**

$$\tilde{p}_j = p_j^{c^\star}, \qquad \text{where } c^\star \text{ solves} \quad f(c) := \sum_{j \text{ available}} \bigl(1 - p_j^c\bigr) = N. \tag{22}$$

4.4 The root exists, is unique, and bisection finds it

Theorem 4.3 (Well-posedness of the tilt)**.** *Let $p_1, \ldots, p_m \in [0, 1]$ and $f(c) = \sum_{j=1}^m (1 - p_j^c)$ on $c \in (0, \infty)$, with the convention $0^c = 0$ for $c > 0$. Then:*

(i) f is continuous on $(0, \infty)$.

(ii) f is strictly increasing provided at least one $p_j \in (0, 1)$.

(iii) Writing $m_0 = \#\{j : p_j = 0\}$ and $m_1 = \#\{j : p_j < 1\}$,

$$\lim_{c \to 0^+} f(c) = m_0, \qquad \lim_{c \to \infty} f(c) = m_1.$$

(iv) Hence for every $N \in (m_0, m_1)$ the equation $f(c) = N$ has exactly one solution $c^\star \in (0, \infty)$, and bisection on any bracket $[\text{lo}, \text{hi}]$ with $f(\text{lo}) \leq N \leq f(\text{hi})$ converges to it.

Proof. (i) Each term $c \mapsto 1 - p_j^c = 1 - e^{c \log p_j}$ is continuous in c for $p_j > 0$; for $p_j = 0$ the term is identically 1 on $c > 0$. A finite sum of continuous functions is continuous.

(ii) Differentiate term by term. For $p_j \in (0, 1)$,

$$\frac{d}{dc}\bigl(1 - p_j^c\bigr) = -p_j^c \log p_j > 0,$$

strictly, because $p_j^c > 0$ and $\log p_j < 0$. For $p_j = 1$ the term is identically 0 and contributes derivative 0; for $p_j = 0$ it is identically 1 and also contributes 0. So

$$f'(c) = -\sum_{j\,:\,0 < p_j < 1} p_j^c \log p_j \;>\; 0 \tag{23}$$

whenever at least one p_j lies strictly between 0 and 1, which makes f strictly increasing.

(iii) For $p_j \in (0, 1]$, $p_j^c = e^{c \log p_j} \to 1$ as $c \to 0^+$, so those terms vanish and only the $p_j = 0$ terms survive, each contributing 1: the limit is m_0. As $c \to \infty$, $p_j^c \to 0$ for every $p_j < 1$ while $p_j = 1$ stays 1, so each of the m_1 terms with $p_j < 1$ contributes 1: the limit is m_1.

(iv) By (i)–(iii), f is a continuous strictly increasing function whose range is exactly the open interval (m_0, m_1). A strictly monotone function is injective, so the preimage of N is a single point; the intermediate value theorem supplies its existence. Bisection is valid for any continuous function that changes sign across a bracket, and monotonicity guarantees the standard bisection invariant ($f(\text{lo}) \leq N \leq f(\text{hi})$) is preserved at each halving. □

The implementation brackets $c \in [1/64,\, 64]$ and halves until the bracket is narrower than 10^{-6}, which is about 26 iterations — microseconds. Each evaluation of f reuses a precomputed array of $\log p_j$, so the bisection costs one `math.Exp` per player per iteration rather than a full re-derivation of every survival curve.

4.5 The properties that make it safe

The tilt touches every number on the board, so it is worth being explicit about what it cannot break.

Proposition 4.4 (Fixed points and order)**.** *For any $c > 0$:*

(a) $p_j = 1 \Rightarrow \tilde{p}_j = 1$*, and* $p_j = 0 \Rightarrow \tilde{p}_j = 0$*.*
(b) $p_i > p_j \Rightarrow \tilde{p}_i > \tilde{p}_j$ *for* $p_i, p_j \in (0, 1]$*.*

Proof. (a) $1^c = 1$ and $0^c = 0$ for every $c > 0$. (b) $x \mapsto x^c = e^{c \log x}$ has derivative $cx^{c-1} > 0$ on $(0, \infty)$, so it is strictly increasing there; apply it to $p_i > p_j$. □

Part (a) is what makes the tilt safe on the two boundary cases that matter. On *my own pick*, $N = 0$ and no picks intervene, so every $p_j = 1$ and the tilt cannot move anybody — the whole chain (survival = 1, $\mathbb{E}[\text{best later}] = v(\text{bestNow})$, urgency = 0) stays exact arithmetic rather than the output of a bisection that returned 1.0000004. The implementation short-circuits on the horizon rather than on N for exactly this reason. Undrafted-radar players with $a_j = 999$ likewise stay at 1.

Part (b) is what makes the tilt safe on the board's *ordering*. The tilt changes what the numbers say; it cannot change who is more likely to survive than whom. Anyone reading the board's sort order sees the same ranking before and after.

4.6 The two clamps

The bracket $[1/64, 64]$ can fail to contain a root, and both failures are real board states rather than defensive padding. By Theorem 4.3(iii):

- $f(64) < N$: even at maximum hunger the pool cannot supply N removals. This is a *thin board* — a late endgame where almost every remaining player is near-certain to survive, or a small test fixture being driven as a twelve-team draft. Clamp $c = 64$. Physically: the model is doing its best and the picks will have to come from players it has never heard of.
- $f(1/64) > N$: even at minimum hunger the model expects more than N removals. The board is stacked with players the model has already written off (deep fallers with p_j near zero) while too few picks remain to take them all. Clamp $c = 1/64$.

On the shipped 2026 board neither clamp fires: over the 1,969 mock frames of §4.1, $c^\star$ ran 0.3812 / 0.6551 / 1.1215 (min / median / max) with **zero** clamped. On the 2024 backtest the same holds — across all 168 vantages $c^\star$ ran 0.2873 / 0.4419 / 1.0073 with **zero** clamped. The bracket is wide enough to be irrelevant on real data, which is the correct state for a clamp to be in.

Note the direction: $c^\star < 1$ on 1,945 of those 1,969 frames, which means the tilt is *raising* survival probabilities almost everywhere. That matches Table 3 exactly — every bin was under-predicting — and it is a reassuring sign that the two diagnostics, an internal budget check and an external backtest, are pointing at the same defect.

4.7 N is supplied by the caller, not derived

The correct removal count differs by horizon, and getting it from the snake arithmetic rather than special-casing it at each call site is what keeps the two uses consistent.

Definition 4.5 (Opponent picks before a horizon)**.**

$$N(\text{at}) = \#\{\, p \in [p_{\text{now}}, \text{at}) : \text{slot}(p) \neq \text{mySlot} \,\}. \tag{24}$$

At at $= q_1$ every intervening pick belongs to somebody else, so $N(q_1) = q_1 - p_{\text{now}} = N$. At at $= q_2$ exactly one intervening pick is *mine* — the one at q_1 — so $N(q_2) = q_2 - p_{\text{now}} - 1$. That subtraction matters: if the second leg of the two-pick plan (§8) counted my own pick as an opponent removal, it would price a player as at risk from a pick I am the one making.

There is a subtlety in the backtest that is worth flagging so nobody reconciles the two numbers wrongly. A backtest vantage *stands on* my own pick and looks forward to my next one, so its window $[\text{from}, \text{to})$ includes my own pick at `from` — I am one of the drafters removing a player, and the labels agree (a player I take at `from` has draft position $<$ `to`, hence $y = 0$). Live, the window opens on somebody else's pick and my next one closes it. Same rule — every pick inside the window removes exactly one player — but the live window is one pick shorter than any backtest row. That is the easy direction: shorter windows tilt less.

4.8 One truth on screen

The tilted probability $\tilde{p}_j$ is what everything downstream consumes: the expected best survivor, the tier-hold probability, the "safe to wait" tag, and the survival column in the data tab. An untilted probability never reaches the screen. This is a deliberate discipline rather than an implementation detail — if two panes quote different odds on the same player, the reader has no way to tell which one the board's ordering actually used, and the board stops being checkable.

5 The expected best survivor

5.1 The question

Fix a position P — say running back. I am not going to get *a specific* running back at my next pick; I am going to get *whoever is left*. So the quantity I want is not any one player's value but

$$\mathbb{E}\big[\max\{v_j : j \text{ at position } P, \text{ still available at } q_1\}\big].$$

Definition 5.1 (Order statistic)**.** Given a finite collection of numbers, an *order statistic* is one of them selected by rank — the largest, the second largest, and so on. Here the relevant one is the maximum, and the collection is random because *which* players are in it is random. In football: "the best guy actually left when my turn comes".

5.2 The derivation

Index the available players at position P in decreasing value, $v_1 \geq v_2 \geq \cdots \geq v_m$, with tilted survival probabilities $\tilde{p}_1, \ldots, \tilde{p}_m$. Let A_j be the event that j survives to q_1, and model the A_j as independent with $\Pr[A_j] = \tilde{p}_j$.

Theorem 5.2 (Expected best survivor)**.** *Define $M = \max\{v_j : A_j \text{ occurs}\}$, with $M = 0$ if no player survives. Then*

$$\mathbb{E}[M] \;=\; \sum_{j=1}^{m} v_j\, \tilde{p}_j \prod_{i<j} \big(1 - \tilde{p}_i\big). \tag{25}$$

Proof. Consider the events

$$B_j \;=\; A_j \cap \bigcap_{i<j} A_i^c \qquad (j = 1, \ldots, m), \qquad B_0 \;=\; \bigcap_{i=1}^{m} A_i^c.$$

In words: B_j is "j survives and everyone better is gone"; B_0 is "the position empties completely".

These partition the sample space. They are disjoint — for $j < k$, B_j requires A_j and B_k requires A_j^c — and they are exhaustive, since either no A_i occurs (giving B_0) or there is a smallest index j with A_j occurring (giving B_j).

On B_j the maximum surviving value is v_j: every survivor has index $\geq j$ (the better-indexed ones are gone by construction), and the list is sorted, so v_j is the largest value among survivors. Ties in value do not matter — if $v_j = v_{j+1}$ the maximum is the same number either way. On B_0, $M = 0$ by definition.

Therefore, splitting the expectation over the partition,

$$\mathbb{E}[M] = \sum_{j=1}^{m} v_j \Pr[B_j] + 0 \cdot \Pr[B_0].$$

Independence gives $\Pr[B_j] = \tilde{p}_j \prod_{i<j}(1 - \tilde{p}_i)$, which is (25). □

Read the j-th term at the table: *his value, times the probability he is the one you end up choosing.* He has to survive, and everybody better has to be gone. That is all (25) says.

5.3 Reading the pieces

The residual mass is worth zero, and that is correct

$\Pr[B_0]$ — the position empties completely — contributes nothing to (25). That is not an omission. If every running back is gone, the value of the best available running back is genuinely zero; there isn't one. The alternative, substituting some notional deep-bench value, would be inventing a player.

The truncation is provably harmless

The implementation walks the list with a running accumulator $\text{acc} = \prod_{i<j}(1-\tilde{p}_i)$, adding $\text{acc} \cdot \tilde{p}_j \cdot v_j$ and then updating $\text{acc} \mathrel{*}= (1-\tilde{p}_j)$. It stops when $\text{acc} < \varepsilon = 10^{-6}$.

Proposition 5.3 (Truncation bound)**.** *Truncating* (25) *at the first* j *where* $\text{acc} < \varepsilon$ *omits at most* $\varepsilon\, v_1$*.*

Proof. acc is non-increasing in j (each factor is in $[0,1]$), and the omitted tail is $\sum_{k\geq j} v_k \tilde{p}_k \prod_{i<k}(1-\tilde{p}_i)$. Every $v_k \leq v_1$ by the sorting, and the remaining probabilities $\Pr[B_k]$ for $k \geq j$ sum to at most $\text{acc} < \varepsilon$ (they are disjoint events all contained in $\bigcap_{i<j} A_i^c$). So the tail is at most εv_1. □

On the shipped board $v_1 \approx 10^4$, so the bound is about 0.01 value points — under a millionth of the numbers being compared.

The check sits at the *top* of the loop rather than the bottom, and that placement does real work. On my own pick every $\tilde{p}_j = 1$, so acc is exactly 0 after the first term; the top-of-loop check then stops the walk after one term instead of stepping through two hundred players multiplying by zero. It also means $\mathbb{E}[M] = v_1$ *exactly* on my own pick, since an exact zero times a value is an exact zero. Urgency's exact zero (§6) is arithmetic, not luck.

5.4 What it replaced, and the discontinuity that removed

Milestone 4 used a threshold rule:

$$\text{bestLater}(P) = \operatorname*{arg\,max}_{j\,:\,p_j \geq 0.5} v_j,$$

the best player who clears a fifty-percent survival cutoff. It is intuitive and it is what a human says out loud ("he'll probably still be there"). It also makes urgency a *discontinuous* function of the probabilities, and that is fatal for a board that re-sorts on every pick.

Concretely. Suppose the position holds $v_1 = 900$ at $p_1 = 0.501$ and $v_2 = 600$ at $p_2 = 0.95$. The threshold rule says bestLater is the 900 player, so urgency $= (v(\text{bestNow}) - 900) \cdot \text{need}$. Now one pick happens somewhere else on the board, nudging p_1 to 0.499. The threshold rule now says bestLater is the 600 player and urgency jumps by $300 \cdot \text{need}$ — enough to re-sort the entire board — on a change of two thousandths in one probability.

Equation (25) has no such cliff. It is a polynomial in the $\tilde{p}_j$, hence continuous (indeed smooth) in all of them, so no single pick can re-order the board by nudging somebody across a line. And it is the same question asked properly: the threshold rule is a crude one-point approximation to the expectation, replacing a distribution with its most likely-ish member.

Example 5.4 (The worked anchor)**.** Three players, $v = (100, 90, 60)$ with $\tilde{p} = (0.2, 0.6, 0.99)$:

$$\mathbb{E}[M] = 100(0.2) + 90(0.8)(0.6) + 60(0.8)(0.4)(0.99) = 20 + 43.2 + 19.008 = 82.208.$$

With an open dedicated starter slot (need = 1) and $v(\text{bestNow}) = 100$, urgency is 17.792. This exact case is a table-driven test in the repository; if you change (25) it fails.

5.5 Naming one player anyway

Urgency prices the whole distribution, but a header that reads "`chase → hall`" needs a single name. The engine reports the *modal* best survivor: the j maximising $\Pr[B_j] = \tilde{p}_j \prod_{i<j}(1-\tilde{p}_i)$ — the same per-player term (25) sums. It answers the question a human actually asks: *who am I probably picking instead?*

It usually agrees with the old threshold answer. Where they differ, the modal answer is the one *higher* up the board, and there is a reason: a player near the top only has to outlast the intervening picks, while a player further down also needs everyone above him to go first. So a 49% player above the line beats a 50% player below it, and when nobody clears 0.5 at all the modal answer is the top of the position rather than the deepest, safest name on it. That is the right failure mode; the old rule's was not.

6 Urgency and need

6.1 The headline number

Definition 6.1 (Urgency)**.**

$$\text{urgency}(P) \;=\; \Big(v\big(\text{bestNow}(P)\big) \;-\; \mathbb{E}\big[M_P\big]\Big) \cdot \text{need}(P), \tag{26}$$

where bestNow(P) is the highest-value available player at P and $\mathbb{E}[M_P]$ is (25) evaluated at P with horizon q_1.

This is the answer to the question in the title. The first factor is the value you lose by waiting; the second is how much you care about this position at all. The board sorts its position groups by (26), descending. The top group is the pick.

Three properties, all of which fall out of what has already been proved:

Proposition 6.2 (Basic properties of urgency)**.** *(a)* urgency$(P) \geq 0$ *always.*

(b) On my own pick, urgency$(P) = 0$ *for every position simultaneously. Off my own pick,* urgency$(P) > 0$ *strictly, provided* need$(P) > 0$ *and the position holds at least one available player with* $v_1 > 0$*.*

(c) urgency(P) *is continuous in every* $\tilde{p}_j$*.*

Proof. (a) By Theorem 5.2, $\mathbb{E}[M_P] = \sum_j v_j \Pr[B_j]$ with $\sum_j \Pr[B_j] \leq 1$ and every $v_j \leq v_1 = v(\text{bestNow})$. Hence $\mathbb{E}[M_P] \leq v_1 \sum_j \Pr[B_j] \leq v_1$, so the bracket in (26) is non-negative; need ≥ 0.

(b) On my own pick $q_1 = p_{\text{now}}$, so by Proposition 4.4(a) every $\tilde{p}_j = 1$; then $\Pr[B_1] = \tilde{p}_1 = 1$ and $\Pr[B_j] = 0$ for $j > 1$, giving $\mathbb{E}[M_P] = v_1$ exactly and urgency 0.

For the other direction, the tempting argument is wrong and it is worth seeing why. One might say "if any $\tilde{p}_j < 1$ then $\Pr[B_1] < 1$". It does not follow: the partition of Theorem 5.2 gives $\Pr[B_1] = \tilde{p}_1$ *exactly* — the product over $i < 1$ is empty — so a small $\tilde{p}_j$ at some $j > 1$ says nothing at all about $\Pr[B_1]$. The correct route goes through B_0 instead.

Off my own pick, $q_1 > p_{\text{now}}$, so for every player $\alpha_j > \beta_j$ in (16); softplus is strictly increasing, so softplus$(\beta_j) -$ softplus$(\alpha_j) < 0$ and $p_j < 1$. The map $x \mapsto x^c$ fixes only 0 and 1, so $\tilde{p}_j < 1$ for *every* j — and therefore

$$\Pr[B_0] = \prod_{i=1}^{m} \big(1 - \tilde{p}_i\big) \;>\; 0.$$

That single strict inequality is the whole argument. Since $\sum_j \Pr[B_j] = 1 - \Pr[B_0] < 1$ and every $v_j \leq v_1$,

$$\mathbb{E}[M_P] \;\leq\; v_1\big(1 - \Pr[B_0]\big) \;<\; v_1,$$

so urgency is strictly positive whenever $v_1 > 0$ and need$(P) > 0$.

(c) $\mathbb{E}[M_P]$ is a polynomial in the $\tilde{p}_j$. □

Remark 6.3 (The exact characterisation, and a floating-point caveat). Rearranging (26) with $\sum_j \Pr[B_j] = 1 - \Pr[B_0]$,

$$v_1 - \mathbb{E}[M_P] \;=\; \underbrace{v_1 \Pr[B_0]}_{\geq 0} \;+\; \sum_j \underbrace{(v_1 - v_j)\Pr[B_j]}_{\geq 0},$$

so urgency is 0 exactly when every term vanishes: either $v_1 = 0$ (a position no source values — kickers and defenses before §11.3 anchors them), or all the probability mass sits on players whose value *equals* v_1. Part (b) is the two cases of that identity we actually use.

The caveat is that "$\tilde{p}_j < 1$" holds in exact arithmetic, not always in `float64`. A player with $a_j =$ `UndraftedADP` $= 999$ early in a draft has both softpluses underflowing to a difference below 2^{-53}, so `PSurviveAt` returns exactly 1.0 and the tilt leaves it there. If such a player were the top of his position, that position would read urgency 0 off my own pick — which is the right answer (nobody is going to take him) but arrives by rounding rather than by the proof.

Part (b) is the *wait signal*, and it costs no extra machinery: zero urgency means the best player at the position will keep, so there is nothing to lose by waiting. On my own pick every urgency is zero simultaneously — nobody can be taken in zero picks — which is why the board needs a tie-break for exactly that frame (§11).

6.2 Need

need$(P) \in [0, 1]$ is how much my roster wants another player at position P right now. The league's starting lineup is a list of *slots*: by default QB / RB / RB / WR / WR / TE / FLEX / K / DEF, plus six bench spots, but the real value is read from Sleeper's league metadata (this user's 2025 league runs *two* flex slots, which the default does not).

Definition 6.4 (Need weights)**.** Write $R_{\text{rem}} = R - r(p_{\text{now}}) + 1$ for the number of rounds still to be played, including the one in progress. Then

$$\text{need}(P) = \begin{cases} 0 & \text{if } P \in \{\text{K, DEF}\} \text{ and } R_{\text{rem}} > 3 \quad \text{(suppression)}, \\ 1.0 & \text{if a dedicated } P \text{ slot is unfilled}, \\ 0.6 & \text{else if a FLEX slot is unfilled and } P \text{ is flex-eligible}, \\ 0.25 & \text{otherwise (a bench pick)}. \end{cases} \tag{27}$$

The three weights are a judgement call, not a measurement, and they should be read as one. 1.0/0.6/0.25 encodes "a starter is worth about four bench players and about one and two-thirds flex fills". Nothing in the backtest grades them — the backtest grades *survival*, and need does not enter survival at all — so they are tuned by eye against how the board reads at a real draft. If they are wrong, the tool mis-*orders* positions without mis-*stating* any probability, which is the better of the two failure modes and is why the calibration work went into survival first.

6.3 Slot filling is greedy, and greedy is optimal here

To know whether a dedicated slot is unfilled, we have to assign my drafted players to slots. The engine does it greedily: fill every dedicated slot first (a slot of position P takes the first unused player of position P), then fill flex slots from whoever is left, then everyone else is bench. This is not merely convenient — it is optimal.

Proposition 6.5 (Greedy dedicated-first maximises filled starters)**.** *Let the lineup have ℓ_P dedicated slots at each position P (ℓ for lineup; s is still my seat) and F flex slots accepting any position in a set $\mathcal{F}$, and let me hold n_P players at position P. Then the dedicated-first greedy assignment fills*

$$\sum_P \min(\ell_P, n_P) \;+\; \min\Big(F,\; \textstyle\sum_{P \in \mathcal{F}} \max(0,\, n_P - \ell_P)\Big)$$

slots, and no assignment fills more.

Proof. Greedy achieves that count by construction. For optimality, take any assignment and apply the following exchange: if some player x of position P sits in a flex slot while a dedicated P slot is empty, move x into the dedicated slot. The dedicated slot accepts him (same position), the flex slot becomes empty, and the total number of filled slots is unchanged. Each such exchange strictly decreases the number of flex-seated players whose own dedicated slot is open, so the process terminates.

The resulting assignment saturates dedicated slots: for each P it fills $\min(\ell_P, n_P)$ of them (it cannot fill more — there are only ℓ_P slots and n_P players — and if it filled fewer, some P player would be unassigned or flex-seated with a dedicated slot open, which the exchange has eliminated). The remaining players available for flex are exactly $\max(0, n_P - \ell_P)$ per position, and at most F of them can be seated. So no assignment exceeds the stated count, and greedy attains it. □

The order matters and the proposition says why: a flex-eligible player squatting in a flex slot while his own position sits open costs nothing in that assignment but blocks a different position later. The greedy rule sidesteps it entirely.

6.4 Roster-aware strategy, for free

This is worth noticing because it is the one place where the engine does something that looks like strategy without anybody programming strategy. Take two elite running backs in rounds 1 and 2. Both dedicated RB slots fill, so need(RB) drops from 1.0 to 0.6 (the flex is still open) and then to 0.25. Meanwhile need(WR) is still 1.0. Receivers sort to the top of the board on their own. There is no archetype detection, no "Zero RB" mode, no build-type classifier — just (26) multiplying by a weight that moved.

6.5 K/DEF suppression is a policy, not a model

need(K) = need(DEF) = 0 until three rounds remain. This is worth calling out because it is the one constraint in the engine that is *not* an attempt to model reality. It is an editorial rule: *nobody needs a tool to tell them to draft a kicker in round 6, and the tool must never do so.*

The evidence that it is a policy and not a belief is that the engine carries a second, looser constant for the same phenomenon. This league takes its first kicker in round 10 and its first defense in round 11 (medians: round 14 for both). So the opponent model of engine v2 — which is asking "what will the

room actually do", a factual question — uses `OpponentKDefLastRounds` = 6, while our own recommendation uses `KDefLastRounds` = 3. Two constants because they are two different questions: what we are willing to recommend, and what the room actually does. Collapsing them would make one of the two answers wrong.

6.6 The endgame guard

One more need modifier, near the end of the draft. Let L be my remaining picks (*not* R, which is rounds) and U my unfilled starting slots.

$$\text{bench weight } *= \begin{cases} 1 & U = 0 \quad \text{(lineup complete: every pick left is a bench pick)}, \\ 0 & L = U > 0 \quad \text{(every remaining pick must fill a starter)}, \\ 0.5 & L = U + 1,\ U > 0 \quad \text{(one pick of slack)}, \\ 1 & \text{otherwise.} \end{cases} \tag{28}$$

The first line is easy to drop and `endgameSlack` does not: it short-circuits on $U = 0$ before the arithmetic. Without it, a complete lineup with one pick left would hit $L = U + 1$ and halve every remaining weight. Nothing would visibly break, because with no open starters the halving is uniform across positions and the board's ordering is unchanged — which is exactly why the case has to be written down rather than trusted to show itself.

The multiplier applies only to positions that fill *none* of my open starting slots — the need function has already returned for anyone who fills a dedicated or a flex slot, so reaching the bench weight *is* the membership test. That ordering is what lets this be a multiplier rather than a set-membership check, and it matters in the one case that matters most: FLEX is a slot name, not a position, so "is RB among my unfilled starters" reads false exactly when the flex slot an RB would fill is the thing still open.

$L < U$ is deliberately *not* handled: that lineup can no longer be completed, and suppressing the rest of the board over a demand that cannot be met would bury the value still on it. Nothing changes, and the board says nothing special.

7 Tier-hold: cliffs as a probability

7.1 Tiers, briefly

A *tier* is a block of players at one position who are close enough in value that which one you get barely matters. The boundary between tiers is where it starts to matter a lot. Tiers come from a hand-transcribed rankings file where one covers a player, and are derived from value gaps otherwise:

$$\text{break a tier when} \quad \frac{\Delta v}{v_{\text{prev}}} > 0.10 \quad \text{and} \quad \Delta v \geq 0.015 \cdot v_{\text{top}},$$

where v_{top} is the value of the best player the derivation is walking — the best at the position when no rankings file is in play, and the best player the *file did not cover* otherwise, since the derivation only ever runs over the uncovered tail (`deriveBlocks` reads `rest[0].Value`). The relative test finds cliffs anywhere on the curve; the absolute floor stops the flat tail from shattering into singletons. Tier 0 means untiered — kickers and defenses, which no source values — and cliff logic skips it entirely.

Two structural rules exist because of one failure mode, and the scope of each is worth being exact about.

Oversize blocks are subdivided — but only the ones a human drew. A published graphic once put 21 receivers in "tier 2"; a tier that large can never trigger a cliff, so the board would never warn you about receivers at all. `subdivide` splits any rankings-file block over `TierMaxSize` = 8 at its largest internal relative drops. It is *not* applied to derived blocks: those come out of `deriveBlocks` and are appended untouched. On today's rankings-free board that shows — every tier is derived, and the derived blocks run QB 5/4/3/2/2/3/5, RB 2/11/7/5/11/15, WR 6/2/3/5/5/6/6/7/29, TE 6/4/5/4. Three of those exceed eight and one holds 29 receivers. The gap is real and is listed in §16; the honest reading is that the guard was built for the failure it was built for, and the value-gap path never got it.

No tier we *create may be a singleton.* A one-player tier is a permanent "last one" alarm, which is noise wearing an alarm's clothes, so `mergeRuntTiers` folds a runt into its neighbour — upward, into the block above it, except for a short leading block which folds forward. Singletons the human drew are left alone: one player alone in tier 1 is a statement about the position, not an artifact.

7.2 The complement trick

The question a cliff alarm should answer is: *will there still be anybody from this tier when I can act again?*

Definition 7.1 (Tier hold)**.** Let $\mathcal{T}$ be the available players in the tier of bestNow(P). Then

$$p_{\text{hold}}(P) \;=\; \Pr[\text{at least one of } \mathcal{T} \text{ survives}] \;=\; 1 - \prod_{j \in \mathcal{T}} \big(1 - \tilde{p}_j\big). \tag{29}$$

The derivation is one line, and it is a technique worth internalising. "At least one" is awkward to compute directly — you would have to sum over every nonempty subset, inclusion–exclusion and all. Its complement, "none of them", is a single product under independence. So compute the complement and subtract. That is the whole trick, and it recurs everywhere in applied probability.

7.3 Why a probability beats a headcount

The original design fired a cliff alarm on a count: amber at two players left, red at one. Counting cannot distinguish the two situations that matter most:

- **Three players, all about to go.** A tier of three where each player has $\tilde{p}_j = 0.3$ holds with probability $1 - 0.7^3 = 0.657$; make it $\tilde{p}_j = 0.1$ and it holds with probability 0.271. Count says "three left, relax". The probability says the tier is evaporating.
- **One player nobody wants.** A single deep player at $\tilde{p}_j = 0.95$ holds with probability 0.95. Count says "last one, take him or lose the tier". The probability says it is a free wait.

These are opposite situations and the count gives them opposite-to-correct answers. So cliff levels read (29): red below 0.15, amber below 0.5. The count is still *reported*, because the copy quotes it and "last one in tier 2" is a claim only the count can make — but under p_{hold} red can fire with three players left, so the last-one wording is kept only when the count really is one and otherwise red reads "tier 2 unlikely to hold — holds 9%".

7.4 Two guards worth their space

An untouched tier is never a cliff

A cliff means a tier is *emptying*, not that it happens to be small. If nobody has been drafted out of the tier yet, no alarm fires whatever (29) says. Without this rule, a rankings file's deliberate one-player tier 1 would print "last one, take him or lose the tier" at pick 1.01 of an empty draft.

On the clock, the horizon moves

This one is subtle and was found by measurement. *Off* the clock, tier-hold prices to q_1, the same horizon as the survival column beneath it, so the two agree and nothing needs explaining. *On* the clock, $q_1 = p_{\text{now}}$ and every survival is 1 by Proposition 4.4(a) — so every tier would hold at exactly 100% and the alarm would go silent on the one frame where you are actually choosing. Measured on the scripted mock: it fired on *none* of the 15 on-the-clock vantages.

The fix is to price to q_2 when on the clock: passing is the decision being priced, so the relevant horizon is the next time I could act on this tier, which is my pick after this one. The two horizons genuinely differ by design, so the copy names the pick it is pricing to — a hold of 3% printed directly above three survival cells reading 100% is two horizons one line apart with nothing marking which is which.

7.5 Runs

The run banner is pure counting over the pick feed and needs no probability at all. Slide a window over the last 6 picks; if ≥ 4 share a position, a run is active for that position. If two positions qualify, the one with more picks in the window wins. The copy branches on the tier state — "POS run in progress — n left in tier t" when the tier still has players, "POS run — tier broke, no value left" when it does not.

The banner needs no special logic to clear or to collapse: when a tier empties, bestNow drops to the next tier and urgency falls on its own, because (26) is reading a different player. One caution the code carries: tier 0 has two causes, and only one of them is "no value left". A position with nobody available is genuinely exhausted; K and DEF are tier 0 permanently, by design, with a full board of players on it. Inferring "tier broke" from the tier number alone put "def run — tier broke, no value left" above a group listing eight available defenses, in exactly the rounds this league drafts them.

8 Two-pick lookahead

8.1 The question urgency is too greedy to ask

Equation (26) optimises one pick at a time. The question at the turn is about two: *receiver now and running back on the way back, or the reverse?* A greedy board cannot see that the receiver tier will be gone by my second pick while the running back tier holds.

Corollary 2.7 is why this matters unevenly. From seat 3 of 12 the two picks at the turn are 19 apart and then 5 apart; the ordering of a pair genuinely changes what you can get. From seat 6 the gaps are 13 and 11 and the question is much softer.

8.2 The pair score

Let q_2 be my pick after next; if $q_2 = 0$ (the draft ends first) there is no plan. For every *ordered* pair of positions (P, Q) with nonzero need — $P = Q$ allowed, since double-tapping a position at the turn is a real strategy:

$$\text{score}(P,Q) \;=\; \underbrace{v\big(\text{bestNow}(P)\big)\cdot \text{need}^{\circ}(P)}_{\text{first leg}} \;+\; \underbrace{\mathbb{E}\big[M_Q^{-x}\big]\big|_{q_2}\cdot \text{needAfter}(Q \mid x)}_{\text{second leg}}, \qquad x = \text{bestNow}(P). \quad (30)$$

The recommendation is the argmax over pairs. With at most six positions that is at most 36 pairs and no search — but it is not free: each pair re-solves the tilt inside the loop, so `BestPlan` measures at 1.3–2.9 ms on the shipped 203-player board against 0.33–0.51 ms for a full six-position urgency pass. A render happens on a pick or a keypress, so that is affordable and the formula stays as written rather than hoisting the solve out and drifting from it. Four details:

- need$^{\circ}$ is not quite need. Membership in the candidate set is decided by (27) in full — the same number that decides whether the pane below the plan line shows the position at all, so the plan can never name a group the reader cannot see. The *weight* is the slack-free version, (27) without the endgame multiplier of (28), written need$^{\circ}$ here and `needFrom` in the code. The reason is symmetry: the second leg is priced by `NeedAfter`, which carries no slack either, and mixing the two made the score depend on the order of two legs that finish at the same roster.
- needAfter$(Q \mid x)$ is need$^{\circ}$ recomputed as if x were already on my roster. Taking a running back with the first pick is exactly what drops RB to flex weight for the second, and the pair score has to see that. It is computed on a *copy* of the roster — appending onto the live slice would write into its spare capacity, invisible to any equality check on the state and still a mutation during a render.
- $\mathbb{E}[M_Q^{-x}]$ excludes x from the pool: I took him. A no-op unless $P = Q$, and that is precisely the case worth getting right.
- The second leg's tilt uses $N(q_2) = q_2 - p_{\text{now}} - 1$ from (24), because my own pick at q_1 is not a removal somebody else makes. Getting this wrong prices the second leg against a phantom opponent who is me.

8.3 The honest simplification

The first leg is priced at $v(\text{bestNow}(P))$ — *today's* best available — even when my next pick is seventeen picks away and that player will plainly be gone by then. This is a deliberate simplification, not an oversight, and it is stated here because it is the single largest approximation in the engine.

The defence is that (30) is a *ranking*, not a forecast. Every pair is scored against the same board and carries exactly the same optimism in its first leg, so the comparison between pairs survives even though the absolute number does not mean anything. Which is why the score is deliberately *not* rendered on screen: printed as "(e 12318)" it read as an expectation while sitting three rows above the same player's survival column reading 0% — one line of the board contradicting the evidence directly under it, on 27 of 166 plan frames of the scripted mock. The pair is the product; the number was noise wearing a label it could not honour.

A proper fix exists and is scoped as future work: search over *my* picks inside the Monte Carlo rollout that engine v2 already runs for opponents. That is milestone 7 material, not a patch.

8.4 Milestone 7: the second leg stops being a formula (2026-08-11)

That future work landed, and it took the *second* leg, not the first. Under `-survival=sim` — the board's default — $\mathbb{E}[M_Q^{-x}]$ in (30) is replaced by what the rollouts actually deliver. For each first-leg candidate

P, extend the v2 window from $[p_{\text{now}}, q_2)$ to $[p_{\text{now}}, q_2]$ and play it 500 times: bestNow(P) comes off the board, the opponents draft exactly as §12 models them, and at q_2 the engine's own greedy rule — $\arg\max_j \text{vor}(j) \cdot \text{needAfter}(\text{pos}_j \mid x)$ over whoever is still there — takes a player. The pair is scored on him. The first leg's optimism is untouched: this milestone made the second leg real, and §8's simplification above still stands as written.

Four things are worth stating precisely, because each is a choice and not an implementation detail.

- **The candidate is removed at the start of the rollout, not at q_1.** That defines what the score is: score(P) is the pair's value *given I get P*. In futures where an opponent would have taken him first the plan's premise has already failed — the first leg would be quoting a man I do not have — so those futures are excluded by construction, and the opponents who did not take him spent their picks elsewhere, which is exactly what removing him first makes them do. It is also the semantics (30) always had, since $\mathbb{E}[M_Q^{-x}]$ excluded him too. "Will I even get him" is answered on the same frame, in its own units, by the survival column.
- **Common random numbers, and they are not optional.** Every candidate at one vantage is evaluated on one seed. Two candidates scored on independent samples differ by their choice *plus* sampling noise, and at 500 futures that noise is worth a few hundred value points — which is the scale of the gaps that decide these rankings. The board would reorder itself between renders on nothing. Measured on the fixture of §8.4: the variance of the score gap between two candidates is under half the unpaired variance.
- **$\mathbb{E}[\max] \geq \max \mathbb{E}$, and the gap is the point.** (30) takes the best of the per-position expectations; a rollout takes the best man across *all* positions in each future. The difference is the value of having more than one way for a future to go well, and it is the lookahead's own contribution as distinct from the survival model's. Measured on the scenario below: +50 and +40 value points on the two candidates that matter.
- **Feasibility is a preference in the policy, not a filter.** Written as a filter — "this leg may only take a man who closes an open slot" — it produced a genuine dead end at 12.09 of the scripted mock: only K and DEF unfilled, three picks left, and our own K/DEF suppression forbidding the only two positions that could satisfy it. The policy had nothing legal to take. Sorting slot-closers ahead of the rest is identical whenever one is alive and degrades to the best available man when none is, which is the same shape mustFill has carried all along.
- **But the policy may only name a position the board is offering.** Candidate membership is decided by (27) *with* the endgame multiplier of (28) — which is exactly 0 at $L = U$ — while the policy prices need with need$^\circ$, which carries no slack by design. (30) got the agreement for free by maximising over the candidate set; a rollout has to be told, and was not. Adversarial review caught it on the scripted mock at 13.09: the plan line read "rb at 13.09 → wr at 14.04" directly above "every remaining pick must fill a starter", with WR carrying need = 0 and no rendered row. It is the wrong *action* as much as the wrong label — with RB, K and DEF open and three picks left you take a back, a kicker and a defense. Measured footprint after the fix: 7 of 116 scripted plan lines, none before pick 120.

What can and cannot be claimed

A decision score has no counterfactual labels, so §13.3's referee cannot grade it and this document does not pretend otherwise. The evidence is a scenario. Build the mirror of §12's flagship board: national ADP says the backs are going now (ADP 103–110 against a pick at 102) and the receivers will keep (ADP 120–122 against my next pick at 115), while all six seats picking in between are full at running back with both receiver slots open. Greedy reads the market — receivers surviving at 65%, backs at 0.8% — and plans *rb now, wr later*, coming back for two men the room is about to eat. The conditioned score plans *wr now, rb later*: take the man you cannot get back, because the one you can wait for is still going to be there.

Attribution matters and is pinned in the tests rather than asserted here. Two things differ between those runs — the survival model *and* the second leg — and on that board the survival model carries most of the flip; greedy's own formula computed on top of sim survivals already prefers the receiver. The lookahead's separable contribution is the $\mathbb{E}[\max]$ gap above.

One honesty check, run because the conditioning could leak: a player's survival to q_2 inside the conditioned rollouts should differ from the unconditioned row §13.3 scores only through the knock-on effects of removing one player from a board of ninety. Worst move measured, 0.064, every sign in the direction removing a player predicts.

Three picks, and the question that dissolved

The same machinery extends to q_3, and for one milestone it did, behind `PlanDepth` = 2. The wheel was where the effect should be largest and it was: over 54 scripted frames at seats 1 and 12, depth three changed the *first* leg on 16 of them, and every flip moved away from running back — coherent with the wheel's own arithmetic, where two back-to-back picks let you double-tap the deep position later and take the scarce one now. Coherent is not right, and this project does not ship a change of that size on plausibility; §10.5 is the cautionary tale.

The question was never resolved. It dissolved. §9 replaces the pair score with a quantity that has no depth parameter at all — the value of the roster the draft actually ends with — so "how many legs" stops being a knob and becomes "all of them". Both confounds go with it: mustFill is no longer an input to the score, and the cost question is answered by the horizon shrinking every round rather than by choosing where to stop.

On screen

One clause, on the plan line, in both tenses: `plan wr at 3.03 → rb at 4.10 · lands tier-3 rb 100%`. It is an *outcome* claim — how often leg two lands the band the plan is counting on — which is why it earns a place on the frame that `CostOfPassing` was taken off. "You stand to lose 990 of TE value" is a forecast in board units about a thing that has not happened; "the tier-2 backs are there when you come back, 78% of the time" is the plan's own premise, stated as odds the reader can disagree with. The claim is *that tier or better*, since landing a better one is strictly good news. It drops whole below 92 columns, before the plan does.

8.5 The one thing that outranks the score

Late in a draft, finishing the lineup beats maximising the score, and by a lot, because K and DEF carry no imported value — both legs of any pair containing one score near zero, so such a pair can never win an argmax over value. Measured: at pick 14.10 with a defense still to fill and exactly two picks left, the plan line read "rb at 14.10 → rb at 15.03": two bench running backs and no defense at all.

So the plan first maximises the number of legs that fill an open starting slot, clamped to what is arithmetically required (mustFill = $\mathrm{clamp}(2-(L-U),0,2)$, with L and U as in (28)), and only then maximises (30). Up to the last couple of rounds mustFill = 0 and this is a plain argmax on score.

9 The finished roster

9.1 Every constant so far was standing in for one number

Look at what the pair score is made of. The need steps 1.0/0.6/0.25; the replacement discount $R(P)$; `EndgameSlack`; mustFill; the two-leg horizon. Each was introduced separately, for a reason of its own, and each was measured or argued into place. But they are all answers to the same question, asked without the means to answer it directly: *what does this pick do to the team I walk out with?*

need guesses what an open slot is worth. $R(P)$ guesses what you could have filled it with later. `EndgameSlack` guesses what a bench pick costs when the picks run out. mustFill guesses when that becomes binding. The two-leg horizon guesses that two picks is far enough to see.

§12 built a simulator that plays a draft forward. §8 pointed it at one extra pick. There is no reason to stop there.

9.2 The objective

Let $\mathcal{R}$ be a finished roster — a multiset of players. Assign it to the lineup by the same greedy rule the roster pane uses, dedicated slots before flex, taking players in value order; write $\sigma(\mathcal{R})$ for the assigned starters and $\beta(\mathcal{R})$ for the spill. Then

$$U(\mathcal{R}) \;=\; \sum_{j\in\sigma(\mathcal{R})} v_j \;+\; \sum_{j\in\beta(\mathcal{R})} w(\mathrm{pos}_j)\, v_j, \qquad w(P)=\begin{cases} 0.25 & P \text{ can reach a flex slot} \\ 0 & \text{otherwise,} \end{cases} \tag{31}$$

with an unfilled starting slot contributing nothing.

Three things fall out of (31) that used to be constants. A flex slot is worth exactly the player the assignment puts in it, so 0.6 is gone. An unfilled slot costs exactly the player who would have stood in it, so `EndgameSlack` and mustFill are gone from the score — feasibility stops being a guard bolted on from

outside and becomes a price. And the later picks *actually fill* the positions $R(P)$ was standing in for, so the replacement discount is gone too.

The bench weight w is the one constant (31) still owns, and it carries the two-index rule of §11 rather than a second guess: a bench back converts to points all season through flex, byes and injuries, while a second quarterback in a 1QB league scores zero by construction. That is not a new claim. It is the same finding that ended the 930-point subsidy early quarterbacks were collecting, applied in the one place it had not yet reached.

9.3 The estimator

For each candidate P, with b_P its best available man:

$$\text{score}(P) \;=\; \mathbb{E}[\,U(\mathcal{R}) \mid b_P \text{ is mine}\,], \tag{32}$$

estimated by playing the window from my next pick to my *last* one, M times. Each future: b_P comes off the board at the start (the conditioning of §8, unchanged — "will I get him" is the survival column's question and is answered there); opponents draft through the same `simCore` the survival table uses; every later pick of mine is taken by the engine's own greedy rule over whoever is really left; and the roster that results is scored with (31).

Two implementation notes, both load-bearing. The leg policy stops sorting the board: within a position, $(v - R)^+ \times$ need is monotone in value, so the argmax factorises into a walk down one static list per position with a per-rollout cursor. The winner of the six-way compare is the winner of the sort, ties included, and the cost of a leg drops from $O(n \log n)$ to $O(1)$ amortised. And common random numbers are not optional here: one seed per vantage, shared by every candidate, so two scores differ by the difference the choice makes and not by sampling noise. Measured at full horizon, the paired variance of a score gap is 13% of the unpaired.

9.4 What the display can say now

The score's units changed, and the display followed, because the engine is now holding three hundred finished versions of my team and had been discarding all of them but the mean. Three claims become sayable that were not:

- *The skeleton.* `plan te 10.10 → rb 11.03 → rb 12.10`. The plan runs to the end; the line renders a prefix of it, because a sixteen-leg line is a printout.
- *Your team from here.* One row per starting slot open today, read off the top choice's own futures: the modal filler when he recurs often enough to name, his band when he does not, the pick he arrives at, and the odds. One brain, so it cannot contradict the verdict above it.
- *Consequences, named in players.* "Taking rb instead costs you mcbride." This one exists *because* of the common random numbers: future m of the top choice and future m of the runner-up are the same world, so their ending rosters differ by what the choice caused and nothing else. Under independent sampling the same diff would be mostly noise, and the sentence would be a lie with a name in it.

9.5 Grading a decision, which §13.3 cannot

A survival prediction has a label: the draft either took him or it did not. A decision has none — nobody recorded what would have happened if you had taken the receiver. So `pick6 regret` builds the counterfactual instead. For each cached real draft, my seat is played by a policy while the other eleven keep their real recorded picks; when my counterfactual takes a man a later real pick names, that manager receives their own next surviving real pick, and every substitution is counted. Ending rosters are scored two ways and never averaged: U on era-adp-rank values, and the same lineup on a *linear* exchange rate, so a win that comes from the value curve's convexity rather than from the decisions shows up as a disagreement between the two columns.

The causality rules are §13.3's, and they are a package: a fold's room curve, its off-board escape rates *and* its measured positional demand all come from drafts that started before it. The 2024 fold has nothing before it and therefore has none of the three; its sim policies run in a configuration a live clock is never in, so it is reported and not gated on.

Two caveats ride on every run. U is also the quantity the new scorer optimises — what is being tested is whether optimising it against the *simulator* transfers to real rooms, and the opponents here are real. And the harness's own value curve is a monotone function of era adp rank, because no historical value

source exists; "best available" is therefore very nearly a greedy optimum of the exchange rate itself, and a policy beats it only through lineup structure.

10 Room-warped prices

10.1 The idea

National ADP is the average of thousands of strangers. A fixed set of twelve leaguemates is not the average of anything. If this room reliably takes quarterbacks seventeen picks earlier than the national market prices them, then every survival probability computed from national ADP is wrong for quarterbacks in a knowable direction.

We have completed drafts from this user's leagues: three of them, 552 picks. Crucially, the curve we build from them reads *only pick order and position* — never a player name, never a historical ADP. So no era-correct price board is needed to build one, and a season with no ADP archive still contributes: "the fourth quarterback went at pick 34.7" is as true in 2024 as in 2026, because it is a statement about behaviour and not about any player.

That is a statement about *era*, and it is emphatically not a licence to ignore *order*. A curve is only usable at the clock if every draft in it has already happened. §13.3 therefore holds each scored fold's *future* out of its own curve as well as the fold itself, and §14.4 reports what that costs — which is most of the evidence that ever supported §10.5's cutoff.

10.2 The room's price curve

Definition 10.1 (Room curve)**.** Write $\mu(P, k)$ for the mean, over the drafts that reached that depth, of the overall pick at which the k-th player of position P was taken. Then

$$\mathrm{adp}_{\mathrm{room}}(P, k) \;=\; \max_{k' \le k} \; \mu(P, k'). \tag{33}$$

The running max is over k' and the quantity inside depends on k' — that is the whole content of the definition, and §10.2 is why it is there.

Table 4 is the curve over all three drafts.

pos	depth	k=1	2	3	4	5	6	7	8
QB	21	23.0	27.0	31.7	34.7	39.0	58.0	68.0	81.3
RB	59	1.7	3.3	4.3	7.0	9.0	11.3	13.7	15.3
WR	70	1.3	4.3	6.3	8.7	10.0	11.0	13.3	15.3
TE	19	23.3	31.7	41.0	56.3	63.7	70.0	75.0	78.3
K	12	117.7	123.3	132.3	136.7	144.7	153.0	160.3	167.7
DEF	13	129.3	135.7	139.3	143.7	154.0	160.0	161.7	168.0

Table 4: $\mathrm{adp}_{\mathrm{room}}(P, k)$ over three completed drafts of this league (552 picks). *depth* is the deepest k the room ever reached at the position; past it the curve has no opinion at all.

Against the 2026 national board, the mean gap over the first five at each position is: QB −17.4, TE −13.4, K −8.6, WR −0.8, RB +0.4, DEF +30.0 picks (negative means the room is earlier than the market). This room is emphatically quarterback- and tight-end-early at the top of those positions, and takes defenses much later than the market does.

The running maxes are not cosmetic

Within a single draft the k-th player at a position obviously cannot go before the $(k-1)$-th. But the *means* can invert, because a deep k is supported by fewer drafts than a shallow one: if two of three drafts stopped at 19 quarterbacks, then $\mathrm{adp}_{\mathrm{room}}(\mathrm{QB}, 20)$ is one draft's number and can easily sit earlier than the three-draft average in front of it. An inverted price curve would tell the board that the 20th quarterback comes off before the 19th, which is not a small error — it is a statement that cannot be true.

There are in fact **three** monotonicity guards in the pipeline, each catching a different inversion, and each was added after measuring one:

1. The running max inside (33), above.
2. A second running max over the *blend* (below). The two inputs are each non-decreasing in k, but the weight between them is not constant — a thinly supported k stays nearer its market price than the

well-supported k in front of it. Measured on the 2026 board: the 13th defense (one draft deep) priced 5.2 picks *earlier* than the 12th (three drafts deep).

3. A *ceiling*. Since only the top K players at a position are warped (§10.5), a warped player must not be priced later than the market prices the next, unwarped man — or the two halves of the sequence do not join up. Measured: at $K = 5$ the warp priced the 5th defense at 144.16 while the 6th kept his raw ADP of 129.90, so the board read the 6th defense as coming off 14.3 picks *before* the 5th.

The general lesson: any time you splice two monotone sequences with a varying weight, or truncate a transformation partway down a sorted list, check monotonicity of the *output*. Monotone inputs do not give it to you.

10.3 Blending, with a pseudo-count

We do not want to replace the market's price with three nights of evidence. We want to move toward it in proportion to how much evidence there is.

Definition 10.2 (Warped effective ADP)**.** For player j whose position-ADP-rank among board players is k,

$$a_j^{\text{eff}} \;=\; w \cdot \text{adp}_{\text{room}}(P, k) \;+\; (1 - w) \cdot a_j, \qquad w = \frac{n_k}{n_k + 2}, \tag{34}$$

where n_k is the number of drafts that actually reached depth k at that position.

Definition 10.3 (Pseudo-count)**.** A *pseudo-count* is a quantity of imaginary prior evidence added to a real count so that a small sample cannot dominate. Here `RoomWarpPseudo` $= 2$ says "the national market is worth two of this room's drafts". So three drafts buy 60% of the warp, one draft buys 33%, and a depth no draft ever reached buys none — the rule "$w \to 0$ outside the observed range" falls out of the same formula rather than being a special case.

Two is deliberately timid. Three drafts of one room is thin evidence and the whole point of a pseudo-count is that the answer degrades toward the market rather than toward whatever three nights produced.

The warp is a **location shift only**. σ_j is never touched. The room disagreeing about a player's price is not the same claim as the room being more predictable about him, and only one of those two is measurable from three drafts.

10.4 Wiring: exactly two readers

Raw ADP is read in six places in the codebase and most of them must *not* warp. So the warped number lives in a separate field `Player.ADPEff` rather than overwriting `Player.ADP`, and precisely two functions read it: the survival curve and the falling flag. Those are the two asking "when does this player come off the board", which is exactly the question a room disagrees with the market about. The display columns, the board's sort tie-breaks, and — critically — the mock draft's simulated picker all keep the raw market price. Warping the mock's picker would warp the fake room itself, and the demo would confirm its own hypothesis.

10.5 Why the full-depth warp fails

This is the most interesting negative result in the project and it deserves the space.

The structural argument

National ADP ranks far more players at a position than any finite draft takes. The 2024 board lists 23 quarterbacks. A 12-team, 15-round draft takes about 19. So:

- Past the room's appetite, $\text{adp}_{\text{room}}(P, \cdot)$ has *no data at all* — the depth column of Table 4 is where it stops.
- Well *before* that point it is already unreliable, because the deep k values are supported by one or two drafts rather than three.
- And there are **more players in that unreliable tail than at the reliable head**. Five quarterbacks at the top against eighteen behind them.

So the tail swamps the head, and it does so with a systematic bias rather than noise: mapping rank $\to$ room-pick necessarily prices deep players *later* than the market does, because the market keeps ranking players the room simply never gets to.

The measurement

Table 5 is the held-out curve (built from the two 2025 drafts only) laid against the 2024 national board's quarterbacks.

rank	player	national ADP	room pick	room − national
QB1	Josh Allen	27.3	24.0	−3.3
QB2	Jalen Hurts	33.9	26.5	−7.4
QB3	Patrick Mahomes	36.2	31.5	−4.7
QB4	Lamar Jackson	40.6	34.5	−6.1
QB5	C.J. Stroud	44.2	40.0	−4.2
QB6	Dak Prescott	56.7	63.0	+6.3
QB8	Anthony Richardson Sr.	64.4	84.5	+20.1
QB10	Jordan Love	78.7	100.0	+21.3
QB12	Tua Tagovailoa	89.6	126.5	+36.9
QB14	Justin Herbert	103.6	138.5	+34.9

Table 5: The sign flip. The room is genuinely earlier than the market on the top five quarterbacks and dramatically later on everyone behind them — because the market keeps ranking quarterbacks the room never reaches. 23 quarterbacks on the board; about 19 taken per draft.

The consequence, which is the opposite of the intent

Table 5 shows the raw curve; what the model actually sees is that gap multiplied by the blend weight $w \approx 0.5$ of (34). Averaged over every prediction, the uncapped warp shifts named quarterbacks +11.0 **picks later**. The warp was built to capture "this room is quarterback-early". Applied at full depth, it says quarterbacks go *later* than the market thinks — the exact opposite of the signal it exists for — because the eighteen tail quarterbacks outvote the five at the head.

And on the fold this argument was developed against it loses the gate: Brier 0.0670 → 0.0671, log-loss 0.2250 → 0.2327, both worse. Read that pair with §13.3 in hand — it comes from a curve built out of that draft's future, and on the two folds whose curves predate them the full-depth warp *wins*. The argument below is a mechanism, not a measurement, and after this section that distinction does all the work.

Rank versus player: the apparent contradiction, resolved

There is something that looks like a contradiction in the measurements and it is worth untangling, because it is the sharpest thing the data says about this room.

- **QB *slots* go earlier than national.** The room takes its QB1 at pick 23.0 against a market ADP of 26.1, and the top five run 17.4 picks early.
- **QB *players* are the worst-calibrated position** in the 2024 backtest. Under the shrunk-plus-tilt model the warp is gated against — the numbers `calibrate` prints, and the same pair quoted in §14 — quarterbacks are predicted at 0.841 against an observed 0.917 — an under-prediction of 0.076, four times the next-largest (+0.019, running backs). Named quarterbacks survive *longer* than the model expects.

Both are true, and they are not in tension once you separate the rank from the man. The room reaches for quarterbacks early — but not for the *same* quarterbacks the national board ranks first. This room's QB4 is not the market's QB4. So the position's slots empty early while any particular named quarterback lasts longer than his national price implies, because the reaches are scattered across a wider set of names.

A rank-based warp captures the first fact. It cannot capture the second, and at depth it actively fights it.

The fix that was tried, and then removed

The obvious response is a cap: restrict the warp to the top K players at each position and leave everyone deeper on raw national ADP. That shipped for a while at `RoomWarpTopK` = 5, and **it has since been removed** — on both folds with a causal curve, uncapped beats it on both metrics, at native depth and at a common board size, while regressing fewer positions (§14.4). What follows is the evidence that chose it, all of which is retracted. On the 2024 fold that passes comfortably — Brier 0.0670 → 0.0660, log-loss 0.2250 → 0.2222 — with an apparent plateau at $K = 4$–5 and the sign flip well past it, and with five of six positions improving on both metrics. That is the sweep the constant was chosen from, and **every number in it is retracted**, for two independent reasons:

1. **It did not replicate.** Two 2025 folds are now scorable against a hand-exported board (§13.3). On both, the $K \leq 5$ restriction loses its per-position gate and the *uncapped* warp — the loser above — wins outright (Table 16). Board depth was the obvious confound, since k is a rank over the board and the 2025 boards are more than twice as deep; it was tested with `-depth 178` and it is not the explanation.
2. **It was never causal.** The 2024 curve was built from two drafts that ran a year after that draft (§13.3). Holding each fold's future out leaves 2024 without a curve, so the plateau, the per-position result and the upper end of the "never worse" band all cease to exist in the regime the tool runs in. Only `calibrate -lookahead` reproduces them.

So what is $K = 5$ still doing there?

Standing on the *mechanism* above and on one weak empirical fact, and nothing else. The mechanism is not in dispute: a national ranked list runs deeper than any finite draft, so past the room's appetite $\mathrm{adp}_{\mathrm{room}}(P, k)$ prices players later than the market *by construction*, and the mean shift measurably flips sign there. That argument does not depend on which fold is in front of you. The empirical fact is that $K = 5$ is never worse than not warping on either causal fold — but so is every other K the sweep tries, including no cap at all, so this rules nothing out.

Read the cutoff as unidentified and the cap as an argument. Nothing was retuned: moving K to whichever value wins the current tally would repeat the exact error that produced the retracted claim, and `-room=false` remains the fallback. What would settle it is a fold that is FFC-priced, is not 2024, and has a cached draft before it. FFC's archive still answers "No ADP data found" for `year=2025` (rechecked 2026-07-30), so recheck near draft day and re-gate then.

11 Value over replacement, and pricing the unpriced

11.1 Why raw value is the wrong tie-break

By Proposition 6.2(b), every position's urgency is exactly zero on my own pick. That is the frame where I am actually choosing, so the tie-break decides what the board points at when it matters most.

The obvious tie-break is $v(\mathrm{bestNow}(P)) \cdot \mathrm{need}(P)$: rank each position by its best available player. But that answers "who is the best player left", which is not the question. The best receiver on a board 69 receivers deep and the best tight end on a board 19 deep can carry the same value while buying completely different things, because behind the receiver sit sixty more and behind the tight end sit two.

Definition 11.1 (Value over replacement)**.** Let $\mathrm{dem}(P)$ be the *demand* for position P: how many players at it this league drafts in one draft. Let $v_{\mathrm{rep}}(P)$ be the value of the $\mathrm{dem}(P)$-th best P on the *pre-draft* board. Then

$$\mathrm{vor}(j) \;=\; \max\big(0,\; v_j - v_{\mathrm{rep}}(\mathsf{pos}(j))\big). \tag{35}$$

$v_{\mathrm{rep}}(P)$ is the player you would have ended up with at that position anyway. Subtracting it measures what taking j actually *buys*.

11.2 Demand is measured, not assumed

$\mathrm{dem}(P)$ is the median count of the position taken per draft over this league's own cached drafts (median, not mean: one of the three drafts is 16 rounds and two are 15, so a mean would quietly weight the long draft's extra twelve picks into every position).

pos	$\mathrm{dem}(P)$	valued on board	$v_{\mathrm{rep}}(P)$	the player you'd have settled for
QB	19	24	314	Tyler Shough (ADP 140.6)
RB	58	51	0	nobody — only 51 are valued, so replacement is free
WR	67	69	10	Malik Washington (ADP 164.2)
TE	17	19	182	Hunter Henry (ADP 146.3)
K	12	15	112	Evan McPherson (ADP 158.2)
DEF	12	19	148	Dallas Cowboys (ADP 153.2)

Table 6: Replacement levels on the shipped 2026 board. The spread *is* the content: quarterbacks carry a 314-point discount and tight ends 182, against none at all for running backs.

Compare the fallback, which is the league's lineup *shape* rather than a measurement (every team's dedicated slots plus an equal share of each eligible flex): that gives QB 12, RB 28, WR 28, TE 16. The gap

is the bench. A league starting one quarterback still drafts nineteen of them, and it hoards twice as many running backs as it can start. The fallback is a floor, not an estimate, and the tool prints which of the two is in play.

Running back is the instructive row: the room drafts 58 backs while only 51 carry a value at all, so replacement at running back is off the bottom of the board and a back's vor is his whole value. That is not a degenerate case, it is what "deep position" means expressed as a number.

$v_{\text{rep}}(P)$ is deliberately *static* — computed over the full pre-draft board, ignoring who has been taken. A replacement level that climbed as a position emptied would make every position look scarcer the longer you waited, which is exactly backwards.

Exactly one call site changed when vor landed: the zero-urgency tie-break in the board's group sort. Everything else still ranks by urgency.

11.3 Pricing kickers and defenses

FantasyCalc values QB/RB/WR/TE only. Kickers and defenses arrive with $v_j = 0$, and a position stuck at zero can never produce urgency however badly you need a kicker in round 15. So they need a value — and inventing one is exactly what this project does not do.

The rejected rule: nearest ADP

The specification said: take the value of the skill player with the nearest overall ADP, interpolating between neighbours. Implemented literally, it produces nonsense, and the reason is a genuinely useful observation: **value is not monotone in ADP**. Two players priced within a fifth of a pick of each other can carry wildly different values. Measured on today's board: Tre Tucker at ADP 128.0 carries 9 while Mark Andrews at ADP 128.2 carries 412 — forty-five times as much, two-tenths of a pick later. Higher up it is the same story with bigger numbers: Trey McBride at 39.5 carries 6,601 against Davante Adams at 39.6 with 2,697. ADP measures how early the market takes a player; value measures how good it thinks he is; scarcity at his position separates the two, and nothing forces them to agree locally.

Interpolating between whoever happens to sit either side therefore inherits that non-monotonicity and amplifies it. Measured on the board at the time: Denver DEF at ADP 103.3 got value 1698.6, Houston DEF at 106.4 got 989.9, and the LA Rams at 108.3 got 251.1 — three defenses ordered nonsensically across five picks of ADP, which then corrupts the available-player sort, since that sorts by value.

The rule that ships: anchor by rank

Let k be the number of valued skill players priced ahead of him — that is, with smaller ADP. Set his value to the k-th largest skill value.

Proposition 11.2 (Monotone by construction)**.** *The rank anchor is non-decreasing in ADP: if $a_i \le a_j$ for two K/DEF players then their anchored values satisfy $v_i \ge v_j$.*

Proof. k counts skill players with smaller ADP, so $a_i \le a_j$ implies $k_i \le k_j$. The sorted skill-value list is non-increasing, so the k_i-th entry is $\ge$ the k_j-th. □

It also lands *exactly on* the imported curve rather than between two points of it, and it needs no interpolation at all. Across all 34 kickers and defenses on the shipped board: zero monotonicity violations. Sample of the resulting curve (computed on the current board):

$$\text{ADP } 10 \to 8059, \quad 50 \to 3208, \quad 100 \to 868, \quad 150 \to 168, \quad 185 \to 9.$$

The other rejected alternative, and a scale lesson

The engine carries a rank-to-value fallback for boards with no imported values at all: $v = \texttt{ValueBase} \cdot e^{-\text{rank}/\texttt{ValueDecay}} = 250\, e^{-\text{rank}/40}$. Using it for K/DEF would have been the obvious move. It gives $250e^{-3.75} = 5.88$ at rank 150, on a board where the 150th-best skill player carries 141 — a **24-fold** scale mismatch.

This is the concrete face of the project's "never mix value modes in one draft" rule. Values are compared only by difference, so their scale is arbitrary — but only if it is *one* scale. Two scales in one board is not a cosmetic problem: it would make kicker urgency permanently invisible, since a 6-point drop cannot compete with a 300-point drop no matter how much you need a kicker.

Kickers and defenses stay at tier 0 on purpose even now that they have values. Their value is borrowed from a skill player at the same price, so a "gap" between two kickers is a gap between two receivers somewhere else on the board; it cannot draw a real tier boundary. Tier 0 is also what keeps cliff logic skipping them, and a red "last one in tier 3" about kickers is exactly the alarm this tool must never raise.

12 The opponent simulation (engine v2)

Everything in §3–§4 treats the picks between mine as random draws from "drafters in general". They are not. They are eleven specific rosters, Sleeper publishes every one of them, and if the three teams picking ahead of me all have full RB rooms, the RB I want is far safer than any price curve can know. Engine v2 replaces the survival number with a Monte Carlo over those actual rosters — and replaces *nothing else*: $\mathbb{E}[M_P]$, urgency, tier-hold, the plan and the banners all consume $\tilde{q}_j$ unchanged, through one chokepoint. It ships as the board's default; `-survival=adp` keeps v1, tilt and all, as the fallback and the baseline.

12.1 The pick model

For an opponent team t picking at overall pick q, the probability it takes available player j is proportional to

$$w_t(j) \;=\; e^{-k(j)/\tau} \cdot \mathrm{need}_t(\mathrm{pos}_j)^{\beta(r(q))},$$

where $k(j)$ is j's 0-indexed rank among currently available players by *effective* price (the room-warped price of §10 when a causal curve exists — the room's appetite belongs in the room's simulation), $\tau = 5$, and only the top 25 ranks get any weight at all. need_t is the same starter/flex/bench rule of §6, computed over t's actual roster, with one substitution: the K/DEF gate is the *opponents'* constant, not ours. The tool refuses to recommend a kicker before the last three rounds; this room demonstrably drafts its first one with seven rounds left, and a simulation that keeps drafted kickers "available" corrupts every number downstream. (That constant moved from 6 to 7 during review: "first kicker in round 10" had been read off a 15-round draft, and the room that matters ran 16 rounds and took two kickers at remaining = 7. The backtest A/B between the two values is a wash; the fact decided.)

$\beta(r) = \mathrm{clamp}((r-3)/4,\ 0,\ 1.5)$ ramps need in from nothing: rounds 1–3 are pure best-available, which is not an assumption — the 552-pick league profile measured 0% QB in round 1, 8% in round 2, 28% in round 3, and the ramp starts exactly where the data turns on. If every candidate's need is gated to zero (a late all-K/DEF pool), the need term is dropped for that one pick rather than dividing by it.

12.2 The rollout, and why the tilt stays home

One recompute plays the window from the current pick to my pick-after-next 500 times. Each rollout deep-copies the rosters of every team that picks in the window (a team at the turn picks twice inside one rollout, and a frozen roster would let it draft two RBs at identical need weight — the exact behaviour v2 exists to fix), samples each opponent pick from the weights above, and records *when* every player was removed. Recording removal times is what lets one rollout batch answer every horizon the engine asks about — NextPick, ActPick, and the lookahead's q_2 — as lookups.

My own picks inside the window remove nobody, live: the question on the board is conditional on my choice, and nobody can take a player from me across my own turn — which the sim now gets *exactly* right, returning survival 1 across back-to-back picks by arithmetic rather than by a clamped tilt.

Survival is the Jeffreys-smoothed count

$$\tilde{q}_j \;=\; \frac{\#\{\text{rollouts where } j \text{ survives}\} + \frac{1}{2}}{500 + 1},$$

because a player taken in all 500 rollouts is not probability zero, and §13 scores log-loss, where one hard zero that survives in reality costs infinity. The rollouts are seeded from the draft id and the pick number: a keypress re-render replays byte-identical futures, and every number on a frame is reproducible.

The tilt does not apply to sim output, and this is the most important integration fact in v2. The exactly-N tilt of §4 exists because independent survivals do not respect the removals budget. A rollout removes exactly the window's picks by construction; the budget the tilt balances is already balanced, and tilting it would distort every number. This also retires the thin-board fragility of §14.4 for the sim path: there is no budget aimed at players the board does not hold, because removals happen to actual players.

12.3 The off-board escape

The first honest backtest of the sim lost to v1 nearly everywhere, and the loss had one mechanism: the sim made *every* window pick remove a ranked player, and real rooms do not. Real rooms spend late picks on handcuffs, rookie fliers and third kickers no board ranks — measured against their own era boards, 56% of this room's final-round picks and 72% of its second-to-last-round picks leave the ranked pool entirely. Every such pick removes nobody I could have wanted, so a sim without an escape eats the board faster than reality and reads everyone as more gone than they are.

The fix is one hazard: with the measured probability for the pick's depth (indexed by rounds *remaining*, so 15- and 16-round drafts agree about the endgame), a simulated pick removes nobody. The rates are measured by `fetch` against each cached draft's own era board and consumed from disk at draft time; a replay holds the replayed draft and its future out of them, by the same two rules the room curve obeys (§13.3).

It is worth recording that the escape was discovered by a *leak*. The first scoring run left drafted-but-unranked players sitting in the backtest pool from vantage one — players only known to exist because a future pick registered them. They absorbed removals exactly the way real off-board picks do, the sim posted a large log-loss win, and the adversarial review killed it: the leak was an accidental, future-informed implementation of the escape. Those numbers are retracted; the causal escape below is the honest version of what they were accidentally measuring.

12.4 What the referee says

A backtest vantage simulates every pick in its window *including the vantage seat's own* — the same window convention the tilt's N uses, and the labels agree (a player the seat took is scored $y = 0$). Both sim variants share one seed, so the room-priced and raw-priced rows differ only by the price, the same construction as the ADP column gate. One provenance note dates everything: FFC's archive began serving 2025 on 2026-08-11 (it answered "no ADP data found" as late as 2026-07-30), so both 2025 folds now score on FFC boards — 156 names, 718 drafts, real spread — and the FantasyPros numbers of §14.4 are a different board's numbers. The table below is the FFC-priced present.

fold	model	Brier ↓	log-loss ↓	verdict
2024	shipped v1	0.0670	0.2250	
	v2 sim (no priors)	0.0695	0.2498	worse on both
	v2 sim + escape	0.0699	0.2511	no priors: no escape exists
2025-a	shipped v1	0.0711	0.3075	
	v2 sim, escape-less	0.0747	0.2651	split
	v2 sim + escape	**0.0701**	**0.2495**	**better on both**
2025-b	shipped v1	0.0774	0.3006	
	v2 sim, escape-less	0.0803	0.3201	worse on both
	v2 sim + escape	**0.0766**	**0.2775**	**better on both**

Table 7: Engine v2 against the shipped v1 row, per fold, on the FFC-priced boards of 2026-08-11. The 2024 fold has no prior drafts, hence no room curve and no measurable escape — a configuration the live 2026 draft, with three cached priors, is never in. On both folds that can test v2 as it would actually run, it beats what shipped on both metrics.

Per position, WR and K improve on both metrics on both causal folds — the deepest position is where roster-aware removal ordering pays — while RB and TE still give back log-loss: the need model drafts tight ends too eagerly and running backs not eagerly enough. That, a graded K/DEF appetite to replace the binary gate (the room's *first* kicker goes at remaining 7; its *median* kicker goes in the last two rounds, and a gate cannot say both), per-manager priors from `scout`, and the autodraft floor are the v2.1 levers — every one named, none built, all refereed by the same gate before they ship.

12.5 The deny chip

One social feature rides the machinery: on my pick only, the seat picking immediately after me is scored with the opponent model, and if its hungriest position is down to the last man of its best remaining band — and my own need for him is flex-weight or less — his row wears a `deny team` n chip. A chip, never the verdict: it cannot move the ranking, and the engine refuses to fire it on a player I need at starter weight, because that is called a pick. Strategically marginal, socially essential.

13 Calibration: how we know any of this

This is the intellectual core of the project. Everything above is a model; this section is how the model is graded, and it is the reason two of the ideas in the previous sections ship and two do not.

13.1 Scoring a probability

Suppose the engine says a player survives with probability $\hat{p}$, and then he either does ($y = 1$) or does not ($y = 0$). (Throughout this section $\hat{p}$ is a *forecast* and y its outcome; the letters p and q went to pick numbers back in §2 and are not reused here.) How do you score that? You cannot say "it was right" or "it was wrong" — a 70% forecast that fails is not wrong, it is a 30% event happening.

Definition 13.1 (Scoring rule)**.** A *scoring rule* is a function $\mathcal{S}(\hat{p}, y)$ giving a penalty (lower is better) for reporting probability $\hat{p}$ when the outcome is $y \in \{0, 1\}$.

Definition 13.2 (Proper scoring rule)**.** A scoring rule is *proper* if, whenever the forecaster's honest belief is that the outcome will be 1 with probability π, reporting $\hat{p} = \pi$ minimises the *expected* score

$$\bar{\mathcal{S}}(\hat{p}; \pi) \;=\; \pi\,\mathcal{S}(\hat{p}, 1) + (1 - \pi)\,\mathcal{S}(\hat{p}, 0).$$

It is *strictly proper* if $\hat{p} = \pi$ is the *unique* minimiser.

Propriety is the property that makes a metric trustworthy. Under an improper rule, a forecaster can score better by lying — by shading probabilities toward 0 and 1 to look decisive, say — and then the metric is measuring bravado rather than knowledge. Under a proper rule, the only way to score well is to actually believe what you say.

Definition 13.3 (Brier score)**.** $\mathcal{S}_{\mathrm{B}}(\hat{p}, y) = (\hat{p} - y)^2$, averaged over predictions [1].

Definition 13.4 (Logarithmic score / log-loss)**.** $\mathcal{S}_{\mathrm{L}}(\hat{p}, y) = -\big(y \log \hat{p} + (1 - y) \log(1 - \hat{p})\big)$, averaged over predictions, with $\hat{p}$ clamped to $[10^{-6}, 1 - 10^{-6}]$ so a single overconfident miss does not become the entire score.

Theorem 13.5 (Both are strictly proper)**.** *$\mathcal{S}_{\mathrm{B}}$ and $\mathcal{S}_{\mathrm{L}}$ are strictly proper.*

Proof. Brier. Expand the expected score:

$$\bar{\mathcal{S}}_{\mathrm{B}}(\hat{p}; \pi) = \pi(\hat{p} - 1)^2 + (1 - \pi)\hat{p}^2 = \pi\hat{p}^2 - 2\pi\hat{p} + \pi + \hat{p}^2 - \pi\hat{p}^2 = \hat{p}^2 - 2\pi\hat{p} + \pi.$$

This is a quadratic in $\hat{p}$ with positive leading coefficient. Differentiate: $\partial_{\hat{p}} \bar{\mathcal{S}}_{\mathrm{B}} = 2\hat{p} - 2\pi$, which vanishes only at $\hat{p} = \pi$, and $\partial^2_{\hat{p}} \bar{\mathcal{S}}_{\mathrm{B}} = 2 > 0$, so that point is the unique minimum. Note the minimum value is $\pi - \pi^2 = \pi(1 - \pi)$, which we will need in a moment.

Logarithmic. $\bar{\mathcal{S}}_{\mathrm{L}}(\hat{p}; \pi) = -\pi \log \hat{p} - (1 - \pi) \log(1 - \hat{p})$ on $\hat{p} \in (0, 1)$. Differentiate:

$$\partial_{\hat{p}} \bar{\mathcal{S}}_{\mathrm{L}} = -\frac{\pi}{\hat{p}} + \frac{1 - \pi}{1 - \hat{p}},$$

which vanishes when $\pi(1 - \hat{p}) = \hat{p}(1 - \pi)$, i.e. $\pi - \pi\hat{p} = \hat{p} - \pi\hat{p}$, i.e. $\hat{p} = \pi$. The second derivative $\pi/\hat{p}^2 + (1 - \pi)/(1 - \hat{p})^2 > 0$ everywhere on $(0, 1)$, so $\bar{\mathcal{S}}_{\mathrm{L}}$ is strictly convex and the stationary point is the unique minimum. □

Two proper rules rather than one, because they punish differently. Brier is bounded and quadratic: it cares about the average size of the error. Log-loss is unbounded and blows up as $\hat{p} \to 0$ on an outcome that happens: it savages confident mistakes. A change that improves one and wrecks the other is a change that traded honest uncertainty for false confidence, and reporting both catches it. In this project the two agreed on every decision that mattered, which is reassuring but not guaranteed.

The coin-flip ceiling

If you predict $\hat{p} = 0.5$ for everything, then $(\hat{p} - y)^2 = 0.25$ whatever happens, so your Brier score is exactly 0.25. That is why 0.25 is quoted as the "coin-flip ceiling": it is what total agnosticism buys, and anything above it is worse than saying nothing.

But 0.25 is a weak bar when the base rate is lopsided, and ours is. Write $\bar{o}$ for the observed base rate. Predicting $\bar{o}$ for everything scores $\bar{o}(1 - \bar{o})$ — by the Brier computation above with $\pi = \bar{o}$. Our observed survival rate is 0.8844, so the honest floor is

$$0.8844 \times 0.1156 = 0.1022,$$

which is exactly the "baseline: constant 0.884" row in the results. *That* is the number a model has to beat, not 0.25.

13.2 The Murphy decomposition, and what a reliability table is for

Definition 13.6 (Reliability table)**.** Sort all predictions into B bins by the value predicted. For each bin b record n_b (count), $\bar{p}_b$ (mean forecast in the bin) and $\bar{o}_b$ (observed rate in the bin). A perfectly calibrated model has $\bar{p}_b = \bar{o}_b$ in every bin.

Theorem 13.7 (Murphy decomposition [2])**.** *If forecasts take finitely many distinct values and the bins group identical forecasts, then with $n = \sum_b n_b$ the total number of predictions (*not *N, which is picks-until-mine) and $\bar{o} = \frac{1}{n}\sum_b n_b \bar{o}_b$ the overall base rate,*

$$\underbrace{\frac{1}{n}\sum_i (\hat{p}_i - y_i)^2}_{\textit{Brier}} = \underbrace{\frac{1}{n}\sum_b n_b(\bar{p}_b - \bar{o}_b)^2}_{\textit{reliability}} - \underbrace{\frac{1}{n}\sum_b n_b(\bar{o}_b - \bar{o})^2}_{\textit{resolution}} + \underbrace{\bar{o}(1-\bar{o})}_{\textit{uncertainty}}. \tag{36}$$

Proof sketch. Within a bin the forecast is the constant $\bar{p}_b$, so the bin's mean squared error is $\frac{1}{n_b}\sum_{i\in b}(\bar{p}_b - y_i)^2$. Add and subtract $\bar{o}_b$ inside the square and expand; the cross term is $-2(\bar{p}_b - \bar{o}_b)\sum_{i\in b}(y_i - \bar{o}_b) = 0$ because $\bar{o}_b$ is the bin mean of y. That leaves $(\bar{p}_b - \bar{o}_b)^2 + \bar{o}_b(1-\bar{o}_b)$ per bin, using $\frac{1}{n_b}\sum_{i\in b}(y_i - \bar{o}_b)^2 = \bar{o}_b(1-\bar{o}_b)$ for binary y. Averaging over bins gives reliability plus $\frac{1}{n}\sum_b n_b \bar{o}_b(1-\bar{o}_b)$, and the same add-subtract trick on the latter around $\bar{o}$ turns it into $\bar{o}(1-\bar{o}) - \frac{1}{n}\sum_b n_b(\bar{o}_b - \bar{o})^2$. □

Read the three terms:

- **Uncertainty** $= \bar{o}(1-\bar{o})$ is a property of the *world*, not of the model. Nothing you do changes it. It is 0.1022 here, and it is the constant- predictor baseline.
- **Reliability** (lower is better) is how far the model's stated probabilities are from the truth *conditional on what it said*. This is exactly the gap between the two columns of a reliability table. A model with reliability 0 is honest.
- **Resolution** (higher is better, hence the minus sign) is how much the model's bins *separate* outcomes from the base rate. A model that says 0.8844 for everything has zero reliability error *and* zero resolution: perfectly honest, perfectly useless. Resolution is discrimination — the ability to tell cases apart.

So Table 3 is a picture of a large reliability term. Every bin under 0.9 under-predicting means every $(\bar{p}_b - \bar{o}_b)^2$ is large and all of the same sign: a systematic bias, which is exactly the kind of error a single scalar can fix. That is the argument for the tilt, made visible.

Computing (36) on our two headline models (using the ten printed bins of Table 12 and Table 13; the decomposition is exact only for constant-within-bin forecasts, so the reconstruction is approximate — it recovers 0.0879 against a true 0.0868 for the base engine and 0.0677 against 0.0670 for the shipped model, the residual being within-bin forecast variance):

model	reliability ↓	resolution ↑	uncertainty	≈ Brier
engine (raw conditional logistic)	0.0231	0.0374	0.1022	0.0879
shipped (shrunk + tilt)	**0.0051**	0.0396	0.1022	0.0677

Table 8: Where the improvement came from, on the 2024 fold. Reliability falls by a factor of four and a half; resolution barely moves. "Shipped" carries no room warp here because this fold has no causal curve (§13.3).

This table is the most informative single result in the paper. The entire gain from the tilt and the shrink is a **calibration** gain, not a **discrimination** gain. The model was always roughly as good at telling players apart as it is now; it was simply saying the wrong numbers. That is worth knowing for two reasons: it tells you the tilt is doing exactly what it was designed to do and nothing sneaky besides, and it tells you where the remaining headroom is. If you want a better engine, you need better *resolution*, and no scalar correction will ever supply it — which is precisely the argument for the opponent-aware Monte Carlo model of milestone 6.

13.3 The backtest

Data: three folds, one per draft

Definition 13.8 (Fold)**.** A *fold* is one completed draft together with the era-correct ADP board it is scored against. Folds are never pooled: different rooms, different round counts, different boards, and pooling would let one fold's tilt exponents be solved over another fold's prices.

fold	draft	teams	rounds	board	source
2024	1133489617308684288	12	15	177	FFC archive, half-PPR 2024
2025-a	1261824503076360192	12	16	387	FantasyPros 2025 export
2025-b	1253161474382102529	12	15	387	FantasyPros 2025 export

Table 9: The three folds. "board" is the number of source names that joined onto Sleeper ids. All three drafts are 12-team half-PPR snakes with `reversal_round` = 0.

They are three leagues, not one league over three seasons. This matters because the room curve of §10 is built from all three at once. Measured from the cached league user lists and reprinted by the tool on every run: 2025-a shares 9 of 12 managers with 2024 and is this user's own room; 2025-b shares 5 and belongs to somebody else; 2025-a and 2025-b share 6. All three carry `previous_league_id: null`, so nothing links them programmatically and the overlap is the only evidence there is. The three rooms' curves nonetheless agree closely — QB1 went at pick 21/29/19, TE1 at 24/21/25 — so the stranger league is not an outlier. But the supportable claim is *"casual home leagues take quarterbacks and tight ends earlier than national ADP"*, not "this room drafts weird".

The 2024 board: FFC's archive. `/api/v1/adp/half-ppr?teams=12&year=2024` returns the 2024 market — 178 players over 906 drafts, window 2024-08-31 to 2024-09-01. The rows carry the full current schema (`adp`, `stdev`, `times_drafted`, `high`, `low`), so per-player sigma, the shrink and the support floor are all directly backtestable rather than only σ_{def}. The draft ran on 2024-08-31, *inside* that window; the tool checks and reports this rather than assuming it.

The 2025 board: a hand-exported FantasyPros table, and what it costs. FFC's archive has no `year=2025` — all four formats answer "No ADP data found", rechecked 2026-07-30, while 2023 and 2024 both work. So the 2025 folds are priced against FantasyPros' overall ADP export: 389 rows, columns `Rank`, `Player (Bye)`, `POS`, `Yahoo`, `Sleeper`, `RTSports`, `AVG`. It is 0.5 PPR, which is the leagues' own scoring, so there is no format mismatch. It lives in the cache directory and not in the repository — it is FantasyPros' data and the repository is public — and a missing file simply skips those folds.

Three consequences follow, and the first is the important one.

1. **There is no `stdev`, no `times_drafted`, no `high`/`low`.** Every player on a 2025 fold therefore runs on $\sigma_{\text{def}} = 6.0$. Those folds **cannot referee** $\sigma_{\min}$, $\sigma_{\max}$, the shrink of §3.8 or the support floor of §15.2: on them the per-player, shrunk, floored and flat rows of Table 11 are literally one model printed five times. They *can* referee the tilt, EBest, the conditioning, the room warp and anything downstream of survival. The tool says which regime each fold is in, loudly, before printing a number, and refuses to run `-tune` on a flat fold.
2. **Comparing folds requires forcing the ffc fold flat too.** A per-player-sigma Brier on 2024 against a flat-sigma Brier on 2025 is two models on two drafts, and the difference would be uninterpretable. So the cross-fold table of §14.4 scores every fold with $\sigma \equiv 6.0$ and prints each fold's own best-available column beside it. Where the two are equal, the fold really was flat — visible proof rather than a claim.
3. **The board is more than twice as deep.** FFC publishes 178 names; FantasyPros publishes 389, for drafts of 180 and 192 picks. Hundreds of those players were never going to be taken by anybody, they survive trivially, and they lift the observed base rate from 0.8844 to 0.9603. *Briers from different folds are therefore not comparable as numbers* — only as distances from their own uncertainty floors, which is why the tool prints $\bar{o}(1-\bar{o})$ per fold and a percentage against it. Depth also has a specific consequence for §10.5, because the warp is indexed by rank *within a position*: the fifth receiver on a 178-name board and the fifth on a 389-name board are not the same player, and the warp's mean shift changes sign between the folds because of it.

Both the `AVG` column (the three-platform consensus, populated on all 389 rows) and the `Sleeper` column (the platform these drafts were actually run on, populated on 358) are selectable, and where the chosen column is blank the consensus stands in and the substitution is counted. `AVG` is the default. §14.4 scores both.

Vantages

Definition 13.9 (Vantage)**.** A *vantage* is a triple (draft, seat, round): standing at seat s's pick in round r, looking forward to seat s's pick in round $r+1$.

For every seat $s \in \{1, \ldots, T\}$ and every round $r \in \{1, \ldots, R-1\}$ we take from = myPick(r) and to = myPick$(r+1)$ using the engine's *own* snake arithmetic with mySlot = s. Rounds come off each draft's own settings and never a constant: the 2024 draft is 15 rounds and 2025-a is 16, so the fold counts are $12 \times 14 = 168$, $12 \times 15 = 180$ and $12 \times 14 = 168$ vantages respectively. Vantage ids are numbered within a fold, because the tilt is solved per vantage and two folds sharing an id would pool two different boards into one solve.

The prediction set, the prediction, and the label

At vantage (from, to), for each player j on the era board:

- **Availability predicate.** Include j if his actual draft position $d_j \geq$ from, or he went undrafted entirely. (If he was already gone, there is nothing to predict.)
- **Prediction.** $\hat{p}_j$ = the engine's `PSurviveAt` with `PickNo` = from and horizon to, using 2024 ADP and 2024 sigma. This is literally the shipped function, called with a state whose `PickNo` is set; it is not a re-implementation.
- **Label.** $y_j = 1$ if $d_j \geq$ to or he went undrafted — he really was still sitting there.

That yields **15,923** labelled predictions on the 2024 fold, 53,635 on 2025-a and 51,052 on 2025-b. The 2025 counts are larger mostly because the board is larger, not because the evidence is: see point 3 above.

The honest caveat: 12 seats multiply vantages, not evidence

This must be said plainly because the prediction count invites the wrong conclusion. Twelve seats over the same 180 picks gives twelve times the labelled data *for free*, and it is legitimate — snake arithmetic is seat-agnostic, so each seat is a genuinely different schedule of vantages over the same reality, and using all of them removes any dependence on which seat we happened to pick.

But it is **not** twelve times the *evidence*. Every seat is scoring the same 180 real picks. If Player X unexpectedly lasted to pick 90 in that draft, all twelve seats record that same surprise, at overlapping horizons. The predictions are heavily correlated and the *effective* sample size is far smaller than 15,923 — closer, in spirit, to one draft's worth of independent surprises.

Concretely, this means: do not compute a standard error from $n = 15{,}923$ and conclude that a -0.0009 Brier difference is significant. It is not, and we do not claim it is. The tool prints this caveat on every run.

The join must be exact-only

Matching 2024 FFC names to Sleeper player IDs uses the project's normal matching pipeline — but with the fuzzy stage *disabled*, and this is not fussiness.

The normal pipeline falls through to Levenshtein-distance matching within a position. A 2024 player who is missing from the *current* active pool (retired, cut, out of the league) would then silently map to a similarly-named active player — whose ID never appears in the 2024 pick list. He would therefore be labelled "survived" at every single vantage. That is not a gap in the data, it is a *bias*, and it points the same direction every time: it inflates the observed survival rate and flatters every model that predicts high.

So calibrate uses `Index.LookupExact` and prints anything it drops. On the 2024 board: 177 of 178 matched, 1 dropped. On the 2025 board: **387** of **389**, **99.5**%, 2 dropped — "Hollywood Brown", who is Marquise Brown to Sleeper and is the same single name the 2024 join loses, and "John Stephens Jr.", deep and teamless. Neither is fuzzed.

That 99.5% takes one specific trick, and without it the figure is 91.8%. The project's usual rule is *join defenses on the team abbreviation, never the name* — in the Sleeper dump all 32 defenses have `full_name: null`, so a name-based matcher misses every one. FantasyPros' export inverts the problem: it puts the *team name* in the name field ("Denver Broncos") and the literal string `DST` where a team code belongs. The abbreviation path therefore resolves to nothing at all and all 30 defenses drop. The fix is to fall through to the normalized team *name*, which Sleeper does carry, split across `first_name` and `last_name` ("Denver" / "Broncos"). The order matters as much as the fallback: a source that supplies a real code is still answered by the code, so nothing about the FFC join changes.

Leave-one-out is not enough: the curve must also be in the past

The room curve is built from this user's drafts — and one of them is the draft being scored. A warp built from the draft it is graded on has memorised the answer and would report a skill it does not have. So the scored draft is held out. That much was always true, and it is not sufficient.

Table 10 is the problem. At the clock, the live tool can only have drafts that have already happened. A fold priced off a curve built from its own future is therefore measuring a tool nobody can run — and this is

fold	started	leave-one-out curve	prior drafts
2024	2024-09-01	2025-a + 2025-b — *both a year later*	none
2025-b	2025-09-02	2024 + 2025-a — *2025-a is 2 days later*	2024
2025-a	2025-09-04	2024 + 2025-b — both earlier	2024, 2025-b

Table 10: Start times read from each draft's own Sleeper metadata. Under leave-one-out alone, the 2024 fold's curve is 100% posterior to the draft it prices and 2025-b's is half posterior.

not symmetric noise that averages out, because **the 2024 fold is the only one that ever preferred a capped warp** (§14.4), so the entire upper bound of the interval that defends the retracted cutoff came from a curve made of that draft's future.

So each fold now builds its curve from the drafts that started *before* it, and from nothing else. A draft with no start time is held out too: unknown order is not prior order. The consequence is severe and is the honest one — **the 2024 fold has no room curve at all**, because no cached draft precedes 2024-09-01, and its room-warp verdict simply cannot be produced in the regime the tool runs in. It still grades everything else: conditioning, the tilt, per-player sigma, the shrink, the support floor.

Both halves are *asserted*, not commented: `checkCurve` refuses to continue if a fold's curve contains the fold itself, contains a draft that started after it, or carries a different number of drafts than the split allowed. The failure mode is invisible in the output — a leaked or look-ahead curve simply makes the numbers better — so it cannot be left to a comment. `calibrate -lookahead` reinstates the old pool, prints a banner saying so, and exists only to reproduce the retracted numbers. The same rule is applied by `live -replay`, through the same function, so the frame a human eyeballs and the numbers in this paper cannot come from different pools.

One further subtlety the implementation gets right and which is easy to miss: the warp ranks players by ADP *within a position*, so the rank input must cover *every* row on the era board, including the one name the exact-only join dropped. Otherwise a missing name shifts every deeper player at that position up by one rank and warps him with the wrong $\mathrm{adp}_{\mathrm{room}}(P, k)$. Measured: the one dropped 2024 name (a WR ranked 37th) would have mispriced the 28 receivers behind him by 1.6 picks on average against a mean warp of 4.4 — an uncontrolled error inside the measurement the whole phase rests on.

Solving the tilt at a vantage

The tilt is not a per-player transform; it is one exponent solved over a whole vantage's board at once. So a flat list of predictions is not enough to compute it, and each row carries its vantage id. Crucially, when the results are broken out by segment (horizon, position, ADP depth), the exponent for the "13+ picks" rows is the one solved over the *whole board* at that vantage — re-solving over the filtered subset would invent a draft in which only long-horizon players exist.

The backtest contains a second implementation of the tilt solver, because the engine's own is unexported and reads its horizon off `NextPick()` — and a backtest vantage *stands on* my own pick, so the engine's own solve degenerates to $c = 1$ and cannot be driven to the right question from outside the package. Two copies of one model is a liability, so it is checked rather than assumed: on every run, both solvers run over one shared 60-player board and the maximum disagreement is printed (currently 0.0×10^{0}), and a test fails the build if they ever diverge past 10^{-12}.

13.4 The baselines

A model must beat all of these or the mathematics is theatre.

(a) **Constant.** Predict the global observed survival rate for everyone. This is the "no model at all" floor and, by Theorem 13.5, it scores exactly the uncertainty term $\bar{o}(1 - \bar{o})$.

(b) **Flat sigma.** Replace every per-player σ_j with $\sigma_{\mathrm{def}} = 6.0$. If this ties the engine, the whole stdev plumbing of §3.3 is decoration.

(c) **Unconditional.** Drop the $S(q)/S(p)$ ratio and use the raw curve $S_j(\mathrm{to})$. If this ties the engine, conditioning on "he is demonstrably still here" (§3.4) is decoration.

14 Results

Unless a subsection says otherwise, all numbers below are the live output of `pick6 calibrate` on the **2024 fold**: 15,923 predictions, 168 vantages, observed survival rate 0.8844. That fold is reported alone because

it is the only one with per-player spread, so it is the only one that can grade the whole model. §14.4 brings the other two in, in the one regime all three share.

One thing that fold can *not* grade is the room warp, and the reason is §13.3: no cached draft precedes it, so under the time-order rule it has no curve. Every `3a` row below therefore ties its unwarped counterpart — that is the result, printed rather than hidden, and §14.4 is where the warp is actually decided.

14.1 The model table

model	Brier ↓	log-loss ↓	mean predicted
engine (per-player sigma)	0.0868	0.3007	0.8231
engine + exactly-N tilt	0.0677	0.2288	0.8734
4b: shrunk sigma	0.0856	0.2895	0.8237
4b: shrunk sigma + tilt	0.0670	0.2250	0.8734
4b: support floor	0.0868	0.3007	0.8231
4b: support floor + tilt	0.0677	0.2288	0.8734
4b: shrunk + floor + tilt	0.0670	0.2250	0.8734
3a: room-warped ADP (full depth)	0.0868	0.3007	0.8231
3a: room warp + tilt	0.0677	0.2288	0.8734
3a: room warp + shrunk + tilt	0.0670	0.2250	0.8734
3a: room warp + shrunk + tilt	**0.0670**	**0.2250**	0.8734
3a: top-5 warp + shrunk + tilt (retracted)	0.0670	0.2250	0.8734
3a: room warp + flat 6.0 + tilt	0.0671	*0.2233*	0.8734
baseline: constant 0.884	0.1022	0.3580	0.8844
baseline: sigma 6.0 flat	0.0825	0.2675	0.8330
baseline: sigma 6.0 flat + tilt	0.0671	*0.2233*	0.8734
baseline: unconditional	0.1111	0.5159	0.7934

Table 11: Every model scored on the same 15,923 predictions. Observed survival 0.8844. The bolded row is what the tool runs. The four `3a` rows tie their unwarped counterparts because this fold has no causal room curve (§13.3); the warp's own evidence is Table 17. *Italicised* log-losses beat what ships, and the tool prints that marker itself — see point 3.

Five things this table says:

1. **The tilt is the single biggest win in the project**, by a distance. It moves Brier from 0.0868 to 0.0677 (−22%) and log-loss from 0.3007 to 0.2288 (−24%), and it pulls mean predicted survival from 0.8231 to 0.8734 against an observed 0.8844. A one-parameter correction solved from a budget constraint, with no fitting to the labels whatsoever.
2. **Conditioning is load-bearing.** The unconditional baseline is the worst row in the table — worse than the constant predictor on Brier (0.1111 vs 0.1022) and catastrophically worse on log-loss (0.5159 vs 0.3580). §3.4 is not a refinement, it is the difference between a model and an anti-model.
3. **Per-player sigma does not, on its own, beat a flat sigma — and this is the most uncomfortable row in the table.** Baseline (b) exists to test whether the whole stdev pipeline of §3.3 earns its keep. It does not clear the bar cleanly. Untilted, flat $\sigma = 6$ beats per-player sigma on *both* metrics (0.0825/0.2675 against 0.0868/0.3007). Tilted, flat still edges it (0.0671/0.2233 against 0.0677/0.2288). Only once shrinkage is applied does per-player sigma draw level (0.0670/0.2250 — better on Brier by 0.0001, worse on log-loss by 0.0017), and **there it stays**.

   The room warp used to be the thing that broke the tie: with the cap on top, the full stack scored 0.0660/0.2222 and beat flat+tilt on both. That number required a curve built from this fold's future (§13.3) and is gone. What the tool prints today is the like-for-like comparison it always should have: the shipped stack with every σ forced flat is the `3a: room warp + flat 6.0 + tilt` row, and on this fold it **beats what ships on log-loss**, 0.2233 against 0.2250. `calibrate` marks that row itself rather than leaving it to be noticed. So on the fold with the only per-player spread in the project, the per-player pipeline does not win on either metric that a reader can check.

   The honest reading: *the per-player spread is real information, but the raw conversion* (12) *over-trusts it badly enough to be a net loss, and most of what shrinkage buys is undoing that damage rather than adding skill.* The Jets-defense case of §3.8 is not an anecdote — it is the whole effect, and there are 53 players like him on the board. On this evidence, running the tool with $\sigma \equiv 6.0$ and the tilt would cost you almost nothing. Per-player sigma stays because it is right in principle, because the shipped stack does win, and because a constant scale cannot possibly be right for both a $\varsigma = 0.4$ receiver and

a $\varsigma = 39.2$ kicker — but "the backtest demanded it" would be false, and one draft is not enough to settle it either way.

4. **The support floor does literally nothing.** Identical to four decimals with and without, tilted and untilted. §15 explains why, and it is structural.
5. **The room warp is absent from this fold entirely**, and it used to be the headline of it. Under leave-one-out alone the full-depth warp lost here (0.0671/0.2327) while the capped warp won (0.0660/0.2222) — and both of those curves were built from drafts that ran a year after this draft. Under the time-order rule the fold has no curve, so all four `3a` rows tie their bases and the warp is decided in §14.4 on the two folds that have one. `calibrate -lookahead` reprints the old numbers; nothing else does.

14.2 Calibration, before and after

bin	engine	+tilt	+shrunk	shipped	observed	n
0.0–0.1	0.0274	0.1535	0.1670	0.1670	0.3413	1383
0.1–0.2	0.1481	0.4130	0.4154	0.4154	0.6432	426
0.2–0.3	0.2495	0.5304	0.5269	0.5269	0.6520	319
0.3–0.4	0.3475	0.6163	0.6082	0.6082	0.7021	292
0.4–0.5	0.4491	0.7028	0.6934	0.6934	0.7190	306
0.5–0.6	0.5496	0.7667	0.7581	0.7581	0.7419	279
0.6–0.7	0.6504	0.8284	0.8211	0.8211	0.7710	310
0.7–0.8	0.7527	0.8855	0.8800	0.8800	0.7995	414
0.8–0.9	0.8544	0.9347	0.9309	0.9309	0.8678	643
0.9–1.0	0.9931	0.9970	0.9967	0.9967	0.9842	11551

Table 12: Bins are set by the *engine's* prediction so all four models share the same rows and are directly comparable within a row. Compare each model's column against the observed column. The last two columns are identical because this fold has no causal room curve (§13.3), so "shipped" *is* "+shrunk" here.

Read the low bins first, because that is where the original model was broken. In the bottom bin the engine said 0.027 against an observed 0.341; the tilted models say 0.15–0.17. Still under-confident, but the error has shrunk from a factor of 12 to a factor of 2.

Now read the high bins, because that is the cost. In the 0.6–0.9 bins the tilted models *over*-predict where the engine under-predicted: at bin 0.7–0.8 the engine said 0.753 against 0.800 observed (too low), the tilted model says 0.886 (too high). The tilt is a global correction and it cannot fix the shape of the curve, only its level; pushing the badly-wrong low bins up necessarily pushes the nearly-right high bins past the mark. The trade is strongly worth it — Table 8's reliability term falls by a factor of four and a half — but it is a trade, and it is visible here.

Table 13 is the shipped model graded on *its own* bins, which is the honest way to read one model's calibration rather than compare four.

bin	mean predicted	observed	n
0.0–0.1	0.0376	0.1417	501
0.1–0.2	0.1513	0.3668	368
0.2–0.3	0.2497	0.4706	340
0.3–0.4	0.3497	0.5981	321
0.4–0.5	0.4512	0.6316	323
0.5–0.6	0.5502	0.6589	343
0.6–0.7	0.6517	0.7209	387
0.7–0.8	0.7525	0.7388	448
0.8–0.9	0.8554	0.7892	664
0.9–1.0	0.9931	0.9782	12228

Table 13: The shipped model, binned by its own prediction. Still under-confident below 0.7 and slightly over-confident above it, but nowhere near the original's one-directional collapse.

14.3 Segments

The tails are where the tool earns its keep, so they get reported separately.

Observations:

segment	n	observed	Brier (tilted → +shrunk)	log-loss
by horizon				
≤ 6 picks	3804	0.9706	0.0255 → 0.0254	0.1044 → 0.1041
7–12 picks	3919	0.9132	0.0629 → 0.0625	0.2176 → 0.2161
13+ picks	8200	0.8307	0.0896 → 0.0884	0.2918 → 0.2853
by position				
QB	2447	0.9166	0.0962 → 0.0970	0.2961 → 0.2966
RB	4181	0.8627	0.0586 → 0.0589	0.1863 → 0.1861
WR	4752	0.8554	0.0675 → 0.0658	0.2494 → 0.2450
TE	1742	0.8961	0.0884 → 0.0849	0.3281 → 0.3101
K	1458	0.9328	0.0485 → 0.0487	0.1571 → 0.1544
DEF	1343	0.9285	0.0387 → 0.0379	0.1137 → 0.1110
by ADP depth				
rounds 1–3 (ADP ≤ 36)	789	0.5057	0.1077 → 0.1073	0.3390 → 0.3357
rounds 4–8 (ADP ≤ 96)	4521	0.8354	0.0848 → 0.0852	0.2900 → 0.2886
rounds 9+	10613	0.9335	0.0574 → 0.0562	0.1945 → 0.1896

Table 14: Segments, showing the effect of adding shrinkage to the tilted model.

- **Short horizons are easy and long ones are hard**, as they must be: at ≤ 6 picks the observed survival rate is 0.97 and the Brier is 0.0255; at 13+ picks the rate is 0.83 and the Brier is 0.0884. The tool is doing most of its real work in the third row.
- **Rounds 1–3 are the genuinely uncertain band**, and they show why a Brier score must always be read against its own segment's base rate. Observed survival there is 0.5057 — an almost perfect coin flip — so that segment's uncertainty floor is $0.5057 \times 0.4943 = 0.2500$, the full coin-flip ceiling. The model scores 0.1073 against it: less than half. That is by some distance the strongest performance in the table, even though 0.1073 is numerically the *worst* Brier of the three depth rows. Judged against the global floor of 0.1022 it looks like the model's weak spot; it is the opposite. This is the segment where a draft is won, and the model is extracting real signal from it.
- **Quarterbacks are the worst-calibrated position** (predicted 0.841 against observed 0.917 under shrunk+tilt), and the room warp is aimed squarely at them. It is the one position it helps on both metrics on both causal folds — Brier 0.0379 → 0.0375 and log-loss 0.1183 → 0.1161 on 2025-a, 0.0384 → 0.0375 and 0.1228 → 0.1184 on 2025-b. It cannot be graded here, because this fold has no causal curve. §10.5 explains the mechanism: it is the rank-versus-player distinction.
- **Tight ends benefit most from shrinkage** (Brier 0.0884 → 0.0849, log-loss 0.3281 → 0.3101), which is what you would expect — it is a shallow, thinly-sampled position where raw spreads are noisiest.

14.4 The second and third folds

Every result above rested on one draft night. Two more are now scorable, and this subsection is what they say. The headline: **the model's core — conditioning and the tilt — replicates; the room warp's cutoff does not.**

Dated caveat (2026-08-11): the 2025 numbers in this subsection were produced on the hand-exported FantasyPros board, which was the only 2025 board that existed when they were run. FFC began serving 2025 on 2026-08-11, `calibrate` *now prefers it, and current runs land on different absolute numbers for both 2025 folds — different board, different depth, real spread. The findings here (replication of the tilt, the cutoff's failure, the depth control) were about* comparisons within a fold *and survived the board change; the absolute figures did not need to. §12 carries the FFC-priced present.*

The only honest comparison

Table 15 scores all three folds with $\sigma \equiv 6.0$, because a board with no `stdev` can run in no other regime. Each fold's own best-available column sits beside it; the two are equal exactly when the board carries no spread, which is why the equality is left visible instead of collapsed away.

Do not read down the Brier column. A 389-name board against a 192-pick draft is mostly players nobody was ever taking; they survive trivially, they lift $\bar{o}$ from 0.8844 to 0.9603, and they shrink every Brier on that row for free. The only cross-fold quantity this data supports is each fold's distance from its *own* uncertainty floor $\bar{o}(1-\bar{o})$:

fold	board	depth	picks	n	$\bar{o}$	Brier flat → own	log-loss flat → own
2024	FFC	177	180	15,923	0.8844	0.0671 → 0.0670	0.2233 → 0.2250
2025-a	FantasyPros avg	387	192	53,635	0.9603	0.0197 → 0.0197	0.0698 → 0.0698
2025-b	FantasyPros avg	387	180	51,052	0.9607	0.0209 → 0.0209	0.0790 → 0.0790

Table 15: The shipped model across folds. "flat" forces every σ_j to σ_{def}; "own" is that fold's best available. The 2025 rows are identical across the arrow because those boards have no spread to shrink.

fold	floor $\bar{o}(1-\bar{o})$	flat model	% of floor
2024	0.1022	0.0671	66%
2025-a	0.0381	0.0197	52%
2025-b	0.0377	0.0209	55%

The model pays 52–66% of a no-model baseline on all three. That is the replication result, and it is the one that mattered most: nothing about the survival machinery falls over on a second room, a second season and a different price source.

What replicates, and what does not

- **Conditioning replicates.** The unconditional baseline is worse than the engine on both metrics on every fold (0.0237/0.0854 against 0.0207/0.0732 on 2025-a; 0.0247/0.0989 against 0.0216/0.0803 on 2025-b). §3.4 is load-bearing everywhere.
- **The tilt replicates, and shrinks.** It improves both metrics on all three folds — Brier 0.0868 → 0.0677 and log-loss 0.3007 → 0.2288 on 2024, 0.0207 → 0.0197 and 0.0732 → 0.0717 on 2025-a, 0.0216 → 0.0210 and 0.0803 → 0.0798 on 2025-b — so the sign replicates cleanly and the structural correction is confirmed. The magnitude collapses by a factor of roughly twenty, and the reason is visible in the tilt report rather than inferred. On 2024 the model expected 16.8 removals against a 12-pick window, so the median exponent solved to 0.44; on 2025-a it expected 12.0 against 12 and on 2025-b 11.8 against 12, with median exponents 1.08 and 1.08 — the correction had almost nothing to correct. That is the tilt behaving exactly as designed: it is a constraint, not a fitted parameter, and it does nothing when the constraint is already satisfied.

  There is a fragility underneath that, and the depth control found it. **On a board truncated to roughly the size of the draft, the tilt can make log-loss worse**, and it does so on both 2025 folds:

fold (board cut to 178)	untilted	tilted	expected	actual	clamped
2025-a	0.0563 / 0.1778	0.0561 / *0.2118*	11.4	10.7	5 of 180
2025-b	0.0587 / 0.1914	0.0581 / *0.1923*	11.6	10.8	**0 of 168**

  Brier / log-loss; "expected" is the removals the untilted model predicted per vantage and "actual" what the board really lost, both against a target of $N = 12$.

  Two mechanisms, and it matters that they are two, because only one of them has an obvious fix. **The first is the target itself.** N counts every pick in the window, but the board only holds players some ADP source ranked and the rest of the picks go to names nobody priced. Truncation makes that leak much worse — 12 picks now remove only ≈ 10.7 board players — so the tilt aims at a budget the board cannot supply and over-corrects, pulling mean predicted survival *below* the observed rate. Aiming at the observed count instead would be scoring the labels against themselves, so the tool prints the gap on every run and leaves it. **The second is the clamp**, which is that same mechanism at its extreme: on 2025-a five of 180 vantages admit no c at all, the solve pins at `TiltCMax` = 64, and every survival on that vantage is raised to the 64th power, i.e. crushed toward zero. That is where the large 0.1778 → 0.2118 move comes from.

  The distinction is load-bearing, because the obvious fix addresses only the second. 2025-b regresses on log-loss with **zero** clamped vantages, so falling back to $c = 1$ on a clamped vantage — which would still be more honest than falling back to 64 — would not have rescued it. This is a truncated diagnostic board and not a fold result, but the regime is exactly where the shipped board lives: FFC's 2026 half-PPR feed is 201 names against a 192-pick draft. See §16.
- **The room warp's cutoff does *not* replicate, the direction reverses, and the fold that disagrees turns out to have been priced off its own future.** This is the significant negative and §10.5 now carries it.

fold	curve	full-depth warp	top-5 warp	mean shift	per-position verdict at $K = 5$
2024	*none*	—	—	—	cannot be scored
2025-a	2024 + 2025-b	0.0197 → **0.0175** 0.0717 → **0.0616**	0.0197 → 0.0197 0.0717 → 0.0698	−3.64	RB, WR, DEF regress
2025-b	2024	0.0210 → **0.0200** 0.0798 → **0.0738**	0.0210 → 0.0209 0.0798 → 0.0790	−2.96	RB, WR, K, DEF regress
and the same table under `-lookahead`*, i.e. the retracted regime:*					
2024	2025-a + 2025-b	0.0670 → 0.0671 0.2250 → 0.2327	0.0670 → **0.0660** 0.2250 → **0.2222**	+2.39	5 of 6 better, WR tied
2025-b	2024 + 2025-a	0.0210 → **0.0189** 0.0798 → **0.0701**	0.0210 → 0.0208 0.0798 → 0.0787	−2.96	RB, WR, K, DEF regress

Table 16: Brier above log-loss in each cell, against the pre-warp shrunk+tilt model. "mean shift" is the average price move in picks, positive meaning the warp prices players later. The top block is the shipped regime, in which every curve predates the draft it prices; the bottom block adds back the drafts that had not happened yet. **The only fold on which the top-5 warp ever beat the full-depth warp is in the bottom block.**

The top block is two folds, not three, and that is the finding. 2024 has no row because nothing in the cache precedes it (§13.3); 2025-b's curve shrinks from two drafts to one, which is why its full-depth numbers move from 0.0189/0.0701 to 0.0200/0.0738 — a smaller pool with a lighter blend weight, $w = 1/3$ rather than 1/2. Both remaining folds prefer the uncapped warp, on both metrics, by margins ten times anything the cap moves.

The confounds, named — and one of them is fatal

The two 2025 folds agree with each other and disagreed with 2024, which rules out one unlucky night as the explanation. Two candidate confounds were on the table.

Board depth: tested, and it is not the answer. The suspicion was reasonable: (34) is indexed by rank *within a position*, and the warp's mean shift flips sign exactly where the board depth changes (+2.39 picks on a 177-name board, −3.64 and −2.96 on a 387-name one). On the deeper board the k-th player at a position sits at a much later ADP, so the same $\mathrm{adp}_{\mathrm{room}}(P, k)$ pulls him *earlier* instead of later — the exact mechanism of §10.5's structural argument, running backwards.

So it was tested rather than believed. `calibrate -depth 178` truncates every era board to its 178 cheapest players before joining, which puts all three folds on boards of the same size and, as a by-product, on the same observed survival rate: $\bar{o}$ becomes 0.8844/0.8800/0.8852 and the uncertainty floors 0.1022/0.1056/0.1016, so for once the Brier column *can* be read downwards. The disagreement is unchanged:

fold (board cut to 178)	unwarped	top-5 warp	full-depth warp
2025-a	0.0561 / 0.2118	0.0559 / 0.2103	**0.0502 / 0.1929**
2025-b	0.0581 / 0.1923	0.0579 / 0.1913	**0.0559 / 0.1815**
2024 (`-lookahead`)	0.0670 / 0.2250	**0.0660 / 0.2222**	0.0671 / 0.2327

At matched depth the full-depth warp still wins decisively on both 2025 folds and the top-5 restriction still fails its per-position gate on both, and 2024 — when its look-ahead curve is put back so that it has one at all — still points the other way. **Board depth does not explain the disagreement.** It remains a real effect on the mean shift, which is −1.24 and −0.99 picks on the truncated 2025 boards against −3.64 and −2.96 untruncated, so truncation moved it a long way toward 2024's +2.39 without moving the verdict at all.

Time order: not tested, because it is not a confound but a defect. The other difference between 2024 and the 2025 folds was not a property of the data at all. Under leave-one-out alone, 2024's curve was built from two drafts that ran a year later (§13.3); the 2025 folds' curves were mostly or entirely built from drafts that preceded them. So the fold that disagreed was also the only fold priced off information no live board could have. Holding each fold's future out removes 2024's curve entirely, and with it the disagreement — **not because the folds now agree, but because only the agreeing folds are left.** That is a smaller claim, and it is the true one.

What remains unseparable. Season and vendor are perfectly collinear. 2024 is FFC, a distribution over 906 observed drafts; 2025 is a FantasyPros three-platform ranking consensus. Every difference between the folds is simultaneously a difference of year, of room and of pricing methodology, and no truncation fixes that. Since the only FFC-priced fold is also the only acausal one, this confound and the last are now the same fold, which is as weak as an experimental design can be.

The cutoff is not identified

The stronger statement, and the one that changes what this paper is allowed to claim: `calibrate` now sweeps K on every fold that has a causal curve, rather than on the one it was chosen from.

	K=1	2	3	4	5	6	8	12	20	∞
2025-a Brier	.0197	.0197	.0196	.0196	.0197	.0194	.0194	.0191	.0184	**.0175**
2025-b Brier	.0210	.0210	.0210	.0209	.0209	.0208	.0207	.0207	.0205	**.0200**
folds won	0/2	0/2	1/2	2/2	1/2	2/2	2/2	2/2	2/2	**2/2**
folds not worse	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
folds clean	1/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2
the retracted regime, `-lookahead`, for comparison:										
2024 Brier	.0665	.0663	.0662	**.0660**	**.0660**	.0661	.0661	.0674	.0672	.0671
folds not worse	3/3	3/3	3/3	3/3	3/3	3/3	3/3	2/3	2/3	2/3

Table 17: `RoomWarpTopK` swept against the unwarped model. "won" is better on both metrics; "not worse" allows a tie at the printed four decimals; "clean" additionally requires that no position regress — the full ship gate. Both causal folds are monotone in K and want no cap. The interior optimum at $K = 4$–5 exists only on 2024, and only when 2024 is allowed a curve built from its own future.

No cutoff clears the full gate on any set of folds. The per-position half of the gate — the half that produced the sentence "no position regresses", which is what justified shipping the warp into survival — is a 2024-only result at every K that was tried, and 2024 has no causal curve. That claim is retracted twice over.

The band is retracted too, and this is the part that is easy to miss. It used to read "every $K \in [1, 8]$ is never worse on any fold, while $K \geq 12$ and uncapped are worse on 2024" — and that upper bound of 8 was set *exclusively* by 2024, i.e. by the look-ahead fold. On the two causal folds **every** K the sweep tries is never worse, including no cap at all, so the band no longer excludes anything and cannot be read as an argument for capping.

The cap was held for one round of evidence, then dropped. It survived the first re-gate on (i) $K = 5$ being never worse than not warping on either causal fold, and (ii) the structural argument of §10.5 — a ranked list runs deeper than any finite draft, so past the room's appetite rank-to-room-pick prices players later than the market *by construction*. Holding it there was the right call at the time: moving K to whichever value won a cross-fold tally would have repeated the exact error that produced the retracted claim.

What settled it was that the structural argument is not merely an argument — it predicts a *measurable* thing, namely that the warped tail prices players worse. On both folds that can test it causally, at native depth and at a common board size, it does not: uncapped wins both metrics and regresses fewer positions than $K = 5$ does (1 and 2 against 5 and 5). Preferring the mechanism story to that is preferring reasoning to measurement, which is the one thing this whole apparatus exists to stop. **The cap is gone; `-room` stays on, because uncapped is never worse on either causal fold.** What would restore it is a causal fold on which uncapped loses — ideally an FFC-priced one, to break the season/vendor collinearity.

Does league purity matter? No; sample size does

Threat 5 of §16 says (34) pools three different leagues. That is a testable design decision, so `calibrate` tests it: for each fold it rebuilds the curve from each *single* draft available to it in turn and from all of them, and scores the same predictions with each. The two single-draft rows are the controlled comparison — the blend weight is $w = n/(n + 2)$, so they share $w = 1/3$ exactly and differ only in whose room supplied the curve. The pooled row moves purity and weight together and is not a third point on the same line.

This table is now one fold wide, and that is the cost of §13.3. A fold can only be split over the drafts that preceded it: 2024 has none and 2025-b has one, so only 2025-a has a purity question at all. The three-fold version, which is what the conclusion below was originally drawn from, required every fold to be allowed its own future.

fold	curve built from	shared managers	top-5 warp	full-depth warp
2025-a	no warp	—	0.0197 / 0.0717	0.0197 / 0.0717
	2024 only	9/12	0.0197 / 0.0701	0.0185 / 0.0649
	2025-b only	6/12	0.0197 / 0.0708	0.0181 / 0.0645
	both	mixed	0.0197 / 0.0698	**0.0175 / 0.0616**
the retracted regime, `-lookahead`*:*				
2024	no warp	—	0.0670 / 0.2250	0.0670 / 0.2250
	2025-a only	9/12	0.0662 / 0.2218	0.0669 / 0.2281
	2025-b only	5/12	0.0664 / 0.2241	0.0670 / 0.2293
	both	mixed	**0.0660 / 0.2222**	0.0671 / 0.2327
2025-b	no warp	—	0.0210 / 0.0798	0.0210 / 0.0798
	2024 only	5/12	0.0209 / 0.0790	0.0200 / 0.0738
	2025-a only	6/12	0.0208 / 0.0789	0.0193 / 0.0721
	both	mixed	0.0208 / 0.0787	**0.0189 / 0.0701**

Table 18: Brier / log-loss. "Shared managers" is the overlap between the league that supplied the curve and the league being scored. The scored draft is excluded from every curve on every row; in the top block, so is every draft that started after it. Only the top block is reproducible by a tool standing at the clock.

Two things fall out, and they point the same way — but read the caveat that follows them, because most of the evidence for both lives in the bottom block.

Purity is at best a second-order effect. Where the higher-overlap source wins it wins by very little (2024: 0.0662/0.2218 from the 9/12 room against 0.0664/0.2241 from the 5/12 room; 2025-b: the 6/12 source beats the 5/12 source on both). Where it loses, it loses just as small (2025-a: the 6/12 *stranger* league beats the 9/12 own-room source under the full-depth warp, 0.0181/0.0645 against 0.0185/0.0649). Three comparisons, two for purity and one against, all of them smaller than the gap between the two warp variants on the same row.

Sample size is a first-order effect. The pooled two-draft curve beats *both* of its own single-draft components on 5 of the 6 fold-by-variant cells. The one exception is the full-depth warp on 2024, which is the variant that fails on that fold anyway.

The caveat. Of those three purity comparisons and six pooling cells, exactly one comparison and two cells — 2025-a's — survive the time-order rule. The rest are rows in which a fold was priced off drafts that had not happened yet. Since the claim being made is about *whose room supplies the curve* and not about *when*, the bottom block is not worthless evidence for it: manager overlap is a fact about the leagues, not about the calendar. But it is evidence from a regime the tool cannot run in, and one surviving comparison is not a result. Treat "sample size dominates purity" as a plausible reading that the causal data neither confirms nor contradicts.

The room curves themselves point the same way and do not depend on any of this: the three leagues take QB1 at picks 21, 29 and 19 and TE1 at 24, 21 and 25. (34) is a portrait of casual home leagues, not of one room.

A side result: the platform's own ADP beats the consensus

The FantasyPros export prices every player twice, and both folds were run on Sleeper. Both columns go through the same warp, the same sigma and their own per-vantage tilt, so only the price differs.

fold	Brier (AVG)	Brier (Sleeper)	log-loss (AVG)	log-loss (Sleeper)
2025-a	0.0197	**0.0150**	0.0698	**0.0495**
2025-b	0.0209	**0.0177**	0.0790	**0.0616**

Sleeper's own column wins on both metrics on both folds, and by margins an order of magnitude larger than anything the room warp moved. Quarterbacks carry most of it (2025-a Brier 0.0375 → 0.0190; 2025-b 0.0375 → 0.0270), receivers and tight ends the rest. The comparison is over the same prediction set by construction: every scored row carries both prices, and the only dilution is the 31 export rows where the Sleeper column was blank and the consensus stood in — on those the two models are literally the same number, which weakens the comparison rather than flattering it.

The one place it wobbles is the depth control. Truncated to 178 names, 2025-a still prefers Sleeper on both metrics (0.0559 → 0.0449 Brier, 0.2103 → 0.1779 log-loss), but 2025-b splits: Brier 0.0579 → 0.0522 better, log-loss 0.1913 → 0.2103 *worse*. So the effect is strongest on the deep board, where the consensus is

averaging three rankings of players none of the three platforms has much of an opinion about.

The intuition that a three-platform average should be better calibrated than any single ranking is nonetheless wrong here, and the natural reading is not "consensus ADP is bad". It is that the drafts and the column come from the same platform, so the column knows the platform's users — the same kind of local knowledge (34) is reaching for, from a far larger sample. This is the largest unexploited result in the paper: the tool's own board is priced off FFC and FantasyCalc, neither of which publishes a Sleeper-specific ADP, so there is nothing to switch on today. The scored default stays `AVG`, because a result this good on the platform's own drafts is also the one most likely to be measuring the room rather than the market, and because the 2024 board has no Sleeper column to check it against.

15 Negative results

Three ideas were built, measured, and did not survive. They get their own section because they are the best evidence that the measurement apparatus of §13 is doing real work — a backtest that only ever confirms is not a backtest — and because in two of the three cases the reason for failure is more instructive than any of the successes.

Each subsection ends with what evidence would overturn it. That is not a formality: two of the three are still in the codebase, exported and tested, one call site away from being switched back on.

15.1 The full-depth room warp

Derived in full in §10.5; listed here so the three negatives sit together.

What it was. Blend every player's national ADP toward this room's own rank-to-pick curve, at every depth. **Why it failed.** A ranked list runs deeper than any finite draft, so past the room's appetite the rank-to-room-pick map has no data — and there are more players in that unreliable tail than at the reliable head. The warp's mean *blended* shift on quarterbacks flipped sign: -1.7 to -3.7 picks on QB1 through QB5, but $+11.0$ **picks overall**, the opposite of the signal it was built to capture. **The score.** On the 2024 fold, Brier $0.0670 \to 0.0671$, log-loss $0.2250 \to 0.2327$; both worse. Restricted to the top five at each position it won there instead (0.0660/0.2222), and that is what ships.

Status. `RoomCurve.EffectiveADP` is still exported, still tested, and still scored as a row in the model table on every run — deliberately, so the next reader cannot assume the warp was never tried at depth. Its only caller is the backtest, and keeping it there is what made everything below visible.

What would overturn it: a second scorable season. That arrived, and it points the other way — and then the first fold's evidence turned out not to be admissible at all. This is the one entry in this section whose verdict has actually moved, so it is worth reading as written rather than as summarised.

One: it did not replicate. On both 2025 folds (§14.4, Table 16) the full-depth warp *wins* on both metrics — $0.0197 \to 0.0175$ and $0.0717 \to 0.0616$ on 2025-a, $0.0210 \to 0.0200$ and $0.0798 \to 0.0738$ on 2025-b — while the top-5 restriction that ships fails the per-position half of its own gate on both.

The obvious explanation was depth: the 2025 boards are 387 names against 2024's 177, so the k-th player at a position carries a much later ADP there, blending toward $\mathrm{adp}_{\mathrm{room}}(P, k)$ moves him earlier rather than later, and the mean shift flips sign with it ($+2.39$ picks on 2024, -3.64 and -2.96 on 2025). **That explanation was tested and it is wrong.** Truncating every board to 178 names (`calibrate -depth 178`) leaves the verdict exactly where it was: the full-depth warp still wins on 2025-a ($0.0561 \to 0.0502$, $0.2118 \to 0.1929$) and 2025-b ($0.0581 \to 0.0559$, $0.1923 \to 0.1815$), and still loses on 2024 when 2024 is given back the look-ahead curve it needs to have one at all. Depth moves the mean shift a long way toward 2024's — to -1.24 and -0.99 picks — and moves the result not at all.

Two: the 2024 verdict came from that draft's future. Both drafts that built its curve ran a year later (§13.3). With each fold's future held out, 2024 has no curve, the "$0.0670 \to 0.0671$" above is reproducible only under `calibrate -lookahead`, and **no causal fold rejects the full-depth warp.** The mechanism argued above is untouched by this — it is an argument about ranked lists and finite drafts, not a score — but the measurement that agreed with it is gone.

So the mechanism in §10.5 is not depth-dependent in the way that would have saved the top-5 restriction, and **this negative result should now be read as unsupported rather than merely in doubt.** What is left to blame for the original disagreement is the one thing this data cannot pull apart, and it has collapsed onto a single fold: season, vendor and causality are all collinear, 2024 being a sample of 906 real drafts, priced a year before the drafts that warped it, while 2025 is a ranking consensus with curves that predate it. What is still needed is a fold that is FFC-priced, is not 2024, and has a cached draft before it.

15.2 The support floor

What it was. FFC reports `high`, the earliest pick anyone in its sample ever took a player at. If my horizon is at or before `high`, then surviving to it means surviving a window in which *no observed draft ever removed him*. That is a zero- event observation, and there is a clean bound for those.

Definition 15.1 (Rule of three)**.** If an event occurred zero times in n independent trials, an approximate 95% upper bound on its underlying rate is $3/n$ [3].

Proposition 15.2 (Where the 3 comes from)**.** *If the true rate is π, the probability of observing zero events in n trials is $(1-\pi)^n$. Setting this to 0.05 and solving,*

$$n\log(1-\pi) = \log 0.05 \quad\Longrightarrow\quad \pi = 1 - e^{\log(0.05)/n} \approx \frac{-\log 0.05}{n} = \frac{2.996}{n}$$

for small π, using $\log(1-\pi) \approx -\pi$. Hence $3/n$.

So the floor is

$$p \;\longleftarrow\; \max\left(p,\; 1 - \frac{3}{\max(n,3)}\right) \qquad \text{whenever horizon} \le \texttt{high}, \tag{37}$$

where $n =$ `times_drafted`. The $\max(n,3)$ keeps a two-draft player from producing a negative floor: at $n=1$ the bound $1-3/1=-2$ is the rule of three saying it has nothing to offer, not a probability.

It only ever *raises* a survival probability, never lowers one — an observed early pick is evidence a player *can* go early, not evidence about how often.

Why it failed, and the reason is beautiful. It qualifies on 9,817 of the 15,923 backtest rows — 62% of them. It **binds on zero**. Against the shrunk curve it binds on two. Brier and log-loss are identical to four decimal places with it and without it, tilted or untilted.

The reason is structural, not seasonal:

Proposition 15.3 (The floor's window always sits where the curve is already near 1)**.** *`high` is a* minimum *over drafts and `adp` is a* mean *over the same drafts. Hence `high` $\le$ `adp` always. Therefore the floor's applicability window {horizon $\le$ `high`} lies entirely at or before the player's own price — and by (9), $S_j(p) \ge 1/2$ for all $p \le a_j$, with S_j climbing rapidly toward 1 as p falls below a_j.*

Proof. The minimum of a finite set of positive numbers is at most their mean, so `high` $\le$ `adp`. Then horizon $\le$ `high` $\le a_j$ gives $(\text{horizon} - a_j)/\sigma_j \le 0$, so by (9) the survival is at least $1/(1+e^0) = 1/2$; the conditional version (13) is at least as large again, since dividing by $S_j(p_{\text{now}}) \le 1$ only increases it. □

So the bound is competing against a curve that already reads near 1 in the entire region where the bound applies. It is dominated almost everywhere *by construction*. It was a good idea aimed at a real failure — a tight curve inventing risk the sample explicitly denies — and the failure simply does not occur in the region where this particular bound can speak.

Measured a second way, on the shipped 2026 board rather than the 2024 backtest: replayed over all twelve seats and every round, it bites on 16 of 34,104 predictions, the largest single move being 74.5% → 82.4%.

Status. Built, exported as `engine.SupportFloor`, unit-tested, graded on every calibrate run, and **deliberately not wired in**. It is one call site away: wrap `PSurviveAt`'s return. The ordering would matter — the floor must come *before* the tilt, because the tilt balances a budget over the finished per-player numbers, and flooring afterwards would hand back mass the tilt had just balanced and break the exactly-N property the tilt exists for.

What would overturn it. A source whose `high` is not a per-draft minimum — for instance a reported 5th percentile of draft position, which can and does sit *after* the mean for a right-skewed player. Then Proposition 15.3 no longer applies and the window can land where the curve is genuinely uncertain. Alternatively, if calibrate ever starts reporting a bind count worth grading, rewire it.

15.3 The SigmaMin tuning artifact

This one is a methodology lesson rather than a modelling one, and it is the most transferable thing in this section.

What happened. `pick6 calibrate -tune` grid-searches $\sigma_{\text{def}} \times \sigma_{\min} \times \sigma_{\max} \times n_0$ — four axes, 594 combinations — and *today* it ranks them by the **tilted** Brier, which is the model that ships. It did not

always: before the tilt and the shrink existed there was neither a tilted column nor an n_0 axis, so it was three axes ranked by the plain Brier. In that state it recommended widening $\sigma_{\min}$ from 0.5 to about 14 — an enormous change, scoring 0.0690 against the then-current 0.0868.

Run *after* the tilt, $\sigma_{\min} = 14$ scores 0.0690 tilted as well — which is now **worse** than the shipped 0.0670. (It is still the untilted optimum. Both numbers reproduce on every run.)

Why. A very large $\sigma_{\min}$ flattens every tight player's curve, which raises his survival probability. That is the same correction the tilt makes, arrived at by a blunter route: widening the curve was compensating for the under-prediction bias of Table 3. Once the tilt removes that bias properly — globally, with one interpretable parameter, and without destroying the per-player spread information that $\sigma_{\min}$ was flattening away — the crutch becomes dead weight.

The lesson. *Tune the model you ship, not a model you no longer run.* An optimum found under one model configuration is not an optimum under another, and the two can disagree not because one search was wrong but because a later change absorbed the same error the earlier optimum was compensating for. Constants tuned against a superseded pipeline are not neutral leftovers — they are corrections for a bias that has already been corrected, applied twice.

What the tilted grid says now, which is not nothing

The tuner having been rebuilt to rank by the shipped metric, it is fair to ask what it recommends. The answer is honest and slightly awkward, so here it is in full: the tilted grid's optimum is *not* the shipped combination. Ranked by tilted Brier, the top of the table reads

σ_{def}	$\sigma_{\min}$	$\sigma_{\max}$	n_0	untilted Brier		tilted Brier
4.5	8.0	10.0	200	0.0750		**0.0667**
4.5	8.0	15.0	50	0.0749		0.0668
4.5	8.0	10.0	10	0.0749		0.0669
6.0	0.5	25.0	25	0.0856	0.0670	(shipped)

Those are the rows the tool prints, and it prints one row per distinct *printed* tilted Brier rather than one per grid point — several combinations tie at 0.0667 and differ only in the eighth decimal, so showing all of them would hide the shape of the curve behind four copies of the same number.

So the overturn condition stated below *is* partially met: with the tilt in place the grid finds 0.0667 against the shipped 0.0670. Three things about that margin, all checkable in the tool's own output.

1. **It is a flat-sigma model in disguise.** Blend at $n_0 = 200$ and the prior swamps every player's own measurement (median `times_drafted` on the 2024 board is 54); clamp the result to $[8, 10]$ and there is almost nothing left to vary. Measured on the 177 joined players: that row gives σ running 8.00 / 8.00 / 8.99 (min / median / max) with 152 of them pinned exactly at the floor, and the second row is barely milder at 8.00 / 8.00 / 10.81 and 151 pinned. Even the gentlest combination that ties the winner — $(4.5, 4.0, 15.0, 50)$, suppressed from the table above as a duplicate score — still pins 81. The grid is not discovering a better per-player scale, it is rediscovering flat sigma, which the results table already grades directly: "baseline: sigma 6.0 flat + tilt" scores 0.0671. A 0.0667 and a 0.0671 that are the same model in different clothes are not evidence for moving four constants.
2. **It sits on a grid edge.** $n_0 = 200$ is the top of the shrink axis, and `edgeWarning` prints "the winner sits on the edge of the shrink axis — widen the grid before believing it" on every run. An optimum at the boundary is the search running out of road, not a minimum.
3. **The margin is** 0.0003 **on one fold.** §16 says why that is not a number to move constants on. It used to be argued here that the room warp ships on 0.0009, three times larger, plus a cutoff plateau and a clean per-position gate — and all three of those have since been retracted (threat 1b), so the comparison is gone rather than reversed. What remains true is the arithmetic: 0.0003 between two models that are the same model in different clothes is not evidence for moving four constants.

Outcome. No constant was moved. $\sigma_{\min}$ is still 0.5, and the shipped row is printed in the grid on every run precisely so this decision stays visible rather than becoming folklore.

What would overturn it. A second scorable season on which the same region of the grid still wins, or an interior winner that is not simply flat sigma wearing four constants. The tuner still exists and still prints; it never auto-writes. Printing rather than writing is the whole reason any of this was catchable.

16 Threats to validity

A number without its caveat is a lie. These are ordered roughly by how much they should worry you.

1. Three drafts, and only one of them can grade the whole model. The backtest is three completed drafts (§13.3). That is better than the one it used to be, and the core **replicates**: the conditioning beats the unconditional curve on every fold, the tilt improves both metrics on every fold, and the model pays 52–65% of a no-model baseline on all three. Those claims are upgraded from one fold to three and this threat no longer governs them.

It still governs everything about σ, because the two 2025 folds are priced off a board with no per-player spread and cannot say anything at all about it (see 4 and 9). And it governs the room warp completely — see threat 1b, which is the more serious half of this and is really a defect rather than a limitation.

1b. The room warp has one fold of causal evidence, and the cutoff has none. This is the most serious thing in the paper and it is a defect that was found, not a limitation that was accepted. Leave-one-out held out the draft being scored and nothing else — so the 2024 fold, which ran on 2024-09-01, was priced off a curve built entirely from two drafts that ran a year later (§13.3, Table 10). The tool now holds each fold's future out too, which leaves 2024 with no curve at all, since nothing in the cache precedes it.

The consequences, in the order they hurt:

1. **The cutoff's entire case was that fold's.** $K = 5$ won on 2024 and only on 2024, and the "never worse for $K \in [1, 8]$" band had its upper bound of 8 set exclusively by 2024 as well. Both are gone. On the two causal folds every K the sweep tries is never worse, including no cap, so the band identifies nothing (Table 17).
2. **Both causal folds prefer the warp this paper originally rejected**, by margins ten times anything the cap moved — so the cap was removed and the board now warps at every depth the curve reaches. §10.5 says so in its own words.
3. **The purity result thins to one fold** (Table 18), because a fold can only be split over drafts that preceded it.
4. **The remaining causal evidence is entirely FantasyPros-priced 2025 data**, which is the same data threat 4 says cannot referee σ. So the one fold that can grade the whole model and the folds that can grade the warp are now disjoint sets.

`calibrate -lookahead` reprints the old regime under a banner, and it is the only way to obtain any of the retracted numbers. What would fix this properly is a draft older than 2024-09-01 in the cache, or an FFC-priced fold that is not 2024.

2. Twelve seats are correlated. 15,923 predictions on the 2024 fold sounds like a lot. It is 168 vantages over 180 real picks, and all twelve seats are scoring the same picks. The effective sample is far smaller than the count suggests. Do not compute a standard error from $n = 15{,}923$ — or from the 53,635 of 2025-a, where the count is inflated a second time by a board more than twice as deep as the draft.

2b. Two folds are compared across different board depths. FFC publishes 178 names, FantasyPros 389. A deeper board grades hundreds of players nobody was ever taking; they survive trivially and drag $\bar{o}$ from 0.8844 to 0.9603, which shrinks every Brier on that row for reasons that have nothing to do with the model. Table 15 therefore reports each fold's distance from its own uncertainty floor and nothing else, and §14.4 says plainly that the Brier column must not be read downwards.

This one is now *controllable*: `calibrate -depth N` truncates every board to its N cheapest players, and at $N = 178$ the three folds land within 0.005 of each other on $\bar{o}$ (0.8844/0.8800/0.8852) and their Briers become directly comparable. It was run, and it did not rescue (34). Note what the control does *not* fix: truncation makes the boards the same size, not the same kind of thing. 2024 is still a distribution over observed drafts and 2025 still a ranking consensus, and season and vendor remain perfectly collinear across the folds.

3. The ADP snapshot postdates the draft — and this is the optimistic case. FFC serves the trailing snapshot of a season's draft week. This draft ran on 2024-08-31, inside the 2024-08-31 – 2024-09-01 window, so the prices we score against are the prices the room actually had. But those prices already know

what the room knew: late injuries, camp news, the whole final week of information. Read every number in §14 as **the model running with final prices** — the best case, not a typical case. On draft day the tool runs on a snapshot that is at most a day old, which is close but not identical; the tool prints its own staleness in the footer for exactly this reason.

4. The 2025 board is not a sample of drafts, and carries no date. FFC's archive still has no `year=2025`, so the 2025 folds are priced off a hand-exported FantasyPros consensus. That is an average of three platforms' rankings, not a distribution over observed drafts: there is no `stdev` (so no per-player σ, no shrink, no support floor), no `times_drafted` (so no sample weight), and no snapshot window (so the era check can match by season and nothing finer — unlike 2024, where the draft is verifiably inside the price window). It is also a file a human downloaded, which means it is frozen at whatever preseason moment they downloaded it and refetching is a chore rather than a request. Recheck FFC's archive near draft day; if 2025 lands there, re-run and re-gate §10.5 against a board that can answer all of the above. That is still the single highest-value follow-up in the project.

5. The room curve is three drafts *of three different leagues*. 552 picks across three nights. The docs used to imply one league across three seasons and that is wrong: measured manager overlap is 9/12 between 2024 and 2025-a, 5/12 between 2024 and 2025-b, 6/12 between the two 2025 leagues, and all three carry `previous_league_id: null`. So (34) is not a portrait of "this room" at all — it is a portrait of casual home leagues in general, which is a weaker and broader claim than the one §10 is named for. The pseudo-count is an explicit acknowledgement of the sample size ($w = 0.6$ at $n = 3$) and the top-5 restriction is a second one; neither makes three nights into a sample.

The membership question itself has been measured (Table 18) and the answer is reassuring for the pooling and deflating for the naming: at a fixed blend weight, a curve from a 5–6/12-overlap stranger league scores about as well as one from the 9/12-overlap own room, and the pooled two-draft curve beats both of its components on 5 of 6 cells. **But five of those six cells come from folds that were allowed to build curves out of their own future** (threat 1b); only 2025-a's row survives the time-order rule, and one comparison is not a result. Read "sample size dominates purity" as plausible and unconfirmed rather than measured. Pooling different leagues is therefore still the default — the room curves themselves agree closely, which is independent of any of this — but it is tolerated rather than demonstrated, and only because these leagues appear interchangeable. A serious drafter's room would be a different curve and none of this evidence would transfer to it.

6. Values are imported; the project makes no projections. Every v_j comes from FantasyCalc, every a_j and ς_j from FFC, every tier from a human or from a value gap. If the imported value curve is wrong about a player, the engine is wrong about him in exactly the same way and has no independent means of noticing. This is a deliberate scope decision — "no projections of our own" is the project's first non-goal — but it means the engine's ceiling is the value source's ceiling. Urgency can only tell you what waiting costs *in the value source's own units.*

7. Independence is assumed twice and is false both times. Theorem 5.2 and (29) both assume players survive independently. They do not: a run on running backs removes several at once, and the manager who takes one is thereby not taking another. The tilt corrects the aggregate consequence of this (the removal budget) but not the correlation structure. Modelling that properly is exactly what the Monte Carlo opponent model of milestone 6 is for, and until it lands the engine's tier-hold probabilities in particular should be read as somewhat too confident in both directions.

7b. On a board no deeper than the draft, the tilt aims at a budget the board cannot supply — and the clamp is only the extreme case of that. §4 bounds c to $[1/64, 64]$ and argues both clamps are real board states rather than defensive padding. They are. But the clamp is not the whole phenomenon, and believing it is would send the next reader after the wrong fix.

`calibrate -depth 178` shrinks the 2025 boards to roughly the size of the draft, and **log-loss then regresses under the tilt on both of them**: $0.1778 \to 0.2118$ on 2025-a and $0.1914 \to 0.1923$ on 2025-b. On 2025-a, five of 180 vantages hit $c =$ `TiltCMax` — no c exists, the solve pins at 64, and every survival on that vantage is raised to the 64th power, i.e. crushed toward zero — and that is where most of the large move comes from. **On 2025-b, zero of** 168 **vantages clamp**, and log-loss regresses anyway.

The shared mechanism is the target. N counts every pick in the window, but the board only holds players some ADP source ranked; truncation raises the share of picks spent on names nobody priced, so 12

picks remove only ≈ 10.7 board players and the tilt over-corrects on *every* vantage, not just the clamped ones. Mean predicted survival is pulled below the observed rate as a result. The clamp is that same error at the point where it becomes unbounded.

Two consequences. First, falling back to $c = 1$ on a clamped vantage — which is still more honest than falling back to 64, and which nothing does today — would fix 2025-a's large move and would not touch 2025-b's at all. Second, the regime matters: on the folds' own untruncated boards no vantage clamps in 516 ($168 + 180 + 168$), but the shipped 2026 board is 201 FFC names against a 192-pick draft, and the live pool is thinner still in effective mass because registered-but-unranked players sit in it at $p = 1$ contributing nothing to the sum. Nothing in the code warns about either.

8. Need weights are unmeasured. 1.0/0.6/0.25 and the endgame slack of 0.5 are judgement, not measurement. Nothing in the backtest touches them, because the backtest grades survival and need does not enter survival. They could be materially wrong without any of the numbers in this paper moving.

9. Per-player sigma is carried on principle, not on evidence. See point 3 of §14: on this backtest a flat $\sigma = 6.0$ with the tilt scores 0.0671/0.2233 against the shrunk per-player stack's 0.0670/0.2250 — a tie at best, and untilted the flat model wins outright. It got no better with the second and third folds, which by construction cannot help: they have no per-player spread at all. Worse, the shipped stack with every σ forced flat beats the shipped stack itself on 2024 log-loss (0.2233 against 0.2250, Table 11 and Table 15), and the room warp — which used to be the thing that broke the tie in per-player sigma's favour — can no longer be scored on that fold at all (threat 1b). So the one metric on which the full stack used to clear flat+tilt no longer separates them either. This is not buried: `calibrate` appends "beats what ships on log-loss" to that row itself, so the table cannot be read without meeting the comparison. §15.3 points the same way from a second direction: the tilted tuning grid's optimum is a configuration that pins 154 of 178 players to one σ. If a second scorable season arrives and per-player sigma still fails to beat a constant, the honest response is to delete a lot of §3.3 rather than defend it. What keeps it in the meantime is that a single scale cannot be right for both a $\varsigma = 0.4$ receiver and a $\varsigma = 39.2$ kicker, and being wrong about those two is the whole job.

10. The two-pick plan's first leg is knowingly optimistic. §8 prices the first leg at today's best available even when the pick is seventeen away. The comparison between pairs survives it; the absolute score does not, which is why the score is not displayed.

11. `TierMaxSize` does not reach derived tiers. §7 states the scope and this is the consequence: `subdivide` runs only on blocks a rankings file drew, so on a rankings-free board nothing caps a value-gap tier's size. Today that leaves an 11-man RB tier 2 near the top of the board and a 29-man WR tier 9 at the bottom, and (29) over eleven or twenty-nine players is never going to fall below `TierHoldWarn`. The cost is silence, not a false alarm — those groups simply never raise a cliff — and it is invisible in every number in §14, because the backtest grades survival and tiers do not enter survival. It is a real gap all the same, and the fix is one call to `subdivide` on `deriveBlocks`' output.

17 Constants

Every tuneable number, its value, where it lives, and the argument behind it. The engine's constants are in `internal/engine/tuning.go`; the fetch-time duplicates are in `internal/adp`, and that duplication is deliberate — the `adp` package precomputes sigma and tiers without importing the engine.

constant	value	where it comes from
Survival curve — `engine/tuning.go`		
`SigmaDefault`	6.0	Median ς across the 2026 half-PPR pool is 10.3; $10.3/1.8138 = 5.68$, so 6.0 is within six percent of the median player's own scale. Well calibrated there, wrong at both tails — which is why it is only the fallback. On this board it never fires at all: FFC reports a spread for every row.
`SigmaFromStdev`	1.8138	$\pi/\sqrt{3}$, exactly. Proposition 3.2. Not tuneable; it is a theorem.
`SigmaMin`	0.5	Judgement. A per-pick hazard of 86% is already "he is gone"; below this the number stops meaning anything. Binds on 3 of 203 players. The `-tune` grid wanted 14 here before the tilt and 8 after it; §15.3 is why it got neither.
`SigmaMax`	25.0	Judgement. $\varsigma \geq 45.3$ maps here; a player the market disagrees about by more than three rounds is already "no idea". Binds on *nobody* on the 2026 board, whose widest spread is $\varsigma = 39.2$.

constant	value	where it comes from
`SurviveThreshold`	0.5	The "safe to wait" cutoff — *display only* since urgency became continuous. No longer any part of the maths.
`SurvivalExpClamp`	30.0	$\log(1+e^{30}) - 30 = 9.4 \times 10^{-14}$, so the linear branch of softplus is exact to 10^{-14}. See §3.6 for why this must never be applied to S itself.
`UndraftedADP`	999.0	Off the drafted radar. Live feeds register handcuffs and rookies no ADP source ranked.
`FallerSigmas`	1.0	Judgement. One of the player's *own* scale units past his price. Measuring in his own σ is the load-bearing part; the 1.0 is taste.
`RuleOfThree`	3.0	$-\log(0.05) = 2.996$. §15.2. Currently unreachable — `SupportFloor` is not wired in.
*The tilt — **engine/tuning.go***		
`TiltCMax`	64.0	Bracket $[1/64, 64]$ for the bisection of Theorem 4.3. On real boards c solves to 0.29–1.12 and neither clamp ever fires: 0 of 168 backtest vantages, 0 of 1,969 mock frames. Wide enough to be irrelevant, which is the correct state for a clamp.
`TiltTol`	10^{-6}	Bracket width at termination; ≈ 26 halvings.
*Expected best / tiers — **engine/tuning.go***		
`EBestEpsilon`	10^{-6}	Truncation of (25); omits at most $\varepsilon v_1 \approx 0.01$ value points on the shipped board. Proved in §5.
`TierHoldWarn`	0.5	Judgement. Amber when the tier is a coin flip to survive.
`TierHoldCliff`	0.15	Judgement. Red when it almost certainly will not.
`TierDropPct`	0.10	Relative value drop that breaks a derived tier. Produces recognisable tiers on the real board — with no rankings file, 7 QB, 6 RB, 9 WR and 4 TE tiers.
`TierFloorPct`	0.015	Absolute floor on that drop, as a fraction of the top value *of the block being derived* — the position's best when no rankings file is in play, the best uncovered player otherwise. Stops the flat tail shattering into singletons.
`TierMaxSize`	8	A published graphic put 21 receivers in one tier; a tier that large can never fire a cliff, so the board would never warn about receivers at all. Applies to rankings-file blocks only — see §16 item 11.
`TierMinSize`	2	Never manufacture a one-player tier — that is a permanent "last one" alarm, the exact failure that got FantasyCalc's global tiers rejected. Singletons a human drew are left alone.
*Need — **engine/tuning.go***		
`NeedStarter`	1.0	Judgement, unmeasured. See §16 item 8.
`NeedFlex`	0.6	Judgement, unmeasured.
`NeedBench`	0.25	Judgement, unmeasured.
`EndgameSlack`	0.5	Judgement. At $L = U + 1$ a bench flier is still affordable but spends the only spare pick. At $L = U$ the multiplier is 0, which is not tuneable — it is the arithmetic of having no spare picks.
`KDefLastRounds`	3	**Policy, not a model.** What we are willing to recommend.
`OpponentKDefLastRounds`	6	**Measurement.** This room takes its first kicker in round 10 and first defense in round 11. Used by engine v2's opponent model only. Two constants because they are two questions.
*Runs, byes, freshness — **engine/tuning.go***		
`RunWindow`	6	Judgement. Half a round of a 12-team draft.
`RunThreshold`	4	Judgement. Four of six is unmistakably a run.
`ByeConflictThreshold`	3	Two starters sharing a bye is unremarkable across nine slots; three is a week you cannot field.
`StaleADPHours`	24	FFC recomputes daily, so a board over a day old has definitionally missed a refresh.
`ADPWindowStaleDays`	2	The source's own window, not our fetch timestamp: a cache-only fetch moves the timestamp and nothing else.
`NewsFreshHours`	48	Long enough to survive a draft-morning fetch about Friday-night news; short enough not to decorate half the board.
`TripwireSlack`	2	`low` is a maximum over the drafts FFC actually saw that player in — 5 to 215 of them on the 2026 board, not the window's 1,187. One pick behind it is a rounding difference; two means the room is genuinely behind every draft in his own sample.
*Value fallback — **engine/tuning.go***		
`ValueBase`	250.0	Only used when no source values anybody: $v = 250e^{-\text{rank}/40}$. Never mixed with imported values — §11.3 shows the 24-fold scale mismatch that would cause.
`ValueDecay`	40.0	As above. Chosen to be convex, not fitted.
*Fetch-time — **internal/adp***		

constant	value	where it comes from
`ShrinkPseudoDrafts`	25.0	n_0 in (18). The 2024 pool's median player was taken in 54 drafts and the thinnest in 11 (2026: 49 and 5); at $n_0 = 25$ the median player's own number outvotes the prior 2:1 and a thin player lands most of the way onto it.
`RoomWarpPseudo`	2.0	$w = n/(n+2)$, so three drafts buy 60% of the warp and a depth nobody reached buys none. Deliberately timid.
`RoomGapTopK`	5	**Display only.** The warp itself is uncapped. This is the depth over which `fetch` averages the room-versus-market gap for its headline, because averaging over all k inverts the sign. A pricing cap `RoomWarpTopK` $= 5$ was chosen off a cutoff sweep on the 2024 fold ($K = 5$ Brier 0.0660, a plateau at 4–5), then removed: that sweep did not replicate on either 2025 fold, and 2024's curve came from drafts a year after it, so it no longer has a causal curve at all. On the two folds that do, every K — including no cap — is never worse than not warping, so the sweep excludes nothing. Held at 5 on the structural argument of §10.5 plus never-worse. Threat 1b.
`SigmaDefault`, `SigmaFromStdev`, `SigmaMin`, `SigmaMax`, `TierDropPct`, `TierFloorPct`, `TierMaxSize`, `TierMinSize`	—	Deliberate duplicates of the engine values, so `fetch` can precompute sigma and tiers without importing the engine. Keep them in sync by hand; there are tests.

The honest summary of this table: **exactly one constant was chosen by measurement** (`RoomGapTopK`), one is a theorem (`SigmaFromStdev`), several are numerical-analysis facts with proofs attached (`SurvivalExpClamp`, `EBestEpsilon`, `TiltTol`, `RuleOfThree`), a handful are grounded in a measured property of the data without being optimised against it (`SigmaDefault`, `ShrinkPseudoDrafts`, `OpponentKDefLastRounds`, `TierMaxSize`), and the rest are judgement. That distribution is worth seeing plainly. The parts of the engine that were graded are graded; the parts that were not are labelled.

18 From formula to code

This is the page to read with the repository open. Every numbered equation above, and every named idea, mapped to the Go function that implements it. Function names are as they appear in the source; lowercase names are unexported.

eq.	what it is	function	file
Snake arithmetic			
(1)	round of a pick	`State.Round`	`engine/state.go`
(2)	index within round	`State.IndexInRound`	`engine/state.go`
(3)	seat picking at p	`State.SlotAt`	`engine/state.go`
(4)	my pick in round r	`State.MyPick`, `State.slotPosition`	`engine/state.go`
(5)	q_1	`State.NextPick`	`engine/state.go`
(6)	q_2	`State.FollowingPick`	`engine/state.go`
(7)	N	`State.PicksUntilMine`	`engine/state.go`
—	my next n picks	`State.MyUpcomingPicks`	`engine/state.go`
—	feed cross-check of (3)	`State.ApplyRemote`	`engine/state.go`
Survival			
(10), (12)	$\varsigma \to \sigma$, clamped	`adp.Sigma`	`adp/aggregate.go`
(9), (13), (16)	conditional survival	`State.PSurviveAt`	`engine/urgency.go`
—	the same at horizon q_1	`State.PSurvive`	`engine/urgency.go`
—	$\log(1+e^x)$ with the clamp	`softplus`	`engine/urgency.go`
—	which ADP the curve reads	`Player.price`	`engine/urgency.go`
(17)	falling flag	`State.Falling`	`engine/urgency.go`
(19), (20)	OLS fit of ς on a	`adp.FitStdevPrior`, `StdevPrior.At`	`adp/aggregate.go`
(18)	variance shrinkage	`adp.Blend`, `StdevPrior.Shrink`	`adp/aggregate.go`
—	pool-wide shrink pass	`adp.ShrinkSigma`	`adp/aggregate.go`
(37)	rule-of-three floor (**not wired in**)	`engine.SupportFloor`	`engine/urgency.go`
The tilt			
(24)	$N(\text{at})$	`State.opponentPicksBefore`	`engine/expect.go`
(22)	solve $f(c) = N$ by bisection	`State.survivalTilt`	`engine/expect.go`
—	$\tilde{p}_j$ at horizon q_1	`State.PSurviveTilted`	`engine/expect.go`

eq.	what it is	function	file
—	backtest's second solver	`solveTilt`, `tilted`	`cmd/pick6/calibrate_score.go`
—	the two solvers agree	`tiltSelfCheck`	`cmd/pick6/calibrate_score.go`
Expected best survivor, urgency, need			
(25)	$\mathbb{E}[M_P]$	`State.EBest`, `State.ebest`	`engine/expect.go`
—	the sum itself, over slices	`expectedBest`	`engine/expect.go`
—	modal best survivor (display)	`State.BestLater`	`engine/urgency.go`
(26)	urgency	`State.Urgency`	`engine/urgency.go`
—	"safe to wait"	`State.SafeToWait`	`engine/urgency.go`
(27)	need weights	`State.Need`, `State.needFrom`	`engine/state.go`
—	need after a hypothetical pick	`State.NeedAfter`	`engine/state.go`
Prop. 6.5	greedy slot filling	`State.FilledSlots`, `State.fillSlots`	`engine/state.go`
—	K/DEF suppression	`State.Suppressed`	`engine/state.go`
(28)	endgame slack	`endgameSlack`, `State.MustFillStarters`	`engine/state.go`
—	board group ordering	`Board.bestAvailable`	`ui/board.go`
Tiers, cliffs, runs			
(29)	tier-hold probability	`State.TierHold`	`engine/detect.go`
—	which horizon it prices to	`State.TierHoldPick`	`engine/detect.go`
—	cliff level from the hold	`State.Cliff`	`engine/detect.go`
—	remaining / total in a tier	`State.TierRemaining`, `State.TierSize`	`engine/state.go`
—	run detection	`State.DetectRun`	`engine/detect.go`
—	tier assignment	`adp.AssignTiers`, `assignPositionTiers`	`adp/aggregate.go`
—	oversize-block subdivision	`subdivide`, `biggestDrop`	`adp/aggregate.go`
—	derived tiers, runt merge	`deriveBlocks`, `mergeRuntTiers`	`adp/aggregate.go`
Two-pick lookahead			
(30)	the pair score and argmax	`State.BestPlan`	`engine/plan.go`
—	rendering both legs by pick	`Board.planCopy`, `Board.planLine`	`ui/board.go`
Room warp			
(33)	build adp_{room}	`adp.BuildRoomCurve`	`adp/room.go`
—	held-out build for the gate	`adp.RoomCurveOf`	`adp/room.go`
—	picks → positions in order	`adp.RoomDraftOf`	`adp/room.go`
—	read one point of the curve	`RoomCurve.At`, `RoomCurve.Depth`	`adp/room.go`
(34)	the blend	`RoomCurve.Warp`	`adp/room.go`
—	whole board, full depth (**loses the gate**)	`RoomCurve.EffectiveADP`	`adp/room.go`
—	whole board, top K (**ships**)	`RoomCurve.EffectiveADPTopK`	`adp/room.go`
—	the three monotonicity guards	`RoomCurve.effectiveADP`	`adp/room.go`
—	apply it at board load	`roomWarp`, `loadBoard`	`cmd/pick6/mock.go`
—	read the cached drafts	`adp.CachedRoomDrafts`, `LoadRoomDrafts`	`adp/room.go`
Value over replacement, K/DEF			
(35)	vor	`State.VOR`	`engine/vor.go`
—	$v_{\text{rep}}(P)$	`State.Replacement`	`engine/vor.go`
—	$\text{dem}(P)$, and its lineup fallback	`adp.PositionDemand`, `State.demandAt`	`adp/demand.go`, `engine/vor.go`
§11.3	rank-anchored K/DEF values	`adp.AnchorKDefValues`	`adp/aggregate.go`
Backtest			
§13.3	the whole command	`runCalibrate`	`cmd/pick6/calibrate.go`
—	era board + exact-only join	`loadEraBoard`, `Index.LookupExact`	`cmd/pick6/calibrate.go`, `rankings/match.go`
—	vantages, predictions, labels	`walk`	`cmd/pick6/calibrate.go`
Thm. 13.5	Brier and log-loss	`scoreOf`, `clampP`	`cmd/pick6/calibrate_score.go`
Thm. 13.7	reliability tables	`reliability`	`cmd/pick6/calibrate_score.go`
—	horizon / position / depth cuts	`segments`, `segmentTable`, `depthSegs`	`cmd/pick6/calibrate_score.go`

eq.	what it is	function	file
—	the baselines	`modelConstant`, `modelFlatSigma`, `modelUnconditional`	`cmd/pick6/calibrate_score.go`
—	4b's two models	`modelShrunkSigma`, `modelSupportFloor`	`cmd/pick6/calibrate_score.go`
§15.2	how often the floor bites	`floorReach`	`cmd/pick6/calibrate_score.go`
§10.5	the 3a gate, per position	`roomGate`, `roomPosGate`, `shiftByPos`	`cmd/pick6/calibrate_room.go`
§15.3	the sigma grid search	`tuneSigma`, `sigmaModel`	`cmd/pick6/calibrate_tune.go`
—	sigma sweeps with parameters	`psurvive`, `sigmaOf`	`cmd/pick6/calibrate_score.go`

One row in that table is a deliberate duplication rather than an oversight: `solveTilt` in the backtest reimplements `State.survivalTilt`, for the reason given at the end of §13.3, and `tiltSelfCheck` is what keeps the two honest.

19 What is next

Two of the three things this section used to ask for have since happened, and are left here with their outcomes because a to-do list that quietly deletes its completed items also deletes the record of whether they were worth doing.

Re-gate when the archive grows — HAPPENED (2026-08-11). FFC's `year=2025` came alive: 156 players, 718 drafts, real spread. Both 2025 folds now score on FFC boards, per-player sigma runs on all three folds, and the cap of §10.5 stayed retracted on the reruns. The unadvertised consequence was a regime change: the 2025 boards are *thinner than their drafts* (156 names against 180–192 picks), which promoted the tilt's thin-board fragility from a diagnostic to the live scoring regime — and became one of the two measured arguments for the sim.

Opponent-aware survival (milestone 6) — SHIPPED (2026-08-11). §12. The prediction above held in an unexpected way: the headroom was indeed resolution, but most of what the rollouts initially bought was eaten by a bias no roster can see — every simulated pick removing a ranked player when real rooms spend half their endgame picks off-board. With the causal escape of §12.3, v2 beats what shipped on both metrics on both folds that can test it as it runs live, and it is the board's default.

Multi-pick lookahead (milestone 7) — SHIPPED (2026-08-11). §8.4. It took the *second* leg rather than the first, which is the one thing this section got wrong when it asked for it: the honest simplification named in §8 is about leg one's optimism, and that survives untouched, because every candidate still carries it equally and it is a ranking. What the rollouts replaced was the expectation — the second leg is now the player who actually landed, averaged over the futures. Two leftovers with their own promotion bars: the third leg (built, measured, off — see the depth-three note in §8.4) and the v2.1 levers named at the end of §12.

Two things that are deliberately *not* next: an ADP over/under-performance model (the blocker is labels — it needs several seasons of historical ADP joined to actual finish, and no source we have serves history), and any attempt to make projections of our own. The first is parked on a data problem. The second is a scope decision and it is not being revisited.

References

[1] G. W. Brier. *Verification of forecasts expressed in terms of probability.* Monthly Weather Review, 78(1):1–3, 1950. The original proposal of the mean squared error of a probability forecast as a verification score. Brier's own formulation was for multi-category forecasts and differs from the binary version used here by a factor of two; the binary $\frac{1}{N}\sum(q_i - y_i)^2$ is what everyone now means by "Brier score".

[2] A. H. Murphy. *A new vector partition of the probability score.* Journal of Applied Meteorology, 12(4):595–600, 1973. The three-way decomposition of the Brier score into reliability, resolution and uncertainty, used here as Theorem 13.7. The decomposition is exact when forecasts take finitely many distinct values and bins group identical forecasts; binning continuous forecasts, as we do, leaves a within-bin variance residual.

[3] J. A. Hanley and A. Lippman-Hanley. *If nothing goes wrong, is everything all right? Interpreting zero numerators.* Journal of the American Medical Association, 249(13):1743–1745, 1983. The "rule of three": with zero events observed in n trials, an approximate upper 95% confidence bound on the underlying rate is $3/n$. The short derivation is reproduced in §15.2. The result is older than this paper and is often attributed more diffusely; this is the reference that made it standard practice.

[4] B. Efron and C. Morris. *Data analysis using Stein's estimator and its generalizations.* Journal of the American Statistical Association, 70(350):311–319, 1975. The practical case for shrinkage: many noisy estimates pulled toward a common centre beat the individual raw estimates, even though each raw estimate is unbiased. Their famous worked example is batting averages, which makes it the right citation for a sports paper. §3.8 applies the idea to per-player draft-position spread, with the prior fitted from the same pool — "empirical Bayes".

[5] N. Balakrishnan (ed.). *Handbook of the Logistic Distribution.* Marcel Dekker, 1992. Standard reference for the logistic distribution: the moment generating function $\Gamma(1-t)\Gamma(1+t) = \pi t/\sin \pi t$ used in Proposition 3.2, the variance $\pi^2 s^2/3$, and the excess kurtosis of $6/5$ quoted in §3. Any standard text on continuous univariate distributions carries the same results; the MGF identity itself is Euler's reflection formula.

[6] T. Gneiting and A. E. Raftery. *Strictly proper scoring rules, prediction, and estimation.* Journal of the American Statistical Association, 102(477):359–378, 2007. The modern reference on propriety. Theorem 13.5 proves the two special cases this project uses directly rather than invoking the general theory, but this is where the general theory lives, including why the Brier and logarithmic scores are essentially the only two anyone should reach for first.

[7] D. R. Cox. *Regression models and life-tables.* Journal of the Royal Statistical Society, Series B, 34(2):187–220, 1972. The origin of the proportional-hazards idea used in §4: multiplying every subject's hazard by a common factor. Cox's setting is covariate regression on survival times and the machinery there is far heavier than anything here; what this project borrows is the single structural observation that scaling cumulative hazard by c is raising survival to the power c, and that this preserves ordering and both fixed points.